

从已验证能力到合法派生

面向 Agent 的预研开发形式方法

(教材式草案 · 持续修订)

ascendc 工程 · 专题分支 `research/formal-lang-dag`

摘要

愿景：把「层层递进、先验证底层再拼高层」的工程直觉，提升为一套可教学、可门禁、可对照实验的形式方法——使 Agent（以及人类协作者）在复杂预研任务中，**只沿已验证能力及其合法组合**生长实现，从机制上压缩跳层发明与幻觉空间；最终服务一般预研开发，而非仅服务某一密码标准的算子清单。

写法：仿教材而非论文——先从特例讲故事，再抽象数学对象，再复盘与前瞻；证明与完备性后置，允许边做边补。**地位：**`docs/research/` 调研草稿，非定稿、非交付验收依据。

压力测试床：ML-KEM-1024 链上的 `pke.keygen / kem.keygen / kem.encaps`，并以 `kem.decaps` 为理论前瞻试金石；`*-correctness-*` 作为「只求正确、不计优化」的对照组标本。另用真 **ML-KEM-768** ($k = 3$) 做参数组迁移复盘（第 8 章）：检验「先锁参数卡再按波次补洞」能否走到 `incubating`，而不是把 1024 零垫改名。

目录

1	愿景、读者与本文怎么读	2
1.1	愿景（一句话）	2
1.2	我们希望最终得到什么	2
1.3	研究路线：从特殊到一般	2
1.4	阅读地图（教材顺序）	2
2	一个完整例子：我们怎样做出 ML-KEM-1024 的 KEM.KeyGen	3
2.1	我们要交付什么	3
2.2	FIPS 203 原文：三层算法怎样套在一起	3
2.3	动手之前，我们手里已经有什么	5
2.4	我们怎样探出来：工程路径与数据依赖	6
2.5	从这个例子出发，我们应当问什么？	8
2.6	本工程现在能回答这四组问题吗？	9
3	方法论所需的数学背景	10
3.1	导论：引子、问题实质、方法实质与工具落点	10
3.1.1	引子：correctness 有了，任务就算完成了吗？	10
3.1.2	问题的实质：开放生成被当成了合法派生	11

3.1.3	方法的实质：要将无界生成问题转化为已认证闭包上的受限规划搜索，以已认证资产为世界模型	11
3.1.4	我们已经收敛出的架构与流程（直觉图像）	11
3.1.5	本章为何要动用数学工具	12
3.1.6	方法论涉及哪些数学工具，落在工程的哪一面	13
3.1.7	从场景回到可检查的问题（备忘）	14
3.1.8	最小集合、必须掌握与本章边界	15
3.1.9	本章主线一览（读图后再进数学）	16
3.2	依赖图与依赖闭包	16
3.2.1	集合、笛卡尔积、关系与函数	17
3.2.2	有向图、路与环	18
3.2.3	有向无环图与拓扑序	19
3.2.4	闭包：从一般概念到依赖闭包与缺项	20
3.3	形式语言与合法拼装	29
3.3.1	字母表、字与语言	29
3.3.2	文法、产生式、派生与派生树	30
3.3.3	成员判定、禁字与组合不免费	31
3.4	作业过程自动机	32
3.4.1	迁移系统、配置与接受	32
3.4.2	证书的非单调更新（脏标记）	33
3.5	工程体现：三件工具在仓库里落成什么	33
3.5.1	依赖图与依赖闭包 → 闭包表、资产清单与 <code>frozen/</code>	34
3.5.2	形式语言 → 拼装计划、门禁与禁止拼法	34
3.5.3	作业过程自动机 → 作业怎么走完、证书何时作废	34
3.6	回扣第 2 章：流程如何对应本章数学	34
3.7	枢纽：依赖、展开与进度各由哪件工具管	38
3.8	本章小结与下一章预告	40
4	从数学到工程对象：节点、边与证书	40
4.1	推导用到的第 3 章工具箱	40
4.2	由顶点导出节点卡	41
4.3	由边关系导出三类边	43
4.4	由闭包定义导出缺项算法	44
4.5	由非单调证书导出脏标记传播	45
4.6	由投影关系导出「菜单 ≠ 说明书」	46
4.7	本章小结：本章推出了什么	46
5	门禁与工作流：Agent 被允许做什么	46
5.1	闭包表算法	46
5.2	门禁判定与最小补洞	48
5.3	四步工作流 = 外层算法	49

5.4	合法性规则 (v0) 清单	50
5.5	文书分工 (由算法接口推出)	50
5.6	反面教材: 放飞=算法前置条件失败	50
5.7	域无关性	51
5.8	下一章预告	51
5.9	本章小结: 本章推出了什么	51
6	复盘: 理论如何对照 kem.keygen 与 kem.encaps	51
6.1	复盘的目的与方法	51
6.2	事后闭包表: kem.keygen	52
6.3	事后闭包表: kem.encaps	53
6.4	三类边在仓库里的实例	54
6.5	具象化对照表 (理论 $\mapsto$ 工程)	54
6.6	回看第 2 章四组问题 (复盘后的判断)	54
6.7	分析结论	55
6.8	本章小结	55
7	前瞻: 用理论指导 kem.decaps (试金石)	55
7.1	为什么拿 Decaps 当试金石	56
7.2	先把任务链理清: 目标、依赖、缺什么	56
7.3	写码前先交闭包表 CT_{decaps}	57
7.4	三种边, 以及我们实际补洞的顺序	58
7.5	文书和代码是怎么一步步落下来的	58
7.6	测了什么, 结果怎么读	59
7.7	和人工优化出来的交付树 (C) 比一比	60
7.8	怎样算成功, 本章怎么判	61
7.9	方法论到底有没有用: 说人话的总结	62
7.10	本章小结	62
8	套别例: ML-KEM-768 从分析到 incubating 的完整复盘	62
8.1	为什么拿 768 当第二块试金石	63
8.2	怎么分析: 相对 1024, 洞在哪里	63
8.3	怎么计划: 参数卡、决议与波次	64
8.4	怎么实现: 按波次落码	65
8.5	怎么测, 以及测挂了怎么改	66
8.6	结果如何	66
8.7	完整复盘: 什么有效, 什么仍欠	67
8.8	题外话: 配额与墙钟上的体感	68
8.9	本章小结	69

9 对照组：correctness 标本的意义	69
9.1 为何保留 correctness	69
9.2 在方法论里扮演什么角色	69
9.3 建议比较的几条轴	69
10 未决议事项、日程与维护	70
10.1 刻意先不拍板的事	70
10.2 近期填空顺序（仍然是「先特殊、后一般」）	70
10.3 文档与分支约定	70
10.4 编译	70

1 愿景、读者与本文怎么读

1.1 愿景（一句话）

我们想表达的是：把复杂实现/预研交给 AI Agent 时，**不应当**把「输出对拍过」当成任务完成，更不应当让它在开放文本空间里自由寻找答案；而应把开放生成收成「沿已验证能力的可检查闭包做受控搜索」——少发散、少无效派生，把项目真正做完。静态依赖图只是「谁依赖谁」的投影；真正的作业过程还要另说。

1.2 我们希望最终得到什么

1. 一套**教材式**叙述：新人 / 新 Agent 能从特例读懂「图从哪来、树怎么长」；
2. 一套**可操作门禁**：任务启动必须先交依赖闭包与缺项，再写码；
3. 一套**对照实验协议**：方法论指导下的实现 vs correctness 标本 vs 人工指导路径；
4. （成型后）可解释的接受/拒绝轨迹，以及必要的证明——**不作为**本稿前置条件。

1.3 研究路线：从特殊到一般

阶段	做什么	刻意后置
特例引出	工程故事 → 图与语法树	完备公式
套公式	用数学 复述 已见现象	证明
改公式	缺口反过来改理论	为漂亮删工程事实
套别例	Encaps 复盘（第 6 章）；Decaps 前瞻 + 判决（第 7 章）；768 参数组迁移（第 8 章）	一次塞太多缺口
迭代方法论	门禁与流程收敛	急进 Rule/Skill
成型后	证明、可解释性	过早锁死抽象

1.4 阅读地图（教材顺序）

1. 第 2 章：**完整例子**（KEM.KeyGen：算法原文、我们有什么、怎样做出来、引出课题）；
2. 第 3 章：**方法论数学背景**（§3.1 导论；§3.2–§3.4 各用一大节写一件工具的数学；再工程体现、回扣、枢纽与总结——**不含密码/NTT 代数**）；
3. 第 4–5 章：**装进工程对象与门禁**（用第 3 章数学对象作算法参数，逐步推出节点/边/证书与写码许可判定）；
4. 第 6–7 章：**复盘与前瞻**（用理论看实现）；
5. 第 8 章：**ML-KEM-768 完整复盘**（分析 → 计划 → 实现/测试/修改 → 结果）；
6. 第 9 章：**correctness 对照组**；
7. 第 10 章：未决议事项与维护约定。

讨论过程的流水账见同目录 形式语言与自动机预研讨论纪要.md (单文件持续刷新)。

2 一个完整例子：我们怎样做出 ML-KEM-1024 的 KEM.KeyGen

本章用一个**已经做完**的真实任务当例子。我们先说清楚「要算什么」，再贴 FIPS 203 里的算法原文，然后交代我们手里已经有什么、前期做过哪些实验、最后怎样拼出 KEM.KeyGen。例子讲完之后，我们再**提问**——这些问题应当能把读者自然带到后文的研究课题里。

2.1 我们要交付什么

我们的目标是：在 Ascend AI Core 上用 AscendC 实现 **FIPS 203 ML-KEM-1024 的密钥生成** (ML-KEM.KeyGen, 即 Algorithm 19)。

读者可以把交付物理解成下面这张黑盒表 (细节以 stable 目录 customs spec 为准)：

方向	内容	说明
输入	seed_d.bin (4B)	Host 写入；设备侧派生 d 、 z
输入	NTT LUT (静态)	与 PKE KeyGen 同表
输出	ek_kem.bin (1568 B)	与 ek_PKE 相同
输出	dk_kem.bin (3168 B)	与 liboqs 展开布局一致

我们不把 d 、 z 、中间 $\hat{A}$ 、 $\hat{s}$ 等当作交付文件落盘。验收方式是：run.sh 跑 CPU 孪生 + SIM，输出与 golden / liboqs 对拍一致。

2.2 FIPS 203 原文：三层算法怎样套在一起

标准把 KEM 密钥生成拆成三层。下面给出与 **FIPS 203 原文一致**的参数、Input/Output 与步骤编号。正文引用 NIST FIPS 203 Algorithm 13 / 16 / 19；本文实例固定为 ml_kem_1024 ($k=4$ 时 $|ek|=1568\text{ B}$ ， $|dk|=3168\text{ B}$)。

Algorithm 19 ML-KEM.KeyGen() (FIPS 203 §7.1)

参数集 (ml_kem_1024)： $n=256$ ， $q=3329$ ， $k=4$ ， $\eta_1=\eta_2=2$ ， $d_u=11$ ， $d_v=5$ 。

Input：(无显式输入； d 、 z 在算法内由 RBG 采样，见步骤 1-2。)

Output：封装密钥 $ek \in \mathbb{B}^{384k+32}$ ；

解封密钥 $dk \in \mathbb{B}^{768k+96}$ 。

1: $d \leftarrow \$\mathbb{B}^{32}$

2: $z \leftarrow \$\mathbb{B}^{32}$

3: if $d = \text{NULL}$ or $z = \text{NULL}$ then

4: return $\perp$

RBG 失败指示

5: end if

6: $(ek, dk) \leftarrow \text{ML-KEM.KeyGen_internal}(d, z)$

7: return (ek, dk)

Algorithm 16 $ML\text{-}KEM.KeyGen_internal(d, z)$ (FIPS 203 §6.1)

参数集 (ml_kem_1024): $n=256$, $q=3329$, $k=4$, $\eta_1=\eta_2=2$, $d_u=11$, $d_v=5$ 。

Input: 随机种子 $d \in \mathbb{B}^{32}$; 随机种子 $z \in \mathbb{B}^{32}$ 。

Output: 封装密钥 $ek \in \mathbb{B}^{384k+32}$;

解封密钥 $dk \in \mathbb{B}^{768k+96}$ 。

- 1: $(ek_{PKE}, dk_{PKE}) \leftarrow K\text{-}PKE.KeyGen(d)$
- 2: $ek \leftarrow ek_{PKE}$
- 3: $dk \leftarrow dk_{PKE} || ek || H(ek) || z$
- 4: **return** (ek, dk)

其中 $H = \text{SHA3-256}$ (FIPS 203 记号 H)。

Algorithm 13 $K\text{-}PKE.KeyGen(d)$ (FIPS 203 §5.1)

参数集 (ml_kem_1024): $n=256$, $q=3329$, $k=4$, $\eta_1=\eta_2=2$, $d_u=11$, $d_v=5$ 。

Input: 随机种子 $d \in \mathbb{B}^{32}$ 。

Output: 加密密钥 $ek_{PKE} \in \mathbb{B}^{384k+32}$;

解密密钥 $dk_{PKE} \in \mathbb{B}^{384k}$ 。

- 1: $(\rho, \sigma) \leftarrow G(d || k)$ 域分离: 字节 33 为模块维 k
- 2: $N \leftarrow 0$
- 3: **for** $i \leftarrow 0$ **to** $k - 1$ **do**
- 4: **for** $j \leftarrow 0$ **to** $k - 1$ **do**
- 5: $\hat{A}[i, j] \leftarrow \text{SampleNTT}(\rho || j || i)$
- 6: **end for**
- 7: **end for**
- 8: **for** $i \leftarrow 0$ **to** $k - 1$ **do**
- 9: $s[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N));$ $N \leftarrow N + 1$
- 10: **end for**
- 11: **for** $i \leftarrow 0$ **to** $k - 1$ **do**
- 12: $e[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N));$ $N \leftarrow N + 1$
- 13: **end for**
- 14: $\hat{s} \leftarrow \text{NTT}(s)$
- 15: $\hat{e} \leftarrow \text{NTT}(e)$
- 16: $\hat{t} \leftarrow \hat{A} \circ \hat{s} + \hat{e}$ NTT 域矩阵-向量乘加
- 17: $ek_{PKE} \leftarrow \text{ByteEncode}_{12}(\hat{t}) || \rho$
- 18: $dk_{PKE} \leftarrow \text{ByteEncode}_{12}(\hat{s})$
- 19: **return** (ek_{PKE}, dk_{PKE})

读者只需抓住一条结构链: $\text{Alg.19} \Rightarrow \text{Alg.16} \Rightarrow \text{Alg.13}$ 。Alg.19 比 Alg.13 多出来的主要是步骤

1-2 的 z 、Alg.16 步骤 3 的拼接——并不是把 NTT 或矩阵乘从头再写一遍。

本仓工程差异（验收已锁定，写在算法旁便于对照）

- 我们可复现跑分时，Host 只提供 4B `seed_d`； d 与 z 在设备 UB 内派生（对应 Alg.19 步骤 1-2 的确定性扩展）。
- 我们对拍 liboqs 时， dk 为 3168B 展开式 $dk_pke\|ek\|H\|z$ ，与 FIPS 代数最小形态一致，但字节布局与偏移以 `customspec` 为准。

2.3 动手之前，我们手里已经有什么

到做 KEM.KeyGen 之前，本仓库并不是从一张白纸开始。我们已经在 AscendC 上做过大量单点与分段实验，并且多数有 CPU+SIM 对拍记录。下面这张**资产清单**（表 2.1）把「当时手里有什么」写清楚；文字说明与表格对照阅读。

四类家底（文字）

1. **平台层 (P0)**：我们验证过 DataCopy、TPipe/TQue、MIX 核同步、MMAD tiling 等。
2. **密码原语层 (P1)**：我们验证过 NTT (poly-batch)、basemul、SHAKE/SHA3、CBD、ByteEncode₁₂、SampleNTT 等。
3. **PKE 全链 (P2)**：我们已有交付级 stable PKE KeyGen (2 launch, KAT 通过)。
4. **对照标本**：我们保留 `*-correctness-*`，只求可执行与字节正确，不计优化。

表 2.1: 资产清单（锚点以活跃 INDEX 为准）

层	能力（简述）	仓库锚点（示例）	证书	备注
P0	DataCopy / 搬运	docs/notes/ascendc-DataCopy...	文档 + 探针	平台知识库
P0	MMAD + tiling	pass-int8-matmul-cube-... 等	CPU+SIM	闭合条件见 notes
P0	TPipe / 细同步	KeyGen prep 技术总结	CPU+SIM	翻车高发区
P1	NTT 三段式 poly-batch	pass-fix-...-vec-k4-v2	CPU+SIM	ML-KEM 活跃基线
P1	SampleNTT / Alg.7	pass-fix-f203-alg7-sample-ntt-k4	CPU+SIM	
P1	CBD $\eta=2$	pass-fix-f203-alg8-cbd-eta2-k4	CPU+SIM	
P1	ByteEncode ₁₂	pass-fix-...-byteencode12-vec-k4	CPU+SIM	
P2	Alg.13 行 3-7 $\hat{A}$	pass-fix-f203-alg13-lines3-7-a-hat-k4	CPU+SIM	
P2	Alg.13 行 8-15 s, e	pass-fix-f203-alg13-lines8-15-s-e-k4	CPU+SIM	
P2	Alg.13 device 全链	pass-fix-f203-alg13-device-keygen-k4	CPU+SIM	≈542k tick
P2	stable PKE KeyGen	stable-fips203-mlkem-pke-keygen-k4	交付 +KAT	KEM 直接依赖
P3	KEM KeyGen correctness	fix-f203-alg19-kem-keygen-correctness-k4	CPU+SIM+liboqs	对照标本
P3	KEM KeyGen device	pass-fix-f203-alg19-kem-keygen-device-k4	CPU+SIM	≈713k tick
P3	stable KEM KeyGen	stable-fips203-mlkem-kem-keygen-k4	交付 +KAT	本例终点

当我们决定做 KEM.KeyGen 时，我们并不是让 Agent 从 FIPS 伪代码一行行翻译成 AscendC，而是心里已经有上表这张「哪些积木是绿的」的清单。

2.4 我们怎样探出来：工程路径与数据依赖

这张清单不是一天长出来的。本节用两张图把同一件事说清楚：图 2.1 是仓库时间线（先长 PKE，再接 KEM 尾缝）；图 2.2 是 FIPS 调用嵌套（19 调 16、16 调 13）与数据依赖（与 §2.2 步骤编号对照）。

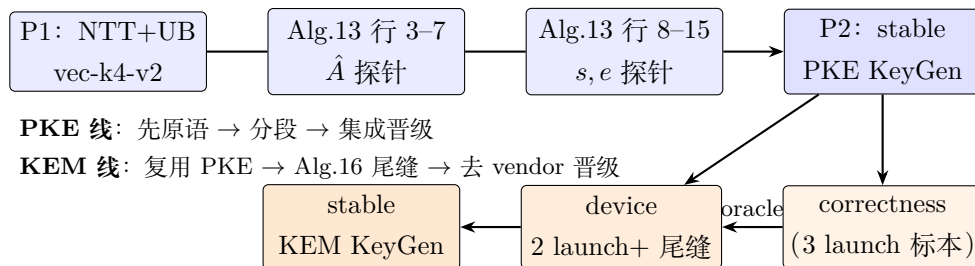


图 2.1: 工程探路：PKE 线到 KEM 线（仓库时间线）

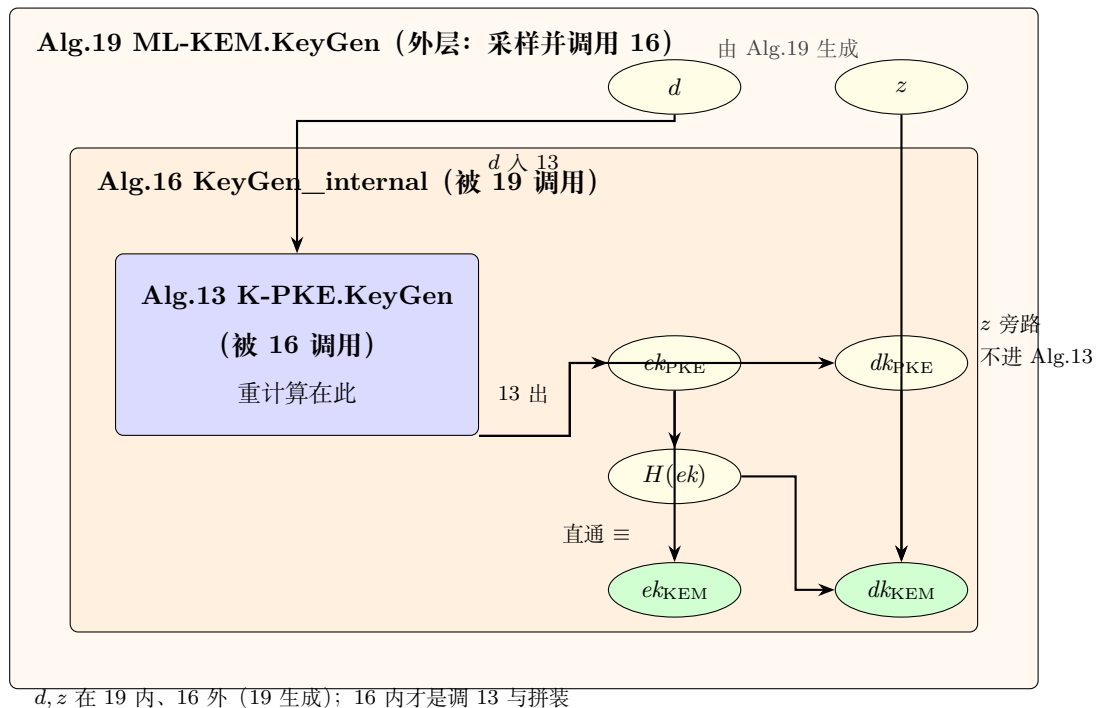
- **PKE 线**：我们先把 NTT、内积等做成向量探针；再把 Alg.13 拆成 $\hat{A}$ 、 s, e 、device 全链，逐段 PASS；最后集成晋级 stable。
- **KEM 线**：我们语义上调用 Alg.13（步骤 1 即 K-PKE.KeyGen）；设备上在 Launch-2 尾部追加 Alg.16 步骤 2-3（ H 、拼 dk ）；编排上保持 2-launch；验收上 correctness 作 oracle，device 去 vendor 后晋级 stable。

表 2.2: 设备侧 Launch 与 FIPS 行的对应（本仓实现）

Launch	设备段	FIPS 行	要点
L1	f203_keygen_prep	Alg.13 步骤 1-15	双 AIV; $\hat{A}$ 、 $\hat{s}$ 、 $\hat{e}$
L2	mmad_custom + 尾缝	Alg.13 步骤 16-21	NTT、内积、ByteEncode
L2 尾	KemKgTailFused	Alg.16 步骤 2-3	z 、 $H(ek)$ 、写 dk

若用一句话概括这次成功：我们把 FIPS 的三层算法，映射成「已证 PKE 链 + 一条单独想清楚的 KEM 尾缝」，而不是在 KEM 任务里重写 NTT。

图 2.1 讲「仓库里怎样长出来」；图 2.2 只看标准调用嵌套与数据——FIPS 里是 Alg.19 调用 Alg.16，Alg.16 再调用 Alg.13（不是平铺的 $19 \rightarrow 13 \rightarrow 16$ ）；重计算在 Alg.13， z 不进 Alg.13，只进 Alg.16 的 dk 拼接； ek_{KEM} 对 ek_{PKE} 是直通。

图 2.2: FIPS 调用嵌套与数据依赖 (Alg.19 调 16, 16 调 13; $d/z/ek/dk$)

读图时抓住三点：

1. **调用嵌套**：Alg.19 调 Alg.16；Alg.16 调 Alg.13。框套框表示「谁调用谁」，不是平铺的 $19 \rightarrow 13 \rightarrow 16$ 。

2. **数据**：椭圆是字节对象； d 进 Alg.13， z 旁路只进 Alg.16 拼 dk ； $ek_{\text{KEM}} \equiv ek_{\text{PKE}}$ 直通，不重做 NTT。
3. **汇点而非树**：NTT、CBD、MMAD 等还不画进本图——它们也不专属于 KeyGen；本图只是更大图里的一个汇点。

2.5 从这个例子出发，我们应当问什么？

例子讲完，真正重要的是**问题**。如果我们的研究课题是「怎样让 Agent 预研少幻觉、少撞墙」，那么面对 KEM.KeyGen 这个故事，我们至少应当认真问下面四组问题。

第一组：标准与工程之间的缝隙

1. FIPS 只告诉我们**算什么**；它并没有告诉我们，在 Ascend 上应当**先验哪儿块、按什么顺序拼**。我们凭什么在动手前就说「可以直接站在 stable PKE 上」——凭的是目录名，还是某种可写的证书？
2. 探针列表 (INDEX) 像一张「能力菜单」。菜单本身是否等于**合法拼装说明书**？若不等，缺的那一页应当长什么样？

第二组：「单点都对」之后，组合还对吗

3. 当 NTT 探针 PASS、PKE KeyGen PASS 之后，**谁来保证「2-launch 接缝」「Alg.16 尾拼 dk 」也一定 PASS**？我们能否把这种「组合不免费」写进理论，而不是每次靠工程师的经验？
4. 若我们把 correctness 路线（更多 launch、更保守同步）与 device/stable 路线（更少 launch、吃到优化）都称为「正确」，**我们究竟在验收什么**——仅仅是输出字节，还是也要验收「沿已证能力生长」？

第三组：Agent 若只知道算法和菜单，会发生什么

5. 若我们只给 Agent 一段 Alg.19 伪代码 + INDEX 里有哪些探针 PASS，**但不给确定的拼装路线**，搜索空间会有多大？我们亲眼见过：过程高度混乱、大量回退、极耗 token，最后只留下「能跑对」的 correctness 标本。
6. 我们能否把搜索空间收束为「只允许沿已证书边生长」，同时又**不把工程优化堵死**？收束的规则应当写在图里，还是写在某种「合法句子」里？

第四组：怎样把一次成功变成可复用的方法

7. 这次 KEM.KeyGen 的成功，能否被复述成一张**闭包表**：依赖谁、缺了谁、哪条边有证书、哪条边被禁止？若下一个人（或 Agent）做 Encaps / Decaps，我们能否**先交这张表再写码**？
8. 当上游改了 tiling 或同步壳，下游哪些节点应当自动标「脏」？我们是否需要一种比「记得上次 PASS」更可靠的失效传播？

2.6 本工程现在能回答这四组问题吗？

四组问题都很好，但「能回答」要分层次：工程实践里我们已有碎片，形式化方法论里我们尚未闭环。表 2.3 诚实对照（2026-07 专题讨论时的判断）。

表 2.3: 四组问题：本工程已能说明与仍缺项对照

问题组	本工程已能说明的	仍缺的	判断
第一组	INDEX、customspec、baseline-registry 已在约束引用来源	机器可读的「证书对象」；菜单 ≠ 拼装说明书的显式规则	部分能答
第二组	我们有 seam 探针/集成探针实践；correctness vs device 在 customspec 有对照	「组合不免费」未写成统一理论；验收口径仍以 I/O 为主	部分能答
第三组	correctness 历史是 反面教材 （高成本、不可控）	正向的「合法派生」门禁尚未工具化；Agent 仍可能绕开	能答负例；正例在建
第四组	单任务可手写 INTEGRATION_PLAN、闭包直觉	统一闭包表 schema；dirty 传播无自动机制	部分能答

逐组一句话

- **第一组**：我们能指着 stable PKE 说「KEM 应站在这里」，但还缺一张人人（含 Agent）都认的证书页。
- **第二组**：我们知道接缝要单独验（例如 2-launch、尾缝），但还没把这条经验收成可检查的规则。
- **第三组**：我们能用 correctness 证明「只给菜单会爆炸」；方法论要证明的是「给规则后能收敛」——这正是本专题要做的事。
- **第四组**：我们能复盘 KEM.KeyGen 的故事，但 Encaps/Decaps 还不能先交闭包表再写码作为硬门禁。

因此：本工程能支撑这四组问题的提出与部分实证，但不能宣称方法论已经成立。后文的工作，就是把表中「仍缺的」一列逐项补上，并用 Encaps / Decaps 做检验。

这些问题的共同指向 上面这些问题，表面上各不相同，但它们指向同一条研究主线：

我们能否把「已验证能力 + 合法组合 + 动态证书」整理成一套可教、可检查、可对照实验的形式方法——
让 Agent 的工作从「在菜单里瞎拼」变成「在自动机上走合法派生」？

后文先用一章方法论导论 + 数学底座（第 3 章）把工程策略说清并落到定义，再把这些对象装进工程语境并写成门禁（第 4-5 章），最后用 Encaps 复盘、Decaps 前瞻检验这套语言是否管用。本章与后文均不展开 PQC / 数论变换等密码代数——那些细节见 docs/notes/；本专题专注方法论。

3 方法论所需的数学背景

第 2 章讲完了一个已经做完的工程故事，并留下四组问题。本章按下列顺序写：

- § 3.1：导论——先讲 correctness 引子，再抽象问题实质与方法实质，然后交代第 2 章策略与三件数学工具落点；
- § 3.2–3.4：三件工具各一大节写数学（依赖图与闭包从集合讲起；形式语言；作业过程自动机）；
- 然后统一写工程体现、回扣第 2 章、枢纽对照，最后本章小结。

每个关键定义仍配双例：先纯数学小例子，再回到第 2 章的 KEM.KeyGen。本章不讨论格、MLWE、NTT 等密码代数。

3.1 导论：引子、问题实质、方法实质与工具落点

若一上来就讲集合与函数，读者容易以为本章只是在堆数学名词。导论按三条线展开：先讲一件真实发生过的事（correctness 引子），再抽象问题的实质与方法的实质，最后才落到架构直觉与三件数学工具。正式定义从 § 3.2.1 开始。

3.1.1 引子：correctness 有了，任务就算完成了吗？

故事发生在 Encaps / Decaps 这一刀。当时仓库里已经有一批对拍通过的底层能力——NTT、内积、hash 等用例大致齐了；Rule、Skill 与目录结构也相对清楚。任务提示写得很轻：大致是让 Agent 自行阅读 FIPS 203，在这些已有能力之上，把封装与解封装「做出来」。

从 correctness 看，Agent 确实交差了：输出能对上期望，像是做完了。把过程与形态摊开，却是另一幅图：

- **只保结果对**：工程路径极其简单粗暴，几乎不为 Ascend NPU 的架构与 AscendC 工具链特点做设计，性能差到不可用；
- **搜索空间几乎不设界**：AscendC 公开资料远少于成熟的 C/C++、Python，Agent 到处碰壁，反复试错与 debug，为了凑出 correctness，不惜采用极端低效的数据布局与任务调度；
- **成本失控**：短时间内耗尽约一个月的 token 配额；
- **还沉淀错东西**：过程中冒出并不正确的「经验」与结论，若不当场拆穿，会污染后续任务。

这件事把一种诱惑钉死了：**correctness 很容易被当成完成**。但若只盯结果对、又把找答案的空间交给 Agent 在开放文本里自由闯，得到的往往是伪完成——结果碰巧对，过程发散、形态不可用、钱与时间被烧光。

有了这层引子，才值得追问：问题的实质究竟是什么？该方法论又要把 Agent 收成怎样一种工作方式？

3.1.2 问题的实质：开放生成被当成了合法派生

大语言模型按概率在开放文本空间里续写，并不自带「只能站在已验证能力上、按允许的方式往上推」这条不变量。人在开放研究里也会发散；LLM 驱动的 Agent 只是把错误探索的**速度与代价**放大了。

引子里那些表象——可试的路像无穷多、乱试极贵、零件都绿整机却不可用——根子往往不在「项目管理缺一张表」，而在于：

执行者把开放生成误当成了合法派生：

菜单里有、局部测通、文本上看起来像解，就被当成「整体已经合法、可以交付」。

工程语境里的「幻觉」，多半不是文学式的胡编，而是**无效派生**：把语言里的非成员当成成员，把未获证节点当成可引用能力，把接缝与接口约定跳过去。模块拆分、单元测试、版本管理仍然必要，但它们管的是代码资产与回归，**不管「Agent 下一步还准许做什么」**。

所以真正要盯住的问题不是「Agent 会不会写出几段能正确对拍的 AscendC 代码」，而是：**如何阻止它把无界生成当作完成预研/实现的合法途径？**

3.1.3 方法的实质：要将无界生成问题转化为已认证闭包上的受限规划搜索，以已认证资产为世界模型

对策不是让 Agent 「优化代码」，而是换一种工作定义：

从「看起来合理就写」，转到「只沿已认证能力的可检查闭包推进」。

别把 Agent 当自由写手，要把它当**证书闭包上的搜索器**。

当前准许的下一步，由三类可检查对象界定（后文用数学钉名）：

1. **已验证能力**：现在还能引用谁；
2. **禁止项**：明确不能再当模板的路（含已关闭路线）；
3. **合法拼装规则**：怎样衔接才算一步合法推进；缺的先补齐并验过，再往上引用。

再加一条过程约束：作业状态本身可检查——走到哪算完成，下层约定一改，上层哪些「曾经通过」作废。

这里说的「有限」，精确含义是：**相对于当前能力集、禁止项与拼装规则，可接受的下一步是可枚举、可检查的**；并不是说研究空间永久冻结——集合可以随新能力获证而长大。

收益是有条件的：当组合深、错路贵时，维护证书与禁止项才划算——少烧无效 token，少交付「只有 correctness、没有工程形态」的伪完成。第 2 章 KeyGen 是收得住的对照；引子里的 Encaps / Decaps 放飞，则是反面教材。INDEX、frozen/、customspec、晋级复制、CPU+SIM 等，已是纪律雏形；本章要把「可检查派生」写成方法，而不是停留在口头约束。

3.1.4 我们已经收敛出的架构与流程（直觉图像）

进入公式之前，先用工程语言画一幅当前较可行的图像（细节后文用定义钉死）：

- **能力库**：一张「谁依赖谁、谁已获证」的图；活跃菜单是线索，不等于合法拼装说明书。

- **一次任务**：不是将依赖的项目整库拷贝，而是从目标出发展开依赖闭包，标出已有 / 缺项 / 禁止，再决定最小补洞。
- **接缝单独成事**：两块各自测通，拼在一起仍要单独验证；
- **节点状态要管**：下层接口约定一改，上层不能继续拿「上次测通」当仍然有效；
- **对拍与拼装路径分开看**：golden / liboqs / FIPS 管「算得对不对」；方法论管「允许怎么往上拼」。

第 2 章的 KeyGen 故事，大体就是沿着这幅图像走出来的；引子里的 Encaps / Decaps 放飞，则提醒我们：没有这幅图像约束时，correctness 仍可能「假完成」。后文再用受控路径做 Encaps / Decaps，是检验方法论能否前瞻，而不只是事后贴标签。

3.1.5 本章为何要动用数学工具

工程纪律可以写进 Rule，但 Rule 难教「为什么在这种场景下要这样约束」，也难在任务之间做**可对照的结构比较**。数学在这里的角色不是炫技，而是三件事：

1. **命名**：给「依赖、闭包、缺项、合法计划、接受/拒绝、脏证书」固定的名字与关系；
2. **分层**：把数据流、能力库、一次作业过程拆开，防止把「字节对了」误当成「派生合法」；
3. **可检查**：为后文门禁（先交闭包表再写码）和对照实验准备共同语言。

一句话：用数学工具，把「已有能力之上，下一步作业该怎么决策」说清楚；不是用方法论去推翻第 2 章的故事，也不是用公式去替代探针与对拍。

注 3.1 (导论用词纪律). 后文三件工具写数学名：**依赖图（及闭包）、形式语言、自动机**——与正式对象对齐。导论里「工程里怎么用」一列只用第 2 章已出现、或一看就懂的说法（菜单、拼装计划、已获证、缺项、闭包表、脏标记、frozen/ 等）。数学记号（如 Dep^* 、 Γ 、Missing、 L 、Dirty）**不在此处充当名称**；它们要到后文相应**定义**出现时才引入，并立刻写明「工程说法 $\leftrightarrow$ 数学记号」。表 3.1 是预告对照，便于往后查，**不是**要求现在就用符号阅读。

表 3.1: 工程说法 $\rightarrow$ 后文数学记号 (预告; 定义处才启用符号)

工程说法 (导论用语)	后文记号 (定义后)	一句话
依赖图	有向无环图 G	谁依赖谁、先证谁
依赖闭包	$\text{Dep}^*(\cdot)$	完成目标理论上必须齐的全部先决
已获证能力集	Γ	当前允许被引用的能力集合
缺项	$\text{Missing}_\Gamma(\cdot)$	依赖闭包里尚未获证的那部分
闭包表	(工作对象)	目标 / 已获证 / 依赖闭包 / 缺项的核对表
拼装计划 / 合法计划	字 w ; 语言 L	一次任务怎么拼; 全体合法拼法
禁止拼法	Forbidden	不得采用的子串或关闭路线
作业过程自动机	以配置为状态的迁移系统	第三件工具的专名
作业状态 (已获证、还欠什么)	配置 (Γ, F, S)	自动机的状态
展开 / 探针取证 / 写入证书	Expand/Probe/Certify	自动机上的转移
脏标记 (证书作废)	Dirty	上游变更后撤回下游证书

3.1.6 方法论涉及哪些数学工具，落在工程的哪一面

问题与方法实质说清之后，我们把方法论收成三件数学工具——分别管「站在谁身上」「怎样拼才合法」「这次作业走到哪」。一句话总览：

依赖图（及闭包）管「能站在谁身上 / 还缺谁」；形式语言管「怎样拼才算合法」；

自动机管「这次作业走到哪、证书何时作废」。

三者在仓库里分别落成**依赖图与闭包表 / 拼装说明书 / 门禁怎么往前走**；缺一，第 2 章里的失控会重新出现。

表 3.2 是导论核心对照；后文数学正文按这三件工具分段写细。

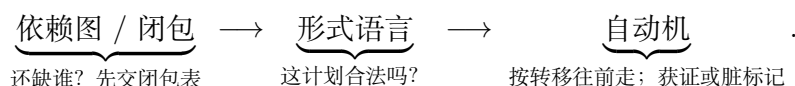
表 3.2: 依赖图 / 形式语言 / 自动机: 导论级对照 (为何学 $\leftrightarrow$ 工程用法)

工具 (数学)	为什么要学 (控制什么)	在工程里怎么用 (落点)
依赖图 (及依赖闭包)	预研不是白手翻译 FIPS, 而是「先有绿积木再往上长」。没有图就说不清: 谁依赖谁、先证谁、有没有环、动手前还缺谁。	顶点 $\approx$ INDEX/stable/接缝能力; 边 $\approx$ 「KEM 依赖 stable PKE」等; 拓扑序 $\approx$ 自底向上探路顺序; 依赖闭包与缺项 $\approx$ 资产清单与 闭包表 要核的两栏; 禁止再走的边 $\approx$ frozen/。
形式语言	INDEX 绿项是 菜单 , 不是 拼装说明书 。单点 PASS 推不出接缝合法 (组合不免费); 要把「合法拼法」写成可检查的计划。	可用步骤 $\approx$ 已证能力与接缝动作; 一次 拼装计划 $\approx$ 某次任务的展开顺序; 合法计划集合 $\approx$ 门禁承认的全部拼法; 成员检查 $\approx$ 门禁查「这份计划是否合法」; 禁止拼法 $\approx$ 已关闭/禁止路线 (含 frozen 判决来源)。
自动机	依赖图是静态库, 形式语言给出合法计划集合; 真正干活还要有 怎么一步步往前走 的规则: 补洞 $\rightarrow$ 探针 $\rightarrow$ 写入证书, 有时还要作废。没有自动机, 就无法说「走到接受」与「上次 PASS 为何作废」。	状态 $\approx$ 当前已获证哪些、前沿还欠什么; 转移 $\approx$ 展开 / 探针取证 / 写入证书; 接受 $\approx$ 目标已获证且前沿清空; 拒绝 / 冻结 $\approx$ 踩了禁止拼法; 脏标记 $\approx$ 上游约定变更后下游不得假绿。

例 3.1 (导论: 用 KeyGen 故事对照三件工具). 续第 2 章。

- **依赖图**: 不是「INDEX 里 NTT 绿了就直接写 KEM」, 而是先画清 stable PKE $\rightarrow$ 尾缝 $\rightarrow$ KEM, 再核对缺项。
- **形式语言**: NTT PASS、PKE PASS 推不出「2-launch + Alg.16 尾拼 dk 」自动合法——接缝须单独写进合法拼装计划。
- **自动机**: 从「已获证清单里已有 stable PKE」走到「kem.keygen 获证」是一条合法轨迹; 若底层 I/O/布局/同步约定变更, 上层应打**脏标记**, 不能因「上次 PASS」假绿。

三件工具如何串起来 一次受控作业, 导论级检查顺序就是:



学了不代替什么: 不代替「核怎么写、算得对不对」——那是能力建设本身的另一条知识栈; 本章三件工具只服务: 已有能力之上, 作业该怎么推进才合法、可控。

3.1.7 从场景回到可检查的问题 (备忘)

上面的场景, 落到检查清单上, 仍会变成若干可写的问题 (第 2 章四组问题是它们在 KeyGen 上的实例): 可信前置从何而来、组合何时完成、搜索如何收束、成功如何交接、正确性与合法性如何分工。导论里不把这些当问题当作本章的最高问题——最高问题是场景本身: 何时该用这套方法、它替用户把「能力事实」翻译成哪一类判断。后文数学与门禁, 是为在这些场景里把判断做可检查。

3.1.8 最小集合、必须掌握与本章边界

这里说的「理解和控制整个工程」，**不是**指掌握写 AscendC、跑 CANN、证 ML-KEM 正确性的全部知识；而是特指：**预研过程是否沿已验证能力合法生长**。

我们对「最小」取工程判定：

若删去三件工具中的某一件，第 2 章四组问题里至少有一类会**重新失去可写的控制句柄**；若再往本章塞进密码代数或可判定性专论，则对「合法生长」的控制**并不增加一等必要句柄**。

表 3.3: 三件工具为何构成「合法生长」控制的最小集合

大段	数学工具	删掉它会失去什么控制	能否用其它工具顶替
一	依赖图与依赖闭包	无法回答「谁依赖谁 / 还缺谁」	否：形式语言与自动机都 预设 依赖世界已画清
二	形式语言	无法区分「菜单绿」与「拼法合法」	否：依赖齐备 $\neq$ 拼装计划合法
三	作业过程自动机	无法描述作业如何接受，也无法作废过期证	否：静态依赖图与语言都不谈过程转移
落地	对照回 KeyGen	术语落不回故事，学完仍不会用	可缩短叙述，但 不可 从学习路径删掉

必须掌握什么（验收清单） 读完本章，至少应能**独立完成**：

1. **依赖图与依赖闭包**：画出依赖图片段与探路顺序；写出依赖闭包与缺项；说明闭包表核什么。
2. **形式语言（合法拼装）**：说明为何「单点 PASS」推不出「拼接 PASS」；说明门禁如何检查拼装计划。
3. **作业过程自动机**：说出「展开 $\rightarrow$ 探针取证 $\rightarrow$ 写入证书」到接受的大意；解释脏标记为何使已获证清单可收缩。
4. **枢纽**：能指出依赖、展开、进度分别由**依赖图与闭包**、**形式语言**、**作业过程自动机**哪一件管。
5. **落地**：指着 KeyGen 故事说清：哪一段落在依赖图与闭包、哪一段落在形式语言、哪一段落在作业过程自动机。

刻意不纳入本章的

不纳入本章	原因（一句话）
格、MLWE、NTT 等代数	回答「算得对不对」，不回答「沿哪条已证边长」
AscendC API / tiling 细则	回答「核怎么写」，由平台工程笔记与探针承担
可判定性、复杂度专论	后置；门禁 v0 先要对象与规则可写
仓库字段 / workflow 细节	第 4-5 章工程化，不在本章展开

3.1.9 本章主线一览（读图后再进数学）

正文按三件数学工具分成三大段；每一大段内部都是：**先写该工具的数学**，再写**它在工程过程中的体现**。从下一节起，用一整大节写依赖图与依赖闭包（从集合讲到闭包）；随后两大节分别写形式语言与作业过程自动机。



图 3.1：第 3 章主线：三大工具段（每段＝数学定义＋工程体现）

	本章建设	本章明确不建设
对象	依赖图与闭包、形式语言、自动机（及证书演化）	密码方案正确性证明
体例	定义 / 命题 / 定理 / 引理 / 推论＋双例	百科式堆砌无证明的名词
深度	以「后文门禁够用」为准；标准结果给证明梗概	可判定性、复杂度专论（后置）

注 3.2 (阅读目标). 读完导论，应能用自己的话复述：这套方法主要帮研究和开发各解决什么问题、它适合什么样的工作（以及和普通「模块＋测试＋版本管理」差在哪）、三件工具各管什么。读完 后文数学后，应能按「必须掌握」清单自检，并对照表 3.1 说出工程术语与数学记号的对应。

下面按三件工具各开一大节写数学；写完再统一谈工程体现、回扣第 2 章、枢纽对照与本章小结。

3.2 依赖图与依赖闭包

本大节专门写第一件数学工具：**依赖图**与其上的**依赖闭包**。目标一句话：说清「谁依赖谁、先证谁」，并算出「动手前还缺谁」。叙述从集合、关系与函数讲起，再到有向图、DAG、拓扑序，最后落到依赖闭包与缺项。学完应能画出依赖关系、写出缺项；工程落点（闭包表等）留到后文「工程体现」统一写。

3.2.1 集合、笛卡尔积、关系与函数

定义 3.1 (集合与属于关系). 称由确定对象组成的汇集为**集合**。若对象 x 属于集合 A , 记 $x \in A$; 否则记 $x \notin A$ 。不含任何元素的集合称为**空集**, 记作 $\emptyset$ 。若 A 的每个元素都属于 B , 称 A 为 B 的**子集**, 记 $A \subseteq B$ 。

定义 3.2 (幂集). 设 V 为集合。由 V 的**全部子集**组成的集合称为 V 的**幂集**, 记作 2^V (亦常记作 $\mathcal{P}(V)$)。用集合造集的写法即

$$2^V = \{A \mid A \subseteq V\}.$$

特别地: $\emptyset \in 2^V$, $V \in 2^V$; 若 V 有限且 $|V| = n$, 则 $|2^V| = 2^n$ (这正是记号 2^V 的来源)。

映射 $f: 2^V \rightarrow 2^V$ 表示: 输入是 V 的某个子集, 输出仍是 V 的某个子集。

注 3.3 (为何是 2^V , 而不是 3^V 、 4^V). 底数 2 来自构造子集时的**二元选择**: 对 V 中每个元素, 只有「放进该子集」或「不放进」两种可能。因此不同子集的个数是 $2 \times 2 \times \cdots \times 2$ (共 $|V|$ 个因子), 即 $2^{|V|}$ 。若改写成 3^V , 一般既不等于「全部子集的集合」, 也没有上述计数意义 (除非另作约定, 例如讨论「每个元素三种染色」的函数集 $\{0, 1, 2\}^V$, 那是另一对象)。

等价观点: 把 2 看成二元集 $\{0, 1\}$, 则 2^V 也可理解为「全体函数 $V \rightarrow \{0, 1\}$ 」(特征函数: 取 1 表示属于子集, 取 0 表示不属于); 这与「全部子集」一一对应。

例 3.2 (纯数学: 幂集). 令 $V = \{a, b\}$ 。则

$$2^V = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}, \quad |2^V| = 4 = 2^2.$$

定义 3.3 (笛卡尔积). 设 A, B 为集合。其**笛卡尔积**为

$$A \times B = \{(a, b) \mid a \in A, b \in B\}.$$

元素 (a, b) 称为**有序对**; 一般地 $(a, b) = (a', b')$ 当且仅当 $a = a'$ 且 $b = b'$ 。

定义 3.4 (二元关系). 设 A, B 为集合。若 $R \subseteq A \times B$, 则称 R 为从 A 到 B 的**二元关系**。当 $A = B$ 时, 也称 R 为 A 上的二元关系。若 $(a, b) \in R$, 常简记为 $a R b$ 。

定义 3.5 (关系的定义域与值域). 对关系 $R \subseteq A \times B$, 定义

$$\text{dom}(R) = \{a \in A \mid \exists b \in B, (a, b) \in R\}, \quad \text{ran}(R) = \{b \in B \mid \exists a \in A, (a, b) \in R\}.$$

定义 3.6 (函数). 设 A, B 为集合。称关系 $f \subseteq A \times B$ 为从 A 到 B 的**函数** (记 $f: A \rightarrow B$), 若对每个 $a \in A$, 存在唯一的 $b \in B$ 使得 $(a, b) \in f$ 。此时记 $b = f(a)$, 称 b 为 a 在 f 下的**像**。

性质 3.1 (关系与函数的对比). 二元关系允许「一个 a 对应多个 b 」; 函数禁止这一点。后文「证书状态赋值」按函数理解; 「依赖边」按一般关系理解。

例 3.3 (纯数学: 课程先修关系). 令 $A = \{\text{高数}, \text{线代}, \text{概率}, \text{机器学习}\}$, 定义 $R \subseteq A \times A$ 为

$$R = \{(\text{高数}, \text{概率}), (\text{线代}, \text{机器学习}), (\text{概率}, \text{机器学习})\}.$$

则 $(\text{高数}, \text{概率}) \in R$, 但 $(\text{概率}, \text{高数}) \notin R$ (有序性)。映射 $\text{grade}: A \rightarrow \{A, B, C\}$ 若存在, 则是函数: 每门课恰一个成绩。

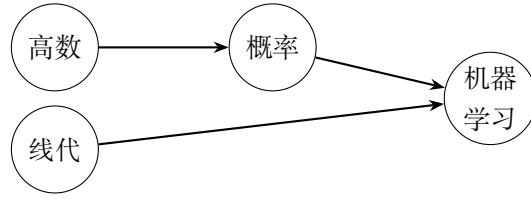


图 3.2: 配图: 纯数学: 课程先修关系

例 3.4 (工程: §2 数据依赖中的关系). 令 $V_0 = \{d, z, \text{Alg.13}, ek_{\text{PKE}}, dk_{\text{PKE}}, H, ek_{\text{KEM}}, dk_{\text{KEM}}\}$. §2 图二中的箭头构成 V_0 上的关系 R_{KG} , 例如 $(d, \text{Alg.13}) \in R_{\text{KG}}$, $(z, \text{Alg.13}) \notin R_{\text{KG}}$. 「把 ek_{PKE} 直通记为 ek_{KEM} 」可视为部分函数 $\iota: \{ek_{\text{PKE}}\} \rightarrow \{ek_{\text{KEM}}\}$. 下图为关系 R_{KG} 的示意 (数据流语义; 闭包节将改用依赖方向).

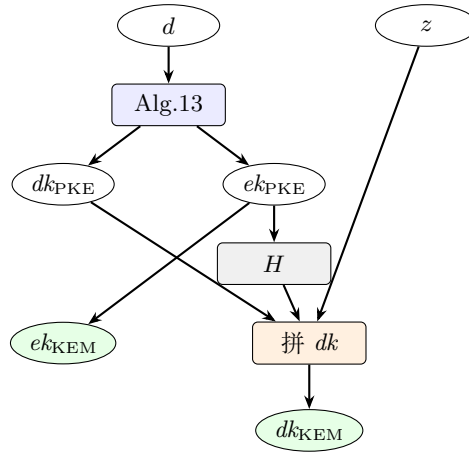


图 3.3: 示意配图 2

3.2.2 有向图、路与环

定义 3.7 (有向图). 有向图是一个有序对 $G = (V, E)$, 其中 V 为非空有限集 (称为**顶点集**), $E \subseteq V \times V$ 为**有向边集**. 若 $(u, v) \in E$, 称有向边 $u \rightarrow v$, 并称 u 为该边的**起点**, v 为**终点**.

定义 3.8 (出邻域与入邻域). 对 $v \in V$, 定义

$$N^+(v) = \{u \in V \mid (v, u) \in E\}, \quad N^-(v) = \{u \in V \mid (u, v) \in E\}.$$

分别称为 v 的**出邻域**与**入邻域**; $|N^+(v)|$ 、 $|N^-(v)|$ 称为**出度**、**入度**.

定义 3.9 (有向途径、路径与环). 设 $G = (V, E)$.

1. 顶点序列 $v_0, v_1, \dots, v_k$ ($k \geq 0$) 称为**有向途径**, 若对所有 $0 \leq i < k$ 有 $(v_i, v_{i+1}) \in E$; 称 k 为该途径的**长度**.
2. 若途径中顶点两两不同, 则称为**有向路径** (简称**路径**).
3. 若 $k \geq 1$ 、 $v_0 = v_k$, 且 $v_0, \dots, v_{k-1}$ 两两不同, 则称为**有向环** (简称**环**).

定义 3.10 (可达). 若存在从 u 到 v 的有向途径, 则称 v 从 u **可达**, 记 $u \rightsquigarrow_G v$ (在上下文明确时可略去 G). 约定任意 v 满足 $v \rightsquigarrow v$ (长度为 0 的途径).

引理 3.1 (路径与途径). 在有限有向图中, $u \rightsquigarrow v$ 当且仅当存在从 u 到 v 的有向路径。

证明. 若存在途径, 删去其上环段可得路径; 反之, 路径本身是途径。 $\square$

例 3.5 (纯数学: 四点有向图). 令 $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (1, 3), (2, 4), (3, 4)\}$ 。则 $1 \rightsquigarrow 4$, 且存在两条不同路径 $1 \rightarrow 2 \rightarrow 4$ 与 $1 \rightarrow 3 \rightarrow 4$ 。4 的入邻域为 $N^-(4) = \{2, 3\}$, 出邻域 $N^+(4) = \emptyset$ 。该图无环。

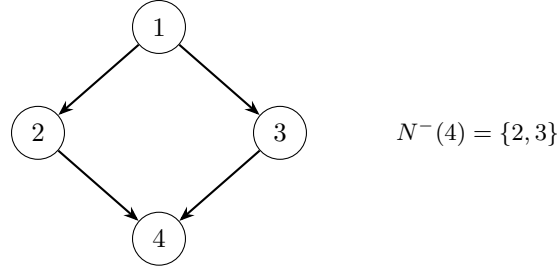


图 3.4: 配图: 纯数学: 四点有向图

例 3.6 (工程: KeyGen 数据流图是有向图). §2 图二给出顶点 (如 d 、 z 、Alg.13、 dk_{KEM}) 与有向边。 dk_{KEM} 的入度大于 1 (多源汇合); $z \rightsquigarrow dk_{\text{KEM}}$ 但 $z \not\rightsquigarrow \text{Alg.13}$ 。这与「树只有单父」不同: 有向图允许多父汇点 (见上节工程例图中「拼 dk 」结点)。

3.2.3 有向无环图与拓扑序

定义 3.11 (DAG). 若有向图 G 不含有向环, 则称 G 为**有向无环图** (Directed Acyclic Graph, DAG)。

定义 3.12 (拓扑序). 设 $G = (V, E)$, $|V| = n$ 。顶点序列 $v_1, \dots, v_n$ 称为 G 的一个**拓扑序**, 若它是 V 的排列, 且对每条边 $(v_i, v_j) \in E$ 都有 $i < j$ 。

定理 3.1 (DAG 与拓扑序的刻画). 设 G 为有限有向图。则下列条件等价:

1. G 是 DAG;
2. G 存在拓扑序。

证明梗概. (2) $\Rightarrow$ (1): 若有环, 则环上边无法全部满足「起点下标小于终点下标」。

(1) $\Rightarrow$ (2): DAG 中必有入度为 0 的顶点; 取其为 v_1 , 删去后再对剩余图归纳。 $\square$

推论 3.1 (可按先决条件调度). 若依赖关系构成 DAG, 则存在线性顺序, 使得每条依赖边都从「先处理」指向「后处理」。

注 3.4 (预研建模提示). 「先验证下层能力, 再拼上层任务」要求依赖关系可拓扑排序, 因而应建模为 DAG。若出现环, 通常意味着节点切分不清或误把双向约束画成了依赖。

例 3.7 (纯数学: 给四点图写拓扑序). 续前例 $E = \{(1, 2), (1, 3), (2, 4), (3, 4)\}$ 。1, 2, 3, 4 与 1, 3, 2, 4 都是拓扑序; 2, 1, 3, 4 不是 (边 $1 \rightarrow 2$ 逆向)。

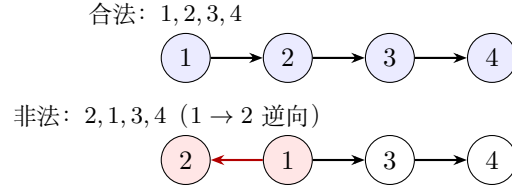


图 3.5: 配图: 纯数学: 给四点图写拓扑序

例 3.8 (工程: 能力依赖链). 将「P0 平台事实 → P1 原语 → P2 PKE.KeyGen → P3 KEM.KeyGen」视为一条链, 则它是 DAG, 且该顺序本身就是拓扑序。真实仓库图会有多父多子, 但仍应保持无环。



图 3.6: 配图: 工程: 能力依赖链

注 3.5 (DAG 不回答的问题). 能力 DAG 描述**引理库中的静态依赖投影** (谁可以依赖谁)。它不自动给出「某次做 KEM.KeyGen 时 Agent 实际展开的那棵树」——后者是派生 (见后文)。

3.2.4 闭包: 从一般概念到依赖闭包与缺项

前几小节已有集合、关系、有向图。这里先回答: **数学里说的「闭包」究竟是什么?**再落到能力依赖图上的**依赖闭包**、**已获证与缺项**。

在运算下封闭

定义 3.13 (一元运算下的封闭性). 设 U 为全集 (讨论所在的大集合), $A \subseteq U$, 又设 $\varphi: U \rightarrow U$ 为一元运算 (函数)。称 A 对 φ **封闭** (或「在 φ 下封闭」), 若

$$\forall x \in A, \quad \varphi(x) \in A.$$

即: 从 A 里任取元素做一次 φ , 结果仍落在 A 里。

定义 3.14 (二元运算下的封闭性). 设 $*: U \times U \rightarrow U$ 为二元运算。称 $A \subseteq U$ 对 $*$ **封闭**, 若

$$\forall x, y \in A, \quad x * y \in A.$$

定义 3.15 (对一族运算封闭). 若给定若干运算 (可混一元、二元), 称 A 对它们**封闭**, 当且仅当 A 对其中每一个运算都封闭。

例 3.9 (纯数学: 封闭与不封闭). 取 $U = \mathbb{Z}$ 。

- 偶数集 $2\mathbb{Z}$ 对加法封闭: 两偶数之和仍为偶数。
- 奇数集对加法**不**封闭: $1 + 1 = 2$ 不是奇数。
- $\{0, 1, 2\}$ 对「加 1」这一元运算**不**封闭, 因为 $\varphi(2) = 3 \notin \{0, 1, 2\}$ 。

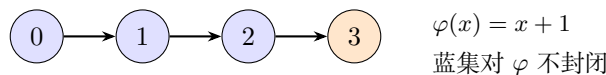


图 3.7: 配图: 纯数学: 封闭与不封闭

例 3.10 (工程直觉 (尚不形式化)). 若「已具备的能力集合」 Γ 在「取直接依赖」这一操作下不封闭, 就意味着: 你以为齐了, 但某个绿结点还缺未列入的先决——这正是后文要引入的工程对象「缺项」所要抓的现象。

闭包: 最小的封闭扩张 直观想法: 给定种子集合 A , 以及若干「生成规则」(运算), 我们希望找到既包含 A , 又对规则封闭, 又尽可能小的集合。这个最小集合就叫做 A 的闭包。

定义 3.16 (闭包 (由封闭性刻画)). 设 U 为全集, $A \subseteq U$, 并固定一组使子集可谈「封闭」的运算规则 $\mathcal{R}$ 。称 $B \subseteq U$ 为 A 关于 $\mathcal{R}$ 的一个**闭包**, 若:

1. $A \subseteq B$ (包含种子);
2. B 对 $\mathcal{R}$ 封闭;
3. 若 $C \supseteq A$ 且 C 对 $\mathcal{R}$ 封闭, 则 $B \subseteq C$ (B 是所有「含 A 的封闭集」中的**最小者**)。

若这样的 B 存在, 常记作 $\text{Cl}_{\mathcal{R}}(A)$, 或在上下文明确时记 $\text{Cl}(A)$ 。

定理 3.2 (闭包的存在性 (有限情形够用版)). 若 U 有限, 则对任意 $A \subseteq U$ 与任意「从已有元素产生新元素」的规则族 $\mathcal{R}$ (形式: 由已有元组确定性地加入有限个新点), $\text{Cl}_{\mathcal{R}}(A)$ 存在且唯一, 并可取为

$$\text{Cl}_{\mathcal{R}}(A) = \bigcap \{C \subseteq U \mid A \subseteq C, C \text{ 对 } \mathcal{R} \text{ 封闭}\}.$$

(任意一族含 A 的封闭集, 其交仍含 A 且封闭; 有限全集上该族非空, 因 U 本身封闭。)

证明梗概. 验证「含 A 的封闭集」对任意交封闭, 故交集是最小元。唯一性由最小性立即得到。□

注 3.6 (两种看法同一个对象). • **由外向内**: 所有合格封闭集的交;

- **由内向外**: 从 A 出发反复施加规则, 直到不能再扩大 (见下一小节迭代)。

后文算法几乎总用「由内向外」。

定义 3.17 (迭代构造 (一般模板)). 设规则是: 对当前集合 X , 令 $\Phi(X)$ 为「把 X 对规则应用一轮后应有的元素」(要求 $X \subseteq \Phi(X)$)。定义

$$A_0 = A, \quad A_{k+1} = \Phi(A_k) \quad (k \geq 0), \quad A_{\infty} = \bigcup_{k \geq 0} A_k.$$

在有限全集且 Φ 单调时, 存在 K 使 $A_K = A_{K+1} = \cdots = A_{\infty}$, 且 $A_{\infty} = \text{Cl}(A)$ 。

例 3.11 (纯数学: 对「加 1」的闭包). 取 $U = \{0, 1, 2, 3, 4\}$, $\varphi(x) = x + 1$ (约定 $\varphi(4) = 4$ 以免越界), $A = \{2\}$ 。则迭代: $\{2\} \rightarrow \{2, 3\} \rightarrow \{2, 3, 4\} \rightarrow \{2, 3, 4\}$ 。故 $\text{Cl}(A) = \{2, 3, 4\}$ ——包含种子, 并对 φ 封闭的最小集。

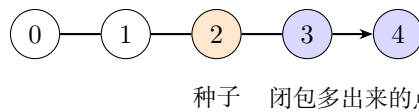


图 3.8: 配图: 纯数学: 对「加 1」的闭包

闭包算子 不必每次都提运算 $\mathcal{R}$ ，也可以直接把「取闭包」看成作用在幂集上的函数（幂集 2^U 的定义见 §3.2）。

定义 3.18 (闭包算子). 设 U 为集合。映射 $\text{Cl}: 2^U \rightarrow 2^U$ 称为**闭包算子**，若对任意 $A, B \subseteq U$ （即对任意 $A, B \in 2^U$ ）:

1. **扩张性**: $A \subseteq \text{Cl}(A)$;
2. **单调性**: $A \subseteq B \Rightarrow \text{Cl}(A) \subseteq \text{Cl}(B)$;
3. **幂等性**: $\text{Cl}(\text{Cl}(A)) = \text{Cl}(A)$ 。

称 $\text{Cl}(A)$ 为 A 的**闭包**；若 $A = \text{Cl}(A)$ ，称 A 为**闭集**（对该算子而言）。

命题 3.1 (闭集即「已经封闭」). A 为闭集当且仅当 $\text{Cl}(A) = A$ 。对由规则 $\mathcal{R}$ 导出的闭包算子，闭集恰好是对 $\mathcal{R}$ 封闭的那些子集。

性质 3.2 (闭包是「补全到封闭」). $\text{Cl}(A)$ 可以理解为：从 A 出发，把所有被规则逼出来的点都补上，直到不能再补。幂等性说：补第二次不会再增加新点。

例 3.12 (纯数学：若干熟悉的闭包)。

场景	种子 A	闭包在补什么
实数区间	$\{0, 1\}$	对「中间点」封闭 $\rightarrow [0, 1]$ （拓扑闭包的朴素版）
线性代数	若干向量	对加与数乘封闭 $\rightarrow$ 生成子空间
关系	边集 E	对「可走更长路径」封闭 $\rightarrow E^*$ （下一小节）

关系的传递闭包：闭包的第一个正式例子 现在回到二元关系。这是连接「一般闭包」与「图上依赖闭包」的桥梁。

以下固定有限集 V ，关系 $R \subseteq V \times V$ 。

定义 3.19 (关系的复合与幂). 对 $R, S \subseteq V \times V$ ，定义复合

$$R \circ S = \{(u, w) \mid \exists v, (u, v) \in R, (v, w) \in S\}.$$

令 $R^1 = R$, $R^{k+1} = R^k \circ R$ ，并令 $R^0 = \Delta_V = \{(v, v) \mid v \in V\}$ 。注意： R^0 是**单独约定的**恒等关系，表示长度为 0 的途径；它**不是**把 R 与自身复合得到的（复合从 R^1 、 R^2 , ... 才开始）。

定义 3.20 (传递闭包与自反传递闭包)。

$$R^+ = \bigcup_{k \geq 1} R^k, \quad R^* = \bigcup_{k \geq 0} R^k = R^0 \cup R^+.$$

称 R^+ 为 R 的**传递闭包**， R^* 为**自反传递闭包**。

命题 3.2 (R^* 确实是闭包). 在「关系集合」 $U = 2^{V \times V}$ 上，若规则是「若有 $(u, v), (v, w)$ 则加入 (u, w) ，且保留全部恒等对」，则从 R 生成的闭包恰为 R^* 。特别地： $R \subseteq R^*$ ， R^* 传递且自反，且是满足这些性质的最小关系。

定理 3.3 (传递闭包与路径). $(u, v) \in R^k$ 当且仅当存在长度恰为 k 的 R -途径从 u 到 v 。因而 $(u, v) \in R^*$ 当且仅当 u 经若干步 (含 0 步) 到达 v 。

例 3.13 (纯数学: 三点关系的传递闭包). 令 $V = \{a, b, c\}$, $R = \{(a, b), (b, c)\}$ 。逐项按定义展开:

$$R^0 = \Delta_V = \{(a, a), (b, b), (c, c)\} \quad (\text{恒等关系; 不是由 } R \text{ 复合得到}),$$

$$R^1 = R = \{(a, b), (b, c)\},$$

$$R^2 = R \circ R = \{(a, c)\} \quad (\text{复合: 经中间点 } b),$$

$$R^+ = R^1 \cup R^2 = \{(a, b), (b, c), (a, c)\} \quad (\text{传递闭包: 只含正长度路径}),$$

$$R^* = R^0 \cup R^+ = \{(a, a), (b, b), (c, c), (a, b), (b, c), (a, c)\}.$$

因此: (a, a) 这类对角元来自 R^0 (0 步路径约定), **不是** $R \circ R$ 算出来的; 本例中 R 本身也没有自环。图中三种线分别对应三类来源:

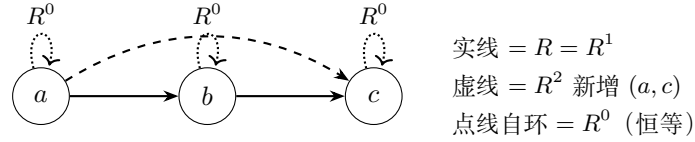


图 3.9: 示意配图 8

若教材图只想强调「传递多出来的边」, 可省略自环, 但须在式子里保留 R^0 , 以免误以为 (a, a) 来自复合。

例 3.14 (工程预告). 若把「 u 是 v 的直接先决」看成关系 R , 则「 u 是 v 的直接或间接先决」就是 $(u, v) \in R^*$ 。下一小节把「目标 T 的全部先决」整理成工程对象**依赖闭包**, 并引入记号 $\text{Dep}^*(T)$ 。

有向图上的依赖闭包 从此固定有向图 $G = (V, E)$, 并取边语义为依赖:

$$u \rightarrow v \quad \text{表示} \quad \text{「} v \text{ 依赖 } u \text{」} \quad (u \text{ 先决, } v \text{ 后继}).$$

与 §2 数据流箭头可能相反; 计算闭包以本节为准。

定义 3.21 (转置图). $G^\top = (V, E^\top)$, $E^\top = \{(v, u) \mid (u, v) \in E\}$ (箭头反向)。

定义 3.22 (直接前驱 / 直接后继).

$$\text{Pred}_G(v) = N^-(v), \quad \text{Succ}_G(v) = N^+(v).$$

定义 3.23 (祖先与后代). 若 $u \rightsquigarrow_G v$, 称 u 为 v 的**祖先**, v 为 u 的**后代** (含 $u = v$)。

定义 3.24 (单点依赖闭包). 工程术语: 目标 T 的**依赖闭包**——完成 T 理论上必须齐备的全部先决能力 (含 T 自身)。

数学记号: 固定依赖图 $G = (V, E)$, 对目标 $T \in V$ 定义

$$\text{Dep}_G^*(T) = \{u \in V \mid u \rightsquigarrow_G T\} = \{u \in V \mid (u, T) \in E^*\}.$$

恒有 $T \in \text{Dep}_G^*(T)$ 。略下标时写作 $\text{Dep}^*(T)$ 。此后正文若写 $\text{Dep}^*(T)$, 一律读作「 T 的依赖闭包」。

注 3.7 (与一般闭包的对接). 取种子 $\{T\}$, 规则为「若 v 已在集合中, 则把 $\text{Pred}(v)$ 全部加入」。则对该规则的闭包恰等于 $\text{Dep}^*(T)$ 。换言之: **依赖闭包** = 「对取前驱封闭」的最小扩张。

命题 3.3 (四种等价说法). 对任意 $u, T \in V$, 下列等价:

1. $u \in \text{Dep}^*(T)$;
2. $(u, T) \in E^*$;
3. 存在路径 $u \rightarrow \cdots \rightarrow T$ 或 $u = T$;
4. 在 $G^\top$ 中 $T \rightsquigarrow_{G^\top} u$ 。

依赖闭包的迭代、算法与算子性质

定义 3.25 (依赖闭包的迭代序列). 为计算 $\text{Dep}^*(T)$, 我们**新引入**一串辅助集合, 记作

$$D_0(T), D_1(T), D_2(T), \dots$$

(字母 D 仅表示「Dependency 迭代的第几层近似」, 下标 $k = 0, 1, 2, \dots$ 表示迭代步数; 前文并未使用过该记号, 本定义即其首次出现。)

约定:

$$\begin{aligned} D_0(T) &= \{T\} && \text{(第 0 步: 只有目标本身)}, \\ D_{k+1}(T) &= D_k(T) \cup \bigcup_{v \in D_k(T)} \text{Pred}(v) && \text{(第 } k+1 \text{ 步: 把当前集里每点的直接前驱都并进来)}. \end{aligned}$$

直观上: $D_k(T)$ 是「从 T 沿依赖边反向走、最多 k 步能摸到的点」所生成的集合 (含更近的点); 随着 k 增大, 集合只增不减。

这与前面「迭代构造 (一般模板)」中的 A_k 是同一类对象: 取种子 $A = \{T\}$ 、一轮算子 $\Phi(X) = X \cup \bigcup_{v \in X} \text{Pred}(v)$ 时, 恰有 $D_k(T) = A_k$ 。

定理 3.4 (迭代收敛到依赖闭包). 对有限图 G , 序列 $\{D_k(T)\}_{k \geq 0}$ 在有限步内稳定, 且

$$\bigcup_{k \geq 0} D_k(T) = \text{Dep}^*(T).$$

更精确地: 存在 $K \leq |V|$ 使对所有 $k \geq K$ 有 $D_k(T) = \text{Dep}^*(T)$ 。

证明梗概. $D_k(T)$ 单调递增且含于有限集 V , 故稳定。稳定时集合对取 Pred 封闭, 且含 T , 故包含一切能走到 T 的顶点; 反之, 任何通向 T 的长度为 ℓ 的路径上的点在 $D_\ell(T)$ 中出现。□

注 3.8 (与一般闭包模板的关系). 因此「迭代收敛到依赖闭包」就是一般闭包「由内向外」构造在依赖图上的具体化: 做够多步之后, $D_k(T)$ 不再变化, 其稳定值就是 $\text{Dep}^*(T)$ 。

引理 3.2 (反向搜索算法). 在有限图上执行: 从 T 出发, 在 $G^\top$ 中做 *BFS/DFS* (等价于沿 G 的边反向走), 访问到的顶点集合恰为 $\text{Dep}^*(T)$ 。时间复杂度 $O(|V| + |E|)$ (邻接表)。

目标集的闭包、基与缺项 前几小节处理的是单个目标 T 。工程上一次任务往往同时盯住多个目标（或先把目标拆成若干子目标），并且手里已有一份「哪些能力当前允许被引用」的清单——下文形式化为**已获证能力集**（数学记号 Γ ）。本小节先把依赖闭包从单点推广到有限目标集，再引入证书与缺项、生成集。

为何需要目标集 若只定义单目标的依赖闭包 $\text{Dep}^*(T)$ ，则同时要完成 T_1, T_2 时，需要的先决整体是 $\text{Dep}^*(T_1) \cup \text{Dep}^*(T_2)$ 。把它写成目标集的依赖闭包 $\text{Dep}^*({T_1, T_2})$ ，可避免每次手写大并集，也便于后文闭包表一行写多个目标。

定义 3.26 (有限目标集). 称非空有限集 $A \subseteq V$ 为一次讨论中的**目标集**。其元素仍是顶点（能力/任务结点）； A 只表示「当前要一起完成的那些目标」。单点情形是特例： $A = \{T\}$ 。

定义 3.27 (目标集的依赖闭包). **工程术语**：目标集 A 的**依赖闭包**——完成 A 中全部目标所需先决的并集。

数学记号：对 $A \subseteq V$ ，定义

$$\text{Dep}_G^*(A) = \bigcup_{T \in A} \text{Dep}_G^*(T).$$

并约定 $\text{Dep}_G^*(\emptyset) = \emptyset$ 。不致混淆时略去下标 G ，写作 $\text{Dep}^*(A)$ 。

命题 3.4 (目标集闭包的等价说法). 对任意 $A \subseteq V$ 与 $u \in V$ ，下列等价：

1. $u \in \text{Dep}^*(A)$;
2. 存在某个 $T \in A$ ，使 $u \in \text{Dep}^*(T)$ （即 $u \rightsquigarrow T$ 或 $u = T$ ）;
3. 存在 $T \in A$ ，使 $(u, T) \in E^*$ 。

因而 $\text{Dep}^*(A)$ 是「能通向 A 中至少一个目标的全部祖先（含这些目标自身）」。

命题 3.5 (Dep^* 作为幂集上的闭包算子). 定义 $\text{Cl} : 2^V \rightarrow 2^V$ 为 $\text{Cl}(A) = \text{Dep}^*(A)$ （此处 2^V 为 V 的幂集，见 §3.2）。则 Cl 满足：

1. **扩张**： $A \subseteq \text{Cl}(A)$;
2. **单调**： $A \subseteq B \Rightarrow \text{Cl}(A) \subseteq \text{Cl}(B)$;
3. **幂等**： $\text{Cl}(\text{Cl}(A)) = \text{Cl}(A)$ 。

因此 Cl 是前文意义下的**闭包算子**。进一步： A 对该算子为闭集（即 $\text{Cl}(A) = A$ ）当且仅当 A 对「取直接前驱」封闭——若 $v \in A$ 则 $\text{Pred}(v) \subseteq A$ 。

证明梗概. (扩张) 每个 $T \in A$ 满足 $T \in \text{Dep}^*(T)$ ，故 $A \subseteq \text{Dep}^*(A)$ 。

(单调) 并集对包含关系显然单调。

(幂等) 若 $u \in \text{Dep}^*(\text{Dep}^*(A))$ ，则 u 能通向 $\text{Dep}^*(A)$ 中某点 v ，而 v 又能通向 A 中某点，故 u 能通向 A 中某点，即 $u \in \text{Dep}^*(A)$ 。

闭集刻画：对取前驱封闭 $\Leftrightarrow$ 含某点则含其直接前驱，归纳得含全部祖先。 □

性质 3.3 (并与交的粗界). 对任意 $A, B \subseteq V$,

$$\text{Dep}^*(A \cup B) = \text{Dep}^*(A) \cup \text{Dep}^*(B), \quad \text{Dep}^*(A \cap B) \subseteq \text{Dep}^*(A) \cap \text{Dep}^*(B).$$

右式一般**不能**改成等号（「交的闭包」可以严格小于「闭包的交」）。

证书、已证书项与缺项 闭包回答「理论上需要谁」；工程上还要问「其中哪些已经允许引用、还缺哪些」。为此先钉死本教材中的**证书**一词（此前第 2 章资产清单、愿景段已口语出现，此处才给形式含义）。

定义 3.28 (证书). 方法论中的**证书**不是公钥基础设施里的数字证书，而是对某个顶点 $v \in V$ （能力/任务结点）的**可引用判决**：在约定验收准则下已通过验证，因而允许被后续任务当作先决引用。

- **主语**是顶点 v ，不是 Agent、也不是操作者本人；
- **工程载体**可以是探针 PASS 记录、stable 交付与 KAT、技术总结中的闭合条件等 (§2 资产清单「证书」列即此类证据形态)；
- **数学抽象**先只关心「 v 当前是否被承认为已获证」，细节状态机见迁移系统一节与第 4 章。

定义 3.29 (已证书集与已证书项). **工程术语**：**已获证能力集**（亦称已证书集）——当前允许被后续任务引用的能力集合。

数学记号：固定讨论时刻，令 $\Gamma \subseteq V$ 表示该集合。对任意 $v \in V$ ：

- 若 $v \in \Gamma$ ，称 v **已获证**，并称 v 为相对 Γ 的一个**已证书项**；
- 若 $v \notin \Gamma$ ，称 v **未获证**。

口语「已经拿到证书 / 已获得证书」即指 $v \in \Gamma$ 。 Γ 随预研过程可变：新顶点验收通过后可写入 Γ ；既有判决因上游变更不再被承认时可移出 Γ 。本小节先把 Γ 当作给定的集合参数；改写 Γ 的转移名称在缺项集定义之后预习，完整转移见后文。此后正文若写 Γ ，一律读作「当前已获证能力集」。

定义 3.30 (缺项集). **工程术语**：相对已获证清单的**缺项**——依赖闭包里还不允许引用的那些先决。

数学记号：对目标集 $A \subseteq V$ 与已获证能力集 $\Gamma \subseteq V$ ，定义**缺项集**

$$\text{Missing}_{\Gamma}(A) = \text{Dep}^*(A) \setminus \Gamma = \{u \in \text{Dep}^*(A) \mid u \notin \Gamma\}.$$

当 $A = \{T\}$ 时简记为 $\text{Missing}_{\Gamma}(T)$ 。直观： u 落在「完成 A 的依赖闭包」里，但当前**还未获证**，故不能直接引用。此后正文若写 $\text{Missing}_{\Gamma}(A)$ ，一律读作「相对 Γ 的缺项」。

注 3.9 (三类转移名称的预习). 完整转移关系在「迁移系统」一节定义。此处先约定三个**工程动作**的名称，并预告后文数学记号：

工程动作	后文记号	本小节所需的含义
写入证书	Certify	对未获证顶点 v ：验收通过后执行 $\Gamma \leftarrow \Gamma \cup \{v\}$
探针取证	Probe	对缺项启动探针/预研，收集可供写入证书使用的证据（本身不自动改 Γ ）
脏标记（作废）	Dirty	撤销部分既有证书：使某些 v 不再属于 Γ (证书库非单调，详见后文)

定义 3.31 (闭包已满). 若 $\text{Missing}_{\Gamma}(A) = \emptyset$ ，称目标集 A 相对 Γ **闭包已满**（亦称「依赖已齐备」）。此时 $\text{Dep}^*(A) \subseteq \Gamma$ ：完成 A 所需的全部先决都已**是已证书项**。

命题 3.6 (缺项的基本性质). 对任意 $A \subseteq V$ 与 $\Gamma, \Gamma' \subseteq V$:

1. $\text{Missing}_\Gamma(A) \subseteq \text{Dep}^*(A)$;
2. $\text{Missing}_\Gamma(A) = \emptyset \iff \text{Dep}^*(A) \subseteq \Gamma$;
3. 若 $\Gamma \subseteq \Gamma'$, 则 $\text{Missing}_{\Gamma'}(A) \subseteq \text{Missing}_\Gamma(A)$ (已证书集只增时, 缺项只减或不增);
4. $\text{Dep}^*(A)$ 可划分为 $(\text{Dep}^*(A) \cap \Gamma) \dot{\cup} \text{Missing}_\Gamma(A)$ (已证书部分与缺项部分不相交, 并起来恰为闭包)。

定义 3.32 (闭包表 (工作对象)). 称一次任务启动时填写、供核对的检查表为**闭包表**。数学上它至少应能填写: 目标集 A 、当前已证书集 Γ 、算出的 $\text{Dep}^*(A)$, 以及缺项集 $\text{Missing}_\Gamma(A)$ (后文还可附加禁字、接缝声明等)。「先交闭包表再写码」指: 实现之前必须先交出可核对的上述四元组 (及约定扩展栏)。

定义 3.33 (门禁 (工作含义)). **门禁**指预研工作流里「允许或拒绝某步动作」的检查规则 (例如: 缺项非空则禁止直接写顶层实现; 引用未获证依赖则拒绝)。其数学内核后文用语言成员判定与配置转移表述; 本小节只需把它当作「对着闭包表做的硬检查」。

注 3.10 (闭包表与门禁如何联动). 若 $\text{Missing}_\Gamma(A) \neq \emptyset$, 门禁应阻止「直接写顶层实现」, 并要求先对缺项做探针取证 (Probe), 再在验收通过后写入证书 (Certify), 直至相对 Γ 闭包已满 (或明确缩小目标)。

定义 3.34 (能力菜单与绿项). **能力菜单** (简称**菜单**) 指仓库中列出的可命名能力清单, 典型来源是各活跃 INDEX.md。菜单上出现的项称为**可见能力**; 若某可见能力当前属于 Γ , 口语可称其为**绿项** («绿» 只表示「此刻已获证», 不是形式状态机字母)。菜单提供构造 Γ 的**证据来源候选**, 本身既不等于 Γ , 也不蕴含 $\text{Missing}_\Gamma(T) = \emptyset$ 。

生成集与基 (为「最小补洞」作准备) $\text{Dep}^*(T)$ 可能很大; 有时只需指出一堆「种子点」, 其闭包已能覆盖整个 $\text{Dep}^*(T)$ 。下面先定义生成集, 再说明「补洞」指什么。

定义 3.35 (生成集). 设 $T \in V$ 。若 $B \subseteq V$ 满足

$$\text{Dep}^*(B) = \text{Dep}^*(T),$$

则称 B 为 $\text{Dep}^*(T)$ 的一个**生成集** (亦称「 B 生成 $\text{Dep}^*(T)$ 」)。特别地, 取 $B = \{T\}$ 时, 由定义 $\text{Dep}^*(\{T\}) = \text{Dep}^*(T)$, 故 $\{T\}$ 总是生成集。

定义 3.36 (极小生成集与基). 若 B 是 $\text{Dep}^*(T)$ 的生成集, 且 B 的任何真子集都不再是生成集, 则称 B 为**极小生成集**。在有限图上极小生成集一定存在 (从任意生成集不断删点直至不能再删)。本教材把「基」暂当作极小生成集的同义语; 更细的唯一性/规范基问题留待门禁章。

定义 3.37 (补洞与最小补洞). 相对已证书集 Γ , 把缺项逐步变成已证书项、从而使目标闭包已满的过程, 称为**补洞**。形式上一轮补洞至少包含: 选取某些缺项 $u \in \text{Missing}_\Gamma(T)$, 经探针取证后写入证书, 得到更大的已获证集合 Γ' 。若优先寻找尽可能小的缺项生成集 (定义见下) 作为补证对象, 则称该选择问题为**最小补洞** (算法细节后置)。

定义 3.38 (相对 Γ 的缺项生成集). 工程上更关心: 在已经知道 Γ 的前提下, 还需要新获证的是哪些点。若 $B \subseteq \text{Missing}_\Gamma(T)$ 满足

$$\text{Dep}^*((\text{Dep}^*(T) \cap \Gamma) \cup B) = \text{Dep}^*(T),$$

则称 B 为相对 Γ 的一个**缺项生成集**: 把「已证书部分」 $\text{Dep}^*(T) \cap \Gamma$ 与「还打算补证的 B 」合在一起, 闭包就能盖住整个 $\text{Dep}^*(T)$ 。最小补洞即在缺项生成集中寻找尽可能小者。

性质 3.4 (菜单、 Γ 与闭包). 由能力菜单的定义: 菜单提供构造 Γ 的证据来源候选, 但

$$\text{菜单上有许多绿项} \not\Rightarrow \text{Missing}_\Gamma(T) = \emptyset.$$

必须先算 $\text{Dep}^*(T)$, 再与当前 Γ 做差得到缺项。菜单回答「可能有哪些能力」; 闭包与缺项回答「为了 T 必须有谁、还缺谁」。

例 3.15 (纯数学: 四点图上的闭包与缺项). 续 $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (1, 3), (2, 4), (3, 4)\}$ (依赖语义: 箭头指向依赖者)。则 $\text{Pred}(4) = \{2, 3\}$,

$$D_0(4) = \{4\}, D_1(4) = \{4, 2, 3\}, D_2(4) = \{4, 2, 3, 1\} = \text{Dep}^*(4).$$

取目标集 $A = \{4\}$ 。若 $\Gamma = \{1, 2, 4\}$, 则

$$\text{Dep}^*(4) \cap \Gamma = \{1, 2, 4\}, \quad \text{Missing}_\Gamma(4) = \{3\}.$$

此时 $\{3\}$ 是一个缺项生成集: 补证 3 之后闭包已满。又 $\text{Dep}^*(\{2, 3\}) = \text{Dep}^*(4)$, 故 $\{2, 3\}$ 是 $\text{Dep}^*(4)$ 的一个生成集 (且为极小: 去掉 2 或 3 任一, 闭包都到不了 4 的全部祖先)。

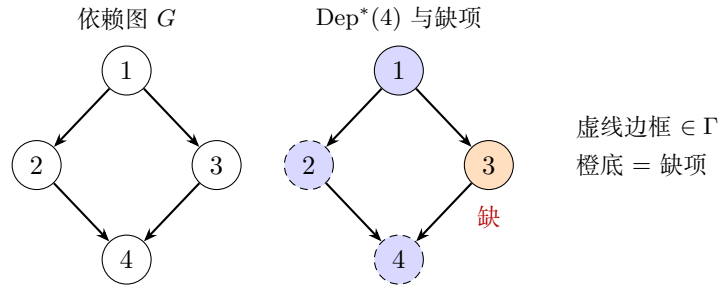


图 3.10: 示意配图 9

例 3.16 (工程: KEM.KeyGen 的依赖闭包示意). 取依赖语义下的示意 DAG (结点名为能力, 非数据): 边表示「箭头终点依赖箭头起点」。令 $T = \text{KEM.KG}$ 。则

$$\text{Dep}^*(T) = \{P0, P1, P2, \text{Tail}, \text{KEM.KG}\}.$$

若当前 Γ 已含 $P0-P2$, 但尾缝仍未获证 ($\text{Tail} \notin \Gamma$), 则 $\text{Missing}_\Gamma(T) = \{\text{Tail}\}$, 目标相对 Γ 未闭包已满。

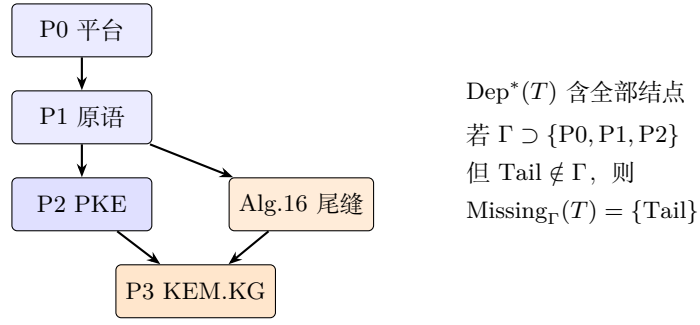


图 3.11: 配图: 工程: KEM.KeyGen 的依赖闭包示意

§2 资产清单 $\approx$ 当时对依赖闭包 $\text{Dep}^*(T)$ 与已获证清单 Γ 的手工对照; 未显式计算缺项 $\text{Missing}_{\Gamma}(T)$ 就开工, 正是「菜单 $\neq$ 说明书」的来源。

3.3 形式语言与合法拼装

本大节专门写第二件数学工具: **形式语言**。目标一句话: 把「怎样拼积木」写成可检查的**合法拼装计划** (字是否属于语言、是否避开禁止拼法)。工程上它回答: 菜单绿不等于拼法合法; 单点 PASS 推不出接缝合法。工程落点留到后文「工程体现」。

3.3.1 字母表、字与语言

定义 3.39 (字母表与字). 工程术语: 可用步骤组成的字母表; 一次**拼装计划**是步骤的有限序列。

数学记号: 非空有限集 Σ 称为**字母表**; 其元素称为**字母**。 Σ 上的**字**是有限序列 $w = a_1 a_2 \cdots a_n$ ($n \geq 0$, $a_i \in \Sigma$); $n = 0$ 时称为**空字**, 记 ε 。称 $n = |w|$ 为字长。全体字之集记 Σ^* (Kleene 星)。又记 $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$ 。在本方法论中: 字母 $\approx$ 已证能力或接缝动作; 字 $w \approx$ 某次拼装计划。

定义 3.40 (前缀、后缀与因子). 设 $w, x \in \Sigma^*$ 。

- 若存在 y 使 $w = xy$, 称 x 为 w 的**前缀**;
- 若存在 y 使 $w = yx$, 称 x 为 w 的**后缀**;
- 若存在 y, z 使 $w = yxz$, 称 x 为 w 的**因子** (子字)。

定义 3.41 (语言). 工程术语: **合法计划集合**——门禁承认的全部拼装计划。

数学记号: 若 $L \subseteq \Sigma^*$, 则称 L 为 Σ 上的一个**语言**。称 w 对 L **合法** (或 w 属于 L), 若 $w \in L$ 。此后若写 $w \in L$, 一律读作「拼装计划 w 合法」。

定义 3.42 (语言的连接与星闭包). 对 $x, y \in \Sigma^*$, **连接** xy 表示把 y 接在 x 后。对语言 $L, M \subseteq \Sigma^*$, 定义

$$LM = \{xy \mid x \in L, y \in M\}, \quad L^0 = \{\varepsilon\}, \quad L^{k+1} = L^k L, \quad L^* = \bigcup_{k \geq 0} L^k.$$

性质 3.5 (连接对合法性不必封闭). 一般地, $x \in L$ 且 $y \in L$ **不能**推出 $xy \in L$ 。(这是后文「组合不免费」的纯语言版本。)

命题 3.7 (Σ^* 的通用性). 任一语言都是 Σ^* 的子集; 讨论「合法计划」即指定某个 $L \subseteq \Sigma^*$ 。门禁的数学内核是: 给定 w , 判定 $w \in L$ 是否成立 (成员判定, 见后文)。

例 3.17 (纯数学: 偶数个 0 的语言(附图)). 取 $\Sigma = \{0, 1\}$, $L_{\text{even}0} = \{w \in \Sigma^* \mid w \text{ 中 } 0 \text{ 的个数为偶数}\}$. 下图用两状态自动机识别该语言 (q_0 接受): 读 0 翻转奇偶, 读 1 不动。

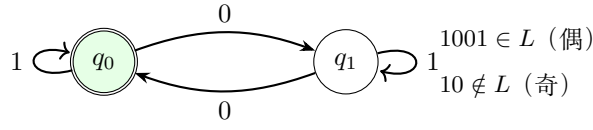


图 3.12: 配图: 纯数学: 偶数个 0 的语言 (附图)

注意: $0 \notin L$ 但 $00 \in L$, 故不能把「每个字母合法」误当成「连接合法」。

例 3.18 (工程: 实现计划当作字(附图)). 将原子动作编码为字母, 例如 $a = \text{RunPKE}$, $b = \text{Alg16Tail}$, $c = \text{WriteIO}$. 合法短计划 $w = abc$ 画成动作序列; 插入未证书动作 x 得 $abxc \notin L$.

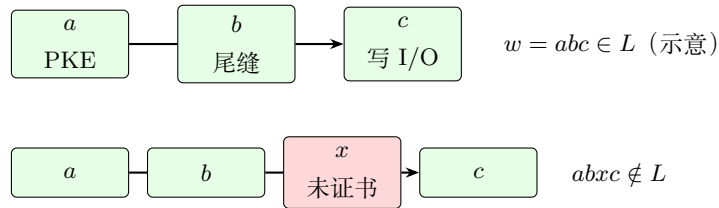


图 3.13: 配图: 工程: 实现计划当作字 (附图)

3.3.2 文法、产生式、派生与派生树

定义 3.43 (上下文无关文法 (本教材用法)). 上下文无关文法是四元组 $\mathcal{G} = (N, \Sigma, P, S)$, 其中:

- N 为有限非终结符集;
- Σ 为有限终结符集, $N \cap \Sigma = \emptyset$;
- $S \in N$ 为开始符号;
- P 为有限产生式集, 每个产生式形如 $A \rightarrow \alpha$, 其中 $A \in N$, $\alpha \in (N \cup \Sigma)^*$.

定义 3.44 (一步派生与多步派生). 若 $\alpha = \alpha_1 A \alpha_2$, $A \rightarrow \beta \in P$, 则称 α 一步派生出 $\alpha_1 \beta \alpha_2$, 记 $\alpha \Rightarrow \alpha_1 \beta \alpha_2$. $\Rightarrow^*$ 表示 $\Rightarrow$ 的自反传递闭包 (零步或多步派生)。

定义 3.45 (文法生成的语言).

$$L(\mathcal{G}) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}.$$

定义 3.46 (派生树). 对一次从 S 到 $w \in \Sigma^*$ 的派生, 其派生树 (语法树) 是满足以下条件的有根有序树: 根标记为 S ; 内部结点标记为非终结符; 叶子从左到右连接恰为 w ; 若内部结点标记 A 且其孩子标记依次拼成 α , 则 $A \rightarrow \alpha \in P$.

定理 3.5 (派生与派生树). 对上下文无关文法, $\mathcal{G}$ 能派生出 w 当且仅当存在以 w 为叶子连接的派生树。

注 3.11 (方法论中的对应). 非终结符 $\approx$ 尚未落到已证书原子步骤的目标; 终结符 $\approx$ 已证书调用或不可再分的合法动作; 产生式 $\approx$ 「目标可展开为哪些子目标 / 接缝」的改写规则。

例 3.19 (纯数学：匹配括号的文法片段). 令 $N = \{S\}$, $\Sigma = \{(\,,\,)\}$, 产生式 $S \rightarrow (S) \mid SS \mid \varepsilon$. 则 $() \in L(\mathcal{G})$, $()() \in L(\mathcal{G})$, 而 $() \notin$ 的补——更精确地说 $() \notin L(\mathcal{G})$. 派生树记录括号如何嵌套, 而不仅仅是字母的前后顺序。

例 3.20 (工程：KEM.KeyGen 的改写). 直觉产生式 (符号仅作说明):

$$\text{KEM.KeyGen} \rightarrow \text{PKE.KeyGen} \cdot \text{Alg16Tail}.$$

若 PKE.KeyGen 已是终结的已证书调用, 而 Alg16Tail 仍是非终结接缝, 则派生树根为 KEM.KeyGen , 一侧叶子为 PKE , 另一侧继续展开尾缝直至获证动作。

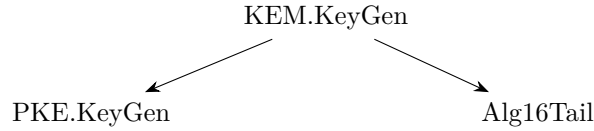


图 3.14: 配图：工程：KEM.KeyGen 的改写

3.3.3 成员判定、禁字与组合不免费

定义 3.47 (成员判定问题). **工程术语**: 检查一份拼装计划是否合法 (门禁里的成员检查)。

数学记号: 给定语言 $L \subseteq \Sigma^*$ 与字 $w \in \Sigma^*$, 判断是否 $w \in L$, 称为 L 的**成员判定**。

定义 3.48 (禁字集). **工程术语**: **禁止拼法**——不得出现在合法计划中的子串或关闭路线。

数学记号: 设 $\text{Forbidden} \subseteq \Sigma^*$. 称语言 L **避开** Forbidden , 若

$$L \subseteq \{w \in \Sigma^* \mid \text{不存在 } f \in \text{Forbidden 作为 } w \text{ 的因子}\}.$$

其中「因子」指存在 x, y 使 $w = xfy$. 此后若写 Forbidden , 一律读作「禁止拼法集合」(工程来源含 frozen/ 判决等)。

定义 3.49 (组合不免费 (方法论公理图式)). 称语言 L 满足**组合不免费**, 若存在 $x, y \in L$ 使得 $xy \notin L$. 在预研建模中我们**不假定** L 对连接封闭; 若工程上需要把 x 与 y 顺序拼装, 通常引入新的非终结符 (接缝) 与产生式, 并要求该接缝获证。

命题 3.8 (菜单不能代替成员判定). 设 $\Sigma_0 \subseteq \Sigma$ 为「菜单上可见的字母」。仅给出 Σ_0 不能唯一确定 L . 尤其, L 还依赖产生式集合与禁字集 Forbidden 。

例 3.21 (纯数学). 取 $L = \{a, b\} \subseteq \{a, b\}^*$. 则 $a, b \in L$ 但 $ab \notin L$, 故组合不免费. 若希望允许 ab , 必须把 ab 显式加入 L , 或增加产生式 (例如 $S \rightarrow ab$)。

例 3.22 (工程). PKE.KeyGen 计划与「某段哈希拼接」各自可能合法, 但「在新的 2-launch 边界下拼接二者」不必自动合法——这正是接缝节点 (如 Alg.16 尾缝、 KemKgTailFused) 存在的理由。frozen/ 文书对应 Forbidden 的工程来源之一。

注 3.12 (门禁雏形). Agent 提交「闭包表 + 展开计划 w 」时, 门禁检查的是: $\text{Missing}_\Gamma(T) = \emptyset$ (全依赖已覆盖) 且存在派生表明 $w \in L(\mathcal{G})$, 并且 w 避开 Forbidden . 这是「先交闭包表, 再写码」的数学表述。

3.4 作业过程自动机

本大节专门写第三件数学工具：**自动机**；在本方法论中的专名是**作业过程自动机**（亦称门禁自动机）。目标一句话：描述一次作业如何按合法转移走到接受，以及证书何时因脏标记作废。它接管「进度」：状态怎么变、何时接受/拒绝。工程落点留到后文「工程体现」。

3.4.1 迁移系统、配置与接受

定义 3.50 (迁移系统). **迁移系统**是三元组 $\mathcal{T} = (Q, \rightarrow, Q_0)$ ，其中 Q 为配置集， $\rightarrow \subseteq Q \times Q$ 为转移关系， $Q_0 \subseteq Q$ 为初始配置集。若 $(q, q') \in \rightarrow$ ，记 $q \rightarrow q'$ 。它是定义自动机时常用的底料；下一款定义把它特化成作业过程自动机。

定义 3.51 (作业过程自动机). **数学工具名**：自动机。

本方法论中的专名：作业过程自动机（门禁自动机）。

固定能力库顶点集与约束库后，作业过程自动机是一个迁移系统，其

- **状态**取为预研配置 $q = (\Gamma, F, S)$ （定义见下）；
- **转移**取为展开 / 探针取证 / 写入证书 / 声明接缝 / 脏标记 / 冻结路线等合法动作；
- **初始状态**由开任务时的已获证清单与前沿给出；
- **接受**见后文「接受配置」。

此后若只说「自动机」，在本章默认指作业过程自动机。

定义 3.52 (预研配置（自动机的状态）). **工程术语**：一次作业的**预研配置**——记下已获证能力、前沿还欠什么、以及约束。

数学记号：预研配置是三元组 $q = (\Gamma, F, S)$ ，其中

- $\Gamma \subseteq V_{\text{lib}}$ 为已获证能力集；
- F 为前沿目标的有限多重集（待展开或待获证的目标）；
- S 为约束库（含 I/O 契约、接缝声明、禁止拼法等）。

定义 3.53 (基本转移（名称可修订）). 在约束 S 允许的前提下，定义以下几类转移（均为 $\rightarrow$ 的子集）。表中先给**工程动作名**，再给后文沿用的数学记号：

工程动作	记号	作用
展开	Expand	按产生式展开 F 中某目标，更新缺项
探针取证	Probe	对缺项启动探针/预研以寻求证据
写入证书	Certify	验收通过后将结点写入 Γ
声明接缝	Compose	声明并认证接缝（落实组合不免费）
脏标记	Dirty	使部分证书失效（见下一节）
冻结路线	Freeze	将某路线写入禁止拼法 / 否定约束

定义 3.54 (接受与拒绝). 固定目标 T 与生产 I/O 契约 $\mathcal{C}$ 。

- 配置 $q = (\Gamma, F, S)$ 称为**接受配置**，若 $T \in \Gamma$ 、 F 为空（或仅含已放电目标），且 C 满足；
- 若转移过程试图使用 Forbidden 中的因子、或引用 $\notin \Gamma$ 的依赖边且未先 Probe/Certify，则进入**拒绝**（具体可建模为沉入拒绝阱状态）。

定义 3.55 (合法性与幻觉 (工作定义)). 1. 称一次作业过程**合法**，若它对应作业过程自动机中从某初始配置到接受配置的有限转移序列，且每一步都符合产生式与 S 中约束。

2. 称行为构成**幻觉**，若它执行了非法转移，或在图上使用了无证书边（等价地：计划字 $w \notin L$ 或未避开 Forbidden）。

语义 oracle (FIPS / liboqs / golden) 判定「计算结果是否正确」；本定义判定「砖是否允许那样砌」。

例 3.23 (纯数学：作业过程自动机小品). 状态 $\{q_0, q_{ok}, q_{bad}\}$ ，字母 $\{a, b\}$ 。 $q_0 \xrightarrow{a} q_0$ ， $q_0 \xrightarrow{b} q_{ok}$ ，其余进 q_{bad} 。则字 aab 被接受， ba 被拒绝。这与「先合法前缀、再结束符」的门禁同构。

例 3.24 (工程). 初始已获证清单含 stable PKE 等；前沿含 kem.keygen。经展开得到尾缝缺项 $\rightarrow$ 声明接缝/探针取证 $\rightarrow$ 写入证书后目标获证，到达接受。若中途使用了**禁止拼法**中的因子（已关闭或明确禁止的路线），则触发拒绝（**冻结路线**已写入禁止拼法集合）。

3.4.2 证书的非单调更新 (脏标记)

定义 3.56 (单调与非单调证书库). 称证书更新机制为**单调**的，若任意转移满足 $\Gamma \subseteq \Gamma'$ （证书集只增不减）。若允许某次转移后 $\Gamma' \subsetneq \Gamma$ ，则称为**非单调**的。

定义 3.57 (脏标记). **工程术语：脏标记**（证书作废）——因上游约定变更，下游既有证书不再被承认。

数学记号：对 $v \in \Gamma$ ，若因上游接口、布局或同步约定变更，使 v 的既有证书不再被承认，则称 v 被标记为 dirty，并在转移 Dirty 下将 v 移出 Γ （或移入「待重证」区）。具体存储格式后文工程化。此后若写 Dirty，一律读作「脏标记 / 证书作废」动作。

命题 3.9 (工程引理库非单调). 本方法论采用的证书库是非单调的：存在合法的 Dirty 转移使 Γ 严格缩小。

推论 3.2 (过期证书不可当作永真公理). 门禁在引用 $v \in \Gamma$ 时，须能够回答「自获证以来上游是否触发 Dirty」。

例 3.25 (纯数学：撤回引理). 证明系统中若发现引理 L_1 的证明有漏洞而撤回，则所有依赖 L_1 的定理证书失效。这与「断言永真后只增不减」的理想图景不同。

例 3.26 (工程). P1 某能力的 I/O、布局或同步约定变更后，曾基于旧约定的 P2/P3 证书应标 dirty，不得因「上次 PASS」而默认继续合法。

3.5 工程体现：三件工具在仓库里落成什么

数学写完之后，把三件工具各自在仓库里的落点收在一处，便于对照，也避免「每讲完一小段数学就插工程」打断主线。

3.5.1 依赖图与依赖闭包 → 闭包表、资产清单与 frozen/

依赖图与依赖闭包在工程里主要落成三样东西（第 2 章已见过雏形）：

1. **资产清单**：当时认为「已获证」的能力列表（ Γ 的手工版）；
2. **闭包表**：目标、已获证、依赖闭包、缺项四栏核对——动手前先交这张表；
3. **frozen/**：明确不再允许当作依赖去走的边（禁止再引用的路线）。

没有依赖图与闭包，就会把「INDEX 上有绿项」误当成「依赖已齐、可以开写」。

3.5.2 形式语言 → 拼装计划、门禁与禁止拼法

形式语言在工程里主要落成：

1. **拼装计划**：一次任务打算按什么顺序、用哪些已证能力与接缝去拼；
2. **门禁的成员检查**：这份计划是否属于合法计划集合，是否避开禁止拼法；
3. **禁止拼法**：frozen/ 等关闭路线的语言侧对应物。

没有形式语言，就会把「菜单上有这些绿项」误当成「任意拼接都合法」。

3.5.3 作业过程自动机 → 作业怎么走完、证书何时作废

作业过程自动机在工程里主要落成：

1. **作业轨迹**：从开任务配置走到「目标获证、前沿清空」的接受配置；
2. **过程动作**：展开缺项、探针取证、写入证书、声明接缝；
3. **脏标记**：上游 I/O、布局或同步约定变更后，下游不得继续假绿。

没有作业过程自动机，就只能事后说「这次碰巧 PASS」，无法描述过程是否合法、证书为何作废。

3.6 回扣第 2 章：流程如何对应本章数学

本段目标 对照图 3.1：不再引入新定义，按三件工具把第 2 章 KeyGen 流程对照回去。学完应能指着故事说清：哪一段是依赖图与闭包（还缺谁），哪一段是形式语言（拼装是否合法），哪一段是作业过程自动机（怎么走完 / 脏标记）。

第 2 章讲的是**已经走过的**工程故事；本章给的是**用来把故事说清楚的一套语言**。本节不再引入**新定义**，只做对照。中间隔了不少页，第 2 章那些表和图容易对不上号。所以下面把最关键的几张**按本章已挂钩对象重画一遍**（已获证清单、依赖闭包、缺项、接缝、预研配置等；需要时括号附记号），让读者一眼看出：挂上数学记号之后，原先那张工程图长什么样。第 2 章里的原表、原图仍保留备查：表 2.1、图 2.1、图 2.2、表 2.2。

表 3.4: 第 2 章里的流程 / 物件, 与本章对象的总对照 (工程词为主)

第 2 章里的流程 / 物件	本章里对应什么 (节号凭直觉即可)
表 2.1 资产清单里的「绿积木」	已获证能力集 Γ ; 顶点已经拿到证书 (§3.5)
活跃 INDEX 上的探针列表	能力菜单 (可用步骤的线索); 不是 合法计划集合, 也不必 等于已获证清单
图 2.1: P0/P1 $\rightarrow$ 分段 $\rightarrow$ stable PKE	能力依赖图上的生长路径; 拓扑序 (§3.4)
「做 KEM 前心里大概有张清单」	手工把依赖闭包与已获证清单对了一遍; 缺项当时并没有显式算出来
图 2.2: 19 调 16 调 13 + 数据	调用嵌套与数据 依赖; 要和能力依赖图、派生树 分开看 (§3.6)
Alg.16 尾缝、2-launch 接缝、拼 dk	接缝 / 声明接缝; 组合并不免费 (§3.9)
correctness 那条高成本探索路	手里只有菜单和伪代码; 缺「拼装计划是否合法」的成员检查 (§3.9)
device 去掉 vendor $\rightarrow$ 再晋级 stable	探针取证 / 写入证书; 预研配置轨迹一路走到接受 (§3.10)
「上次 PASS 了, 这次仍可能翻车」	证书可以往回缩; 脏标记 (§3.11)

总对照表 (先扫一眼)

流程一: 资产清单的「数学完全体」 表 2.1 在第 2 章只回答两件事: 手里有什么、证书长什么样。表 3.5 把它重画成开任务那一刻、相对目标 $T = \text{kem.keygen.k4}$ 的闭包对照, 多加了三列: 「是否已获证?」「是否落在依赖闭包里?」「扮演什么角色」。

表 3.5: 资产清单的数学完全体 (相对 $T = \text{kem.keygen.k4}$ 的开任务快照)

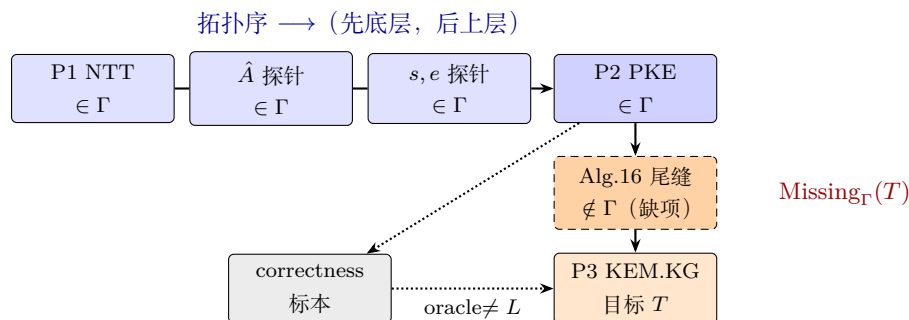
层	能力 (简述)	已获证?	在依赖闭包?	角色 (本章用语)
P0	DataCopy / MMAD / TPipe 等	是	是	已获证的祖先
P1	NTT / SampleNTT / CBD / ...	是	是	已获证的祖先
P2	Alg.13 分段与 device	是	是	已获证的祖先
P2	stable PKE KeyGen	是	是	直接先决 (已获证 $\cap$ 依赖闭包)
—	Alg.16 尾缝 / 2-launch 接缝	否	是	缺项
P3	KEM KeyGen correctness	(标本)	—	语义对照用; 不算 合法派生的证据
P3	KEM KeyGen device / stable	开任务时否	是 (目标侧)	还等着写入证书的目标

开任务时可以粗略写成: 已获证清单已覆盖 P0/P1/P2, 但缺项仍含尾缝, 所以依赖闭包**还没齐**。INDEX 菜单上标绿的项, 只是构造已获证清单时可以去翻的证据来源, **代替不了**本表。

由此可以看清两件事:

- 「站在 stable PKE 上」应写成「该能力已获证」，不能只看目录名里有没有 `stable`；
- 菜单、已获证清单、缺项是三样东西，谁也替不了谁（这正是第一组问题的落点）。

流程二：探路图的「数学完全体」 图 2.1 画的是仓库时间线。图 3.15 沿用同一骨架，标出拓扑序方向、哪些点已获证、哪里是缺项，以及 correctness 那条旁路（沿着它走， $\notin L$ ）。



实线边 $\approx$ 能力 DAG 上可以引用的依赖 / 接缝；

虚线框 = 开任务时还没拿到证书的缺项；灰框 = 语义对照，不当合法派生的模板。

图 3.15: 工程探路图的数学完全体 (Γ 、拓扑序、缺项、oracle 旁路)

- 蓝底结点：开任务时已经在 Γ 里；探路顺序和拓扑序一致；
- 虚线橙框里的尾缝：落在 $\text{Dep}^*(T)$ 里，却不在 Γ 里，所以是缺项；
- correctness 灰框：可以拿来当字节对照，但不会自动推出 $w \in L$ 。

流程三：目标、闭包与缺项（对着完全体表直接读） 一旦选定 $T = \text{kem.keygen.k4}$ ，表 3.5 和图 3.15 合在一起就能直接读出：

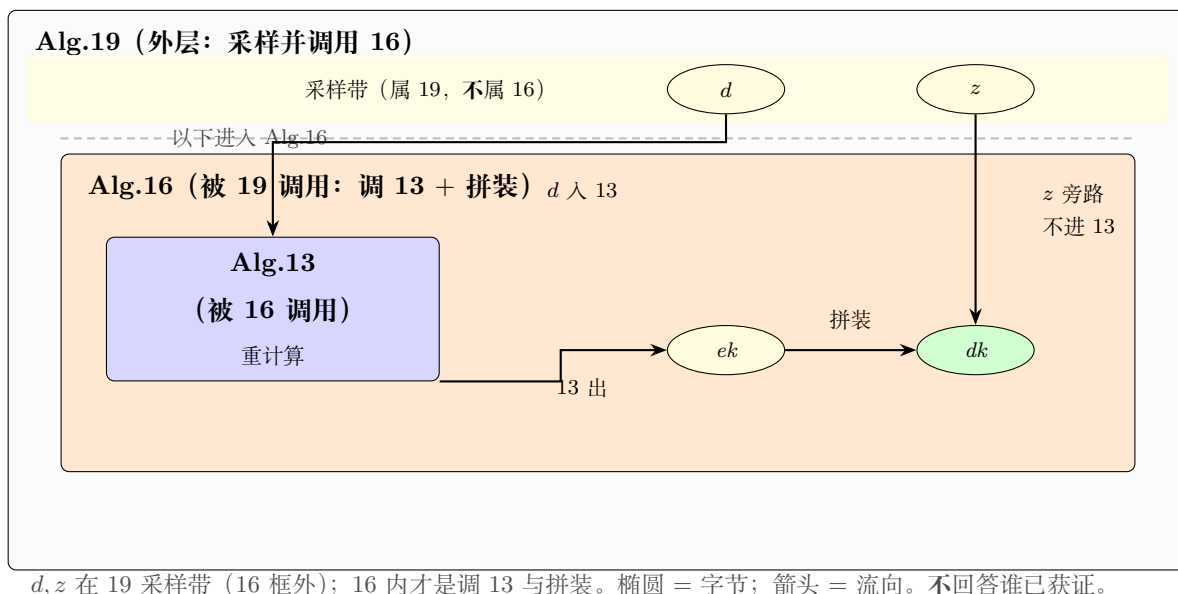
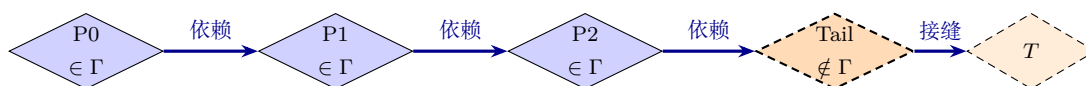
$$\begin{aligned}\text{Dep}^*(T) &\supseteq \{P0, P1, P2, \text{Tail}, T\}, \\ \Gamma_{\text{开任务}} &\supseteq \{P0, P1, P2\}, \\ \text{Missing}_\Gamma(T) &\ni \text{Tail}.\end{aligned}$$

第 2 章当时并没有强制「先交出 $\text{Missing}_\Gamma(T)$ ，再动手写码」——闭包表要变成门禁，缺口就在这里。

流程四：依赖、展开与进度——三件不同的事画在一页 图 2.2 画的是调用嵌套 + 数据字节；图 2.1 更接近能力怎么长出来。图 3.16 把三层画在同一页，并用三种不同图元区分（别只看颜色）：

- 层 A：大框套小框 = 谁调用谁；椭圆 = 数据；箭头 = 数据流向；
- 层 B：菱形 = 能力顶点；实心 = $\in \Gamma$ ，虚边 = 缺项；粗箭头旁写「依赖」；
- 层 C：圆 = 配置状态；边上写转移名；双圆 = 接受。

层 A：调用嵌套 + 数据依赖 (≈ 图 2.2)

层 B：能力 DAG + Γ (≈ 表 3.5)

图元：菱形 = 能力; 实心 $\in \Gamma$, 虚边 = 缺项; 粗蓝箭头 = 依赖/接缝。不回答字节路径。

层 C：一次配置轨迹 (device $\rightarrow$ stable)

图元：圆 = 配置; 虚线箭头 = 转移名; 双圆 = 接受。回答「这次怎么走」; 不是整库拷贝。

图 3.16: 依赖、展开与进度 (图元不同): 嵌套数据流 / 菱形能力依赖 / 圆形作业进度

读图可以记一句：层 A 看字节与调用；层 B 看能不能引用；层 C 看这次怎么获证。三者若都画成圆角方框，最容易串层——本图刻意用椭圆 / 菱形 / 圆拆开。

流程五：尾缝与 2-launch——组合并不免费 对照表 2.2 和图 3.15: 「PKE 已经在 Γ 里」加上「哈希、拼装片段也能对拍」，推不出「接好的 KEM.KeyGen 自动就合法」。尾缝必须当成新节点，先走 Compose/Probe，再 Certify——也就是说，可以找到 $x, y \in L$ 使得 $xy \notin L$ (§3.9)。

流程六：correctness——缺的是成员判定 图 3.15 里灰框旁路写着 $\text{oracle} \neq L$: 伪代码加菜单，最多提示一下 Σ ; 没有成员判定时，搜索空间几乎就是 Σ^* 。correctness 能证明字节对得上，证明不了那条路是合法派生。

流程七：晋级——就是一次配置转移 合法叙事对应一条配置轨迹（和图 3.16 的层 C 一致）：

$$(\Gamma_0, F_0, S_0) \xrightarrow{\text{Expand}} \dots \xrightarrow{\text{Probe/Compose}} \dots \xrightarrow{\text{Certify}} (\Gamma_m, \emptyset, S_m), \quad T \in \Gamma_m.$$

若中途引用了禁字集 Forbidden 中的因子（已关闭或明确禁止的路线），应直接拒绝（Freeze / 沉入拒绝阱）。

流程八：上游一变——Dirty 若某已获证的上游能力（例如 P1）对外约定变更——I/O、布局或同步约定不再与获证时一致——则原先依赖该约定的下游（P2/P3）不能因为「上次 PASS」就继续留在 Γ 里；必须走 Dirty (§3.11)。

表 3.6: §2.6 四组问题的本章重述（对照表 2.3；工程词为主，括号附已挂钩记号）

问题组	第 2 章里的疑惑（缩写）	用本章对象怎么说
第一组	凭目录名，还是凭证书，才算站在 PKE 上？菜单等于说明书吗？	站得住 $\Leftrightarrow$ 相关能力已获证 ($\in \Gamma$)；菜单 $\neq$ 已获证清单 $\neq$ 合法计划集合。
第二组	NTT、PKE 都 PASS 了，谁来保证尾缝和 2-launch？	组合并不免费：接缝还得经声明接缝并获证；语义对照过关 $\neq$ 拼装计划合法。
第三组	只给伪代码和菜单，搜索为什么会炸？	没有成员检查，搜索空间接近任意拼接；要收束，就只能沿着已获证边 / 合法计划往下长。
第四组	能不能先交闭包表？上游改了，证书怎么作废？	闭包表至少写清：目标、已获证、依赖闭包、缺项；上游一变，打脏标记。

四组问题：换成本章用语再说一遍

读完本节，带走三句话就够

1. 表 3.5、图 3.15、图 3.16 是第 2 章故事的**数学完全体**；原来的表和图仍在第 2 章备查。
2. 最容易搅在一起的三层是：数据流、能力库、一次派生——先分开，再提问。
3. 四组问题到本章已经能**把话说清楚**；真正变成任务启动时的硬检查，是下一章起的工程化工作。

3.7 枢纽：依赖、展开与进度各由哪件工具管

三件工具的数学与工程落点都写过了。收束前再钉死一句读章枢纽：**依赖、展开与进度是三件不同的事**，并分别由三件工具接管——**依赖**归依赖图与依赖闭包，**展开**归形式语言，**进度**归作业过程自动机。下面用三个定义把对象名钉死，并对照它们各回答什么、不回答什么。

定义 3.58 (能力 DAG (库)). **能力 DAG** 是 DAG $G_{\text{lib}} = (V_{\text{lib}}, E_{\text{lib}})$ ，顶点为可命名、可获证的能力，边为依赖（语义依赖或已声明接缝依赖）。它描述引理库的**静态**结构，不随单次任务进度改写边集（结点证书状态可变，见配置层）。

定义 3.59 (一次任务与派生对象). 固定目标 $T \in V_{\text{lib}}$ 与文法 $\mathcal{G}$ (见形式语言一节). 一次任务指试图使 T 获证的一次预研作业. 该作业中, 从开始符号出发按产生式改写所得的符号串序列称为**派生**; 其树形记录称为**派生树**. 派生树是 G_{lib} 上依赖信息的一次**使用痕迹**, 不是整库拷贝。

定义 3.60 (配置轨迹). 作业过程自动机上的状态序列 $q_0 \rightarrow q_1 \rightarrow \cdots \rightarrow q_m$, 其中每个 q_i 为配置 (Γ_i, F_i, S_i) (见作业过程自动机一节), 称为**配置轨迹**. 它记录证书如何增减、前沿如何消解——即「进度」。

命题 3.10 (三件工具分工).

对象	归属工具	回答	不回答
能力 DAG	依赖图与闭包	库中谁依赖谁	某次任务如何逐步展开
派生树	形式语言	该次任务的展开句子 / 树形	全局共享边的全貌
配置轨迹	作业过程自动机	证书如何增减; 接受/拒绝	输出是否符合 <i>FIPS</i> 字节

依赖归依赖图; 展开归形式语言; 进度归作业过程自动机。

例 3.27 (纯数学: 引理库 vs 一次证明树). 下面用**左右对照**两幅子图 (同排并置, 不是上下叠放): 左边是共享的引理库 DAG; 右边是证明 T_1 时实际用到的派生树. 库中的 C 可被许多证明共享, 但右图只出现本次用到的边. (子图内部依赖边仍自上而下画, 勿把「左右对照」误读成「整图是上下结构」.)



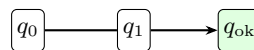
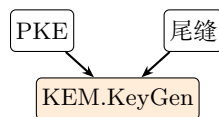
图 3.17: 配图: 纯数学: 引理库 vs 一次证明树 (左右并置)

例 3.28 (工程: 三层对照图). 下面这张是**上下结构** (不是左右两分): 上栏画能力库的依赖直觉; 下栏再左右分开——下左是本次 KeyGen 的一次派生, 下右是配置轨迹示意。

上: 库 (INDEX 依赖直觉)



下左: 一次派生 (本次 KeyGen) 下右: 配置轨迹 (示意)



Γ 增大 / F 消解

图 3.18: 配图: 工程: 三层对照图 (上: 库; 下左: 派生; 下右: 轨迹)

3.8 本章小结与下一章预告

§ 3.1 导论先交代方法论面向的场景，再交代工程策略与三件工具；随后用三大节分别写完数学：

$$\underbrace{\text{依赖图与依赖闭包}}_{\S 3.2} \rightarrow \underbrace{\text{形式语言（合法拼装）}}_{\S 3.3} \rightarrow \underbrace{\text{作业过程自动机}}_{\S 3.4}$$

再写工程体现、回扣 KeyGen，并以枢纽节收束「依赖 / 展开 / 进度」的分工。迷路时先回表 3.2 与图 3.1。有意留白：产生式在仓库里如何形式化、闭包表文件长什么样、可判定性如何证——留给后文。

下一章（第 4 章）要把这些对象填成工程上可写的字段：节点属性、三类边、证书状态怎么命名，等等。

4 从数学到工程对象：节点、边与证书

第 3 章已经给出依赖闭包 Dep^* 、已获证集 Γ 、缺项 Missing_Γ 、禁止拼法 Forbidden ，以及转移 $\text{Expand/Probe/Certify/Dirty}$ 。本章不再只堆工程口诀，而是把第 3 章对象当作算法的参数与子程序来构造；同时把算法的输出长什么样用表格钉死——推导与表单缺一不可。

写法约定：每个算法固定四要素——参数、输入、输出、步骤；

参数挂第 3 章数学对象；步骤后给出命题/推论；

凡输出是「卡 / 表 / 栏」，正文必须给出字段表与至少一张填满的实例表。

门禁四步流与「先交闭包表再写码」的完整操作面在第 5 章，那里会继续调用本章算法作为子程序。

4.1 推导用到的第 3 章工具箱

下文算法默认携带下列参数（均已在第 3 章定义；此处只列接口）。第三列标明它属于第 3 章哪一件数学工具——依赖图 / 依赖闭包、形式语言、或作业过程自动机。

算法参数体例（强制） 参数栏里写出的每个数学对象，必须在步骤中被读取或改写；禁止「参数写了 $G = (V, E)$ ，步骤却只用 V 」这类无效输入。若某子程序确实不消费边，则参数只声明顶点集 V （或其它真实用到的对象），完整图留给调用闭包/边分类的算法。记法 $G = (V, E)$ 表示参数是整图 G ；此时步骤须用到 G ，或显式用到其分量 V 与 E 。若步骤只经子程序传入整图 G ，则参数栏写 G 即可，不必再把未直接点名的 V, E 拆成独立参数。

数学对象	第 3 章工具	本章用法
能力 DAG $G = (V, E)$	依赖图	顶点 = 可命名能力；边 = 依赖/接缝关系的载体
$\text{Dep}_G^*(T)$ 、迭代 $D_k(T)$	依赖闭包	算「完成 T 理论上要齐谁」；步骤直接调用第 3 章迭代
$\Gamma \subseteq V$	依赖闭包 （已获证）	当前已获证集；证书状态是 Γ 的精细标签；亦作为作业过程自动机配置的一分量
$\text{Missing}_\Gamma(T) = \text{Dep}^*(T) \setminus \Gamma$	依赖闭包 （缺项）	缺项；闭包表与补洞的核心输出
Forbidden、约束库 S 中的禁拼片段	形式语言	否定边 / 禁止拼法的来源；成员检查时避开 Forbidden
配置 $q = (\Gamma, F, S)$ 与基本转移	作业过程自动机	解释「写回 / 打脏」在改哪一分量；接受/拒绝由转移给出

4.2 由顶点导出节点卡

第 3 章把证书主语定为顶点 $v \in V$ ，但未规定工程上如何记录 v 。下面的算法把「 v 可被引用」所需的检查信息，编码成一张有限字段卡。

下面把「顶点 $\mapsto$ 节点卡」写成可执行手续。体例上：本算法**只吃**顶点集与约束，**不吃**边集——边的分类见算法 4.2，闭包沿边展开见算法 4.3；调用方若持有完整 DAG，此处只传入其顶点部。

算法 4.1：编码节点卡 EncodeNode

参数：能力顶点集 V ；分层映射 $\lambda: V \rightarrow \{P0, P1, P2, P3\}$ ；当前已获证集 $\Gamma \subseteq V$ ；约束库 S （含与各顶点相关的 Forbidden 片段）；仓库锚点映射 $\rho: V \rightarrow \text{Paths}$ （活跃权威路径，禁止指向 **frozen**/ 正向源码）。

输入：顶点 v 。

输出：节点卡 $\text{Card}(v) = (\text{id}, \text{layer}, \text{interface}, \text{cert}, \text{forbid}, \text{repo})$ ；若前置检查失败则拒绝输出。

- 1: 前置（用到 V ）：**若 $v \notin V$ ，拒绝并报错「未知能力顶点」。
- 2: 令 $\text{id}(v)$ 为 v 的稳定名字（跨任务可引用；不得用临时目录名冒充）。**
- 3: 令 $\text{layer}(v) = \lambda(v)$ （P0–P3 只描述能力角色，不要求与磁盘树同构）。**
- 4: 从 S 中抽取 v 对外可复用的 I/O、dtype、布局与同步约定，写入 $\text{interface}(v)$ 。**约定：后续边与引用只允许挂 **interface**，不允许默许「拷源码」。
- 5: 若 $v \in \Gamma$ ，令 $\text{cert}(v)$ 记录获证证据指针（路径、CPU+SIM/KAT、版本锚点）；若 $v \notin \Gamma$ ，令 $\text{cert}(v) = \perp$ 。**
- 6: 令 $\text{forbid}(v)$ 为 S 中与 v 相关的否定约束（可空）。**
- 7: 令 $\text{repo}(v) = \rho(v)$ ；若 $\rho(v)$ 落在 **frozen**/ 正向实现路径，则拒绝输出并报错（否定说明可引用 **frozen** 判决，正向 **repo** 不可）。**
- 8: 返回六元组 $\text{Card}(v)$ 。**

输出长什么样：节点卡字段表 算法 4.1 的输出不是抽象六元组口号，而是下面这张可填字段卡（名字可本地化，语义不要丢）：

字段	含义与填写纪律
<code>id</code>	稳定名字，跨任务可引用（如 <code>pke.keygen.k4</code> 、 <code>kem.keygen.k4</code> ）。不要用「这次 PR 的临时文件夹名」当 <code>id</code> 。
<code>layer</code>	方法论分层 P0–P3（见表 4.1）。分层描述能力角色， 不必 与磁盘目录一一对应。
<code>interface</code>	I/O、dtype、形状/布局、同步与 <code>launch</code> 约定中， 对外可复用的那部分 。能挂到边上的是 <code>interface</code> ，不是「把源码拷过来」。
<code>cert</code>	证书证据：权威路径 + 验收级别（CPU+SIM，必要时 KAT）+ 版本锚点（ <code>commit</code> / <code>tag</code> / <code>STATUS</code> 日期）；未获证时填 $\perp$ 或「未获证」。
<code>forbid</code>	与本节点相关的否定约束（例如「不得以某某 <code>frozen</code> 为模板」）。可空，但有则必须可检查。
<code>repo</code>	权威目录锚点（活跃 <code>stable-*</code> / <code>pass-*</code> / 登记表条目等）。 <code>frozen</code> 目录可以出现在否定说明里， 不得 充当正向 <code>repo</code> 。

表 4.1: 分层语言 P0–P3 (λ 的建议取值)

层	含义（放在本仓库语境里说）
P3	顶层算子 / 算法入口（如 <code>KEM.KeyGen</code> 、 <code>KEM.Encaps</code> 、 <code>KEM.Decaps</code> ）
P2	算法构件（如 <code>PKE.KeyGen</code> / <code>Encrypt</code> / <code>Decrypt</code> 全链）
P1	领域原语（ <code>NTT</code> 、 <code>basemul</code> 、 <code>CBD</code> 、 <code>ByteEncode</code> 、 <code>SHAKE...</code> ）—— 当能力名用 ；本章不讲其代数细节
P0	平台公理（ <code>DataCopy</code> 、向量算子、 <code>MMAD+tiling</code> 、 <code>TPipe/TQue</code> 同步壳...）

命题 4.1 (节点卡是门禁检查的充分记录). 固定 G, Γ, S 。对任意 $v \in V$ ，由算法 4.1 得到的 $\text{Card}(v)$ 足以判定：

1. v 当前是否已获证（看 $\text{cert}(v) \neq \perp$ ，等价于 $v \in \Gamma$ ）；
2. 引用 v 时对外应遵守的接口（`interface`）；
3. 是否触碰与 v 相关的禁止项（`forbid`）。

换言之：门禁若只关心「能否引用 v 」，不必再翻开 v 的实现源码。

证明梗概. 第 3 章定义「已获证」为 $v \in \Gamma$ ，算法 4.1 中写入 `cert` 的步骤正是该隶属关系的工程编码；`interface` 与 `forbid` 直接截取自约束库 S ，而合法引用在定义上只依赖证书资格与 S 中接口/禁止约束。□

推论 4.1 (禁止同任务发明未获证底层). 若目标 T 满足 $\lambda(T) = \text{P3}$ ，且存在 $u \in \text{Missing}_\Gamma(T)$ 使 $\lambda(u) \in \{\text{P0}, \text{P1}\}$ ，则任何声称「在做 T 的同一任务里顺便实现 u 并立刻引用」的计划，都与「先 `Probe`/`Certify` 再引用」的合法转移相悖（见第 3 章基本转移）。

例 4.1 (填满的节点卡: `pke.keygen.k4`). 续第 2 章。对 $v = \text{pke.keygen.k4}$ 跑算法 4.1 ($\lambda(v) = \text{P2}$, $v \in \Gamma$)，输出应能填成下表（非正式 SCHEMA，演示「填得满」）：

字段	示例填写
id	pke.keygen.k4
layer	P2
interface	与 stable PKE KeyGen 交付 I/O 一致；NTT LUT 静态约定；设备侧布局与后续 KEM 尾拼兼容的那部分写清
cert	examples/stable/ml-kem/ml-kem-1024/stable-fips203-mlkem-pke-keygen-k4/；CPU+SIM 对拍；STATUS / 交付纪要作版本锚点
forbid	不得把 liboqs C 源码当作 AscendC 实现规格；不得从 frozen/ 抄实现
repo	上述 stable 目录（及对应 baseline-registry 条目，若有）

当 KEM.KeyGen 引用它时，引用的是这张卡上的 `interface+cert`，不是「目录里好像有个绿名」。

4.3 由边关系导出三类边

第 3 章的依赖边与接缝、禁止拼法在工程上常混称「关系」。下面算法把一次「关系声明」分类，便于写入 E 或 Forbidden。

算法 4.2：边分类 ClassifyRel

参数：能力 DAG $G = (V, E)$ （本算法可读 V 、写 E ，从而更新 G ）；禁止拼法族 Forbidden；接缝声明集 $\mathcal{C}_{\text{seam}} \subseteq V \times V$ （由约束库维护：哪些「 A 与 B 已各自获证仍须单独认证」）。

输入：关系请求 r ，形态为以下之一：(i)有序对 (u, v) 表示「 v 语义依赖 u 」；(ii)接缝请求 $\text{compose}(A, B)$ ；(iii)禁止请求 $\text{forbid}(x)$ （关闭路线、禁模板等）。

输出：标签 $\tau(r) \in \{\text{semantic}, \text{compose}, \text{forbidden}\}$ ，以及应写入的载体（ G 的一条边、接缝节点候选、或 Forbidden 的一条）；失败则拒绝。成功后调用方持有的图为更新后的 G 。

- 1: 若 $r = \text{forbid}(x)$ ，令 $\tau = \text{forbidden}$ ，把 x 并入 Forbidden（或写入对应 `forbid` 字段），返回。
- 2: 若 $r = \text{compose}(A, B)$ ，或 $(A, B) \in \mathcal{C}_{\text{seam}}$ ：若 $A \notin V$ 或 $B \notin V$ ，拒绝；否则令 $\tau = \text{compose}$ ：新建或引用接缝顶点 $s_{A \circ B}$ ；若 $s_{A \circ B} \notin V$ ，则更新图 G ：令 $V \leftarrow V \cup \{s_{A \circ B}\}$ ；要求日后单独 Probe/Certify；不把「 A 与 B 均已获证」推成 $s_{A \circ B}$ 已获证。
- 3: 否则若 $r = (u, v)$ ：若 $u \notin V$ 或 $v \notin V$ ，拒绝；若任务语义为「 v 的正确性依赖 u 的 interface」，令 $\tau = \text{semantic}$ ，并更新图 G ：令 $E \leftarrow E \cup \{(u, v)\}$ （若尚无该边）。
- 4: 返回 $\tau(r)$ 与载体（及更新后的 G ，若本步改写了 V 或 E ）。

输出长什么样：三类边含义表

标签 τ	含义	写入何处 / 后果
semantic	v 要对，离不开 u 的 interface	进入 E ；参与 Dep*
compose	A, B 各有证， $A \circ B$ 仍须单独认证	接缝顶点 s ；不因 $A, B \in \Gamma$ 自动获证
forbidden	明确禁止的路线/模板	进入 Forbidden 或节点 <code>forbid</code>

命题 4.2 (组合不免费的算法表述). 设 $A, B \in \Gamma$ 且 $\text{compose}(A, B)$ 被算法 4.2 标为 compose , 接缝顶点为 s 。则

$$A \in \Gamma \wedge B \in \Gamma \not\Rightarrow s \in \Gamma.$$

要使 $s \in \Gamma$, 必须另经合法的 Probe/Certify (或等价验收) 写入证书。

证明. 由算法步骤 2: A, B 获证只保证接缝请求可被提出, 不修改 Γ 对 s 的隶属; 第 3 章 Certify 才是唯一把顶点写入 Γ 的基本转移 (在约束允许时)。□

注 4.1 (边 $\neq \#include$). semantic/compose 边描述的是能力—证书关系, 不是编译依赖。自包含原则写进 interface: 对外挂接口; 共享库例外必须进入 S , 不能默许 Agent 自由翻抄。

例 4.2 (KeyGen 上的三类边 (填表示意))。

标签	实例	说明
semantic	$\text{kem.keygen.k4} \leftarrow \text{pke.keygen.k4}$	P3 语义依赖已获证 P2
compose	Alg.16 尾拼 / dk 展开	PKE 证书覆盖不了, 须单独验收
forbidden	禁止 frozen / correctness vendor 模板	写入 Forbidden 或 forbid

4.4 由闭包定义导出缺项算法

第 3 章已有 Dep^* 的迭代收敛定理与 Missing_Γ 定义。这里把它们写成可调用子程序, 供第 5 章闭包表算法使用。

算法 4.3: 依赖闭包 DepStar

参数: 有限依赖图 $G = (V, E)$ (第 3 章: V 为顶点集, E 为依赖边集)。

输入: 目标 T (或目标集 A , 逐点并)。

输出: $D = \text{Dep}_G^*(T)$; 若 $T \notin V$ 则拒绝。

- 1: 在输入图 $G = (V, E)$ 上检查: 若 $T \notin V$, 拒绝。
- 2: 令 $D_0 = \{T\}$ 。对任意 $v \in V$, 记前驱 $\text{Pred}_G(v) = \{u \in V \mid (u, v) \in E\}$ (显式读取边集 E)。
- 3: 对 $k = 0, 1, 2, \dots$, 令 $D_{k+1} = D_k \cup \bigcup_{v \in D_k} \text{Pred}_G(v)$, 直至 $D_{k+1} = D_k$ 。
- 4: 返回 D_k (第 3 章已证: 稳定值等于 $\text{Dep}_G^*(T)$, 且步数 $\leq |V|$)。

算法 4.4: 缺项 MissingOf

参数: 有限依赖图 G ; 当前已获证集 Γ 。

输入: 目标 T 。

输出: $M = \text{Missing}_\Gamma(T)$ 。

- 1: 将同一图 G 作为参数调用算法 4.3, 得 $D = \text{Dep}_G^*(T)$ (若 T 不是 G 的顶点, 则随 DepStar 拒绝)。
- 2: 返回 $M = D \setminus \Gamma$ 。

命题 4.3 (缺项判定的直接推论). 算法 4.4 输出 $M = \emptyset$ 当且仅当 $\text{Dep}^*(T) \subseteq \Gamma$ (即第 3 章「闭包已满」)。

4.5 由非单调证书导出脏标记传播

第 3 章指出 Γ 可因 Dirty 收缩。工程上需要可计算的作废范围。

算法 4.5: 脏标记传播 DirtyPropagate

参数: 能力 DAG $G = (V, E)$; 当前已获证集 $\Gamma \subseteq V$ 。

输入: 接口/约定发生变更的顶点集合 U 。

输出: 新的已获证集 Γ' , 以及被打脏的集合 Δ ; 若 $U \not\subseteq V$ 则拒绝。

- 1: 在输入图 $G = (V, E)$ 上检查: 若 $U \not\subseteq V$, 拒绝。
- 2: 令 $\Delta_0 = U$ 。对任意 $u \in V$, 记后继 $\text{Succ}_G(u) = \{w \in V \mid (u, w) \in E\}$ (显式读取边集 E 的出邻)。
- 3: 迭代: 令 $\Delta_{k+1} = \Delta_k \cup \bigcup_{u \in \Delta_k} \text{Succ}_G(u)$, 直至稳定, 得 Δ (变更锥: 以 U 为种子、沿出边在 G 上可达的全部顶点)。
- 4: 令 $\Gamma' = \Gamma \setminus \Delta$, 返回 (Γ', Δ) 。

输出长什么样: 证书状态表 工程交接时, 对每个顶点除「是否在 Γ 」外, 再贴粗标签 σ (不改变 Missing 的集合定义):

状态 σ	工程含义
unproven	只有名字或愿望, 尚无按约定跑通的证据 ($v \notin \Gamma$)。
probe	探针级: 平台/接缝行为已用 CPU+SIM (或任务声明级别) 跑通; $v \in \Gamma$, 弱证书。
lemma	可稳定引用; interface 写清, 可进入依赖闭包的「已获证」栏。
deliverable	交付级 (如 stable): 在 lemma 之上还有交付目录、登记表与交接文书。
dirty	作废标记: 落入算法 4.5 的 Δ 后, $v \notin \Gamma'$; 旧对拍不得再充当有效 cert。

阶梯关系可记为:

$$\text{unproven} \longrightarrow \text{probe} \longrightarrow \text{lemma} \longrightarrow \text{deliverable}, \quad \text{以及回退} \quad \text{dirty}.$$

命题 4.4 (证书阶梯与 Γ 的关系). 为工程交接, 对 $v \in \Gamma$ 可再贴标签 $\sigma(v) \in \{\text{probe}, \text{lemma}, \text{deliverable}\}$ (未获证则 $\sigma(v) = \text{unproven}$)。则:

1. 门禁「可引用」的初级判定只依赖 $v \in \Gamma$, 与 $\sigma(v)$ 的粗细无关;
2. σ 只影响验收级别与交付文书, 不改变 Missing_Γ 的集合定义;
3. 一经算法 4.5, $v \in \Delta \Rightarrow v \notin \Gamma'$, 此时对拍旧日志不得再充当有效 cert。

注 4.2 (对拍 $\neq$ 证书全部). 对拍通过是 Certify 的必要证据之一, 不是充分条件: 仍须引用合法、接缝已认证、未命中 Forbidden、未落入 Δ 。

4.6 由投影关系导出「菜单 $\neq$ 说明书」

命题 4.5 (菜单不蕴含闭包已满). 令 Menu 为活跃 INDEX 绿项对应的顶点集。则即使 Menu 很大, 一般**不能**推出 $\text{Missing}_\Gamma(T) = \emptyset$ 。必须先跑算法 4.3–4.4。(此即第 3 章「菜单、 Γ 与闭包」性质的算法版。)

仓库机制与数学对象的对照, 由此成为**推论式读法**而非平行口号:

仓库机制	对应数学 / 算法输出	误读
活跃 INDEX	Menu ; 构造 Γ 的候选	当成语言 L 本身
frozen /+ 判决	Forbidden 的来源 (算法 4.2 的 forbidden)	当成可抄实现库
baseline-registry	golden 允许块 $\subseteq$ 已验证计算证据	AscendC 必须同构的规格
customspec	计划文书, 待第 5 章与闭包表分工	可跳过 MissingOf
STATUS / 对拍	cert 证据片段	永久执照 (无视 Dirty)

4.7 本章小结：本章推出了什么

1. 算法 4.1–4.5 把第 3 章的 $V, E, \Gamma, \text{Dep}^*, \text{Missing}, \text{Forbidden}, \text{Dirty}$ 变成可执行构造; 命题 4.1–4.5 是对应结论。
2. 关键结论: 节点卡是引用检查的充分记录; 组合不免费; 缺项空 $\Leftrightarrow$ 闭包已满; 脏传播使 Γ 非单调; 菜单不蕴含 L 成员。
3. 下一章把这些算法嵌进门禁: 闭包表 = 目标声明 + DepStar + MissingOf + 边分类, 再导出「何时允许写码」。

5 门禁与工作流：Agent 被允许做什么

第 4 章给出了从数学对象到节点/边/证书的构造算法。本章继续推导: 把第 3 章的作业过程自动机转移, 写成 Agent 可执行的门禁算法, 并推出「何时允许写码」的判定式。

门禁的数学含义: 只允许作业过程自动机上的合法转移;

门禁的操作面: 先交闭包表, 再写码; 缺项先补洞; 禁止项不得当模板。

5.1 闭包表算法

第 3 章定义闭包表至少含 $A, \Gamma, \text{Dep}^*(A), \text{Missing}_\Gamma(A)$ 。下面把它变成算法输出, 并加上接缝与否定栏 (仍由第 3 章对象生成)。

算法 5.1: 填写闭包表 BuildClosureTable

参数: 能力 DAG $G = (V, E)$; 当前已获证集 $\Gamma \subseteq V$; 约束库 S (含 Forbidden 、接缝声明 C_{seam}); 分层 $\lambda: V \rightarrow \{P0, P1, P2, P3\}$; 状态标签 σ (命题 4.4; 用于识别已打脏但仍可能误留在文书里的点)。子程序: 算法 4.3、4.4、4.2、4.1。

输入：目标 T ；I/O 契约 $\mathcal{C}$ ；声明验收级别 $\ell \in \{\text{probe}, \text{lemma}, \text{deliverable}\}$ 。

输出：闭包表 $\text{CT} = (T, \mathcal{C}, \ell, \Gamma, D, M, \text{Seams}, \text{ForbiddenHit}, \text{Warn})$ ，其中 $D = \text{Dep}_G^*(T)$ ， $M = \text{Missing}_\Gamma(T)$ ；Warn 为警告袋（可空）。

- 1: **目标声明：**在图 $G = (V, E)$ 上，若 $T \notin V$ 或 $(T, \mathcal{C}, \ell)$ 任一项缺失，拒绝输出（任务未声明）。否则写入 $(T, \mathcal{C}, \ell)$ 。
- 2: 从约束库 S 取出本任务可见的 Forbidden 与 $\mathcal{C}_{\text{seam}}$ （后续接缝栏与否定栏只读这两份摘录，但仍以 S 为唯一来源——禁止另起一份未登记的禁止表）。
- 3: 以参数 G 调用 $\text{DepStar}(T)$ 得 D （该子程序沿 E 取前驱）；以参数 G, Γ 调用 $\text{MissingOf}(T)$ 得 M 。
- 4: 对每张需要引用的 $v \in D \cap \Gamma$ ：以 G 的顶点集 V 等参数调用 $\text{EncodeNode}(v)$ ，核对 $\text{cert} \neq \perp$ ；若维护了 σ 且 $\sigma(v) = \text{dirty}$ ，则把 v 移出「可引用已获证」视角并记入 Warn。
- 5: **接缝栏：**对所有与 T 相关的 compose 请求，以参数 $G, \text{Forbidden}, \mathcal{C}_{\text{seam}}$ 跑 ClassifyRel ，收集尚未获证的接缝顶点集 Seams；把 $\text{Seams} \setminus \Gamma$ 并入缺项视角（实现上可并入 M 或单列，但门禁必须看见它们）。
- 6: **分层警告（用到 λ ）：**若存在 $u \in M$ 使 $\lambda(u) \in \{\text{P0}, \text{P1}\}$ ，将「禁止在做 T 的同一任务里顺便发明该底层」写入 Warn（对照推论「禁止同任务发明未获证底层」）。
- 7: **否定栏：**计算 $\text{ForbiddenHit} = \{x \in \text{Forbidden} \mid x \text{ 与本次计划相交}\}$ ；若计划显式引用 ForbiddenHit 中模板，标记为硬冲突。
- 8: 返回 CT。

输出长什么样：闭包表栏位 算法 5.1 输出的 CT 在工程上至少填成下表各栏（第 3 章四元组 + 接缝/否定扩展）：

栏	填什么
目标 T	id、I/O 黑盒 $\mathcal{C}$ 、声明验收级别 $\ell \in \{\text{probe}, \text{lemma}, \text{deliverable}\}$
已获证	$D \cap \Gamma$ 中节点（附节点卡 repo /版本锚点）；须未脏
依赖闭包 D	$\text{Dep}^*(T)$ ：语义依赖 + 已知接缝项所覆盖的先决
缺项 M	$\text{Missing}_\Gamma(T)$ ；每项可附「拟补洞手段」
接缝 Seams	须单独认证的接缝顶点；未获证者门禁必须看见
否定	本次计划与 Forbidden 的相交；非空则硬冲突
ForbiddenHit	
警告 Warn	可空；含脏证书误留、同任务发明 P0/P1 缺项等非硬冲突提示

先交闭包表，再写码。

customspec、STATUS、baseline-registry 是重要文书，但**替代不了** CT：前者多半描述「打算怎么实现 / 实现结果如何」，CT 回答「凭什么现在允许开始实现」。

定理 5.1 (写码许可的充分必要条件 (v0))。固定 G, Γ, S 与目标 T 。在算法 5.1 输出的 CT 上，定义谓词

$$\text{PermitCode}(\text{CT}) : \iff M = \emptyset \wedge \text{Seams} \subseteq \Gamma \wedge \text{ForbiddenHit} = \emptyset \wedge \mathcal{C} \text{ 已声明}.$$

则：

1. 若 $\text{PermitCode}(\text{CT})$ 为真，则相对 Γ 目标闭包已满、接缝已获证、未踩禁止拼法，此时进入「按 ℓ 实现 T 」与第 3 章「闭包已满后方可冲击顶层」一致；
2. 若 $M \neq \emptyset$ 或 $\text{Seams} \not\subseteq \Gamma$ ，则任何跳过 Probe/Certify、直接写 T 顶层实现的动作，都不是合法转移（应被门禁拒绝）；
3. 若 $\text{ForbiddenHit} \neq \emptyset$ ，则计划字触碰 Forbidden，属于第 3 章工作定义下的幻觉/拒绝类行为。

证明梗概. (1) $M = \emptyset \Leftrightarrow \text{Dep}^*(T) \subseteq \Gamma$ 为第 3 章命题；接缝与 Forbidden 条件分别对应 Compose 已写入证书与避开禁止拼法。(2) (3) 直接对照第 3 章基本转移与合法性定义：未获证引用与命中 Forbidden 均为非法转移。□

例 5.1 (填满的闭包表示意：KEM.KeyGen).

栏	示例
目标	<code>kem.keygen.k4</code> ；ek/dk 字节；验收 $\ell = \text{deliverable}$ (CPU+SIM，必要时 KAT)
已获证	<code>pke.keygen.k4</code> (stable 节点卡)；相关 P1 在 PKE 获证时已闭合
依赖闭包	PKE.KeyGen + KEM 相对 PKE 的尾缝 (2-launch / Alg.16 / dk 展开等)
缺项	若尾缝尚未单独获证 $\Rightarrow$ 非空；不得在「写 KEM」任务里顺便重写 NTT
接缝	Alg.16 / dk 等接缝顶点；未获证则 $\text{PermitCode} = \text{false}$
否定	不得以 frozen / correctness vendor 为模板；不得把 liboqs C 当 AscendC 规格

若尾缝尚在 $\text{Seams} \setminus \Gamma$ ，则许可为假——即使 PKE 已绿。

5.2 门禁判定与最小补洞

算法 5.2：门禁判定 GateCheck

参数：本算法只读闭包表 CT 的各栏（不另取 G ）。若输入尚不是表，则携带与算法 5.1 相同的参数，先调用之得到 CT。

输入：闭包表 CT（或先输入 T, C, ℓ 再内生调用 BuildClosureTable）。

输出：ALLOW（允许按 ℓ 写 T ）或 BLOCK(R)（拒绝原因袋 R ）。

- 1: 若输入不是完整 CT，则用本算法参数调用算法 5.1 生成 CT；失败则 BLOCK($\{\text{no_table}\}$)。
- 2: 若 CT 未完整，返回 BLOCK($\{\text{no_table}\}$)。
- 3: 若 $\text{ForbiddenHit} \neq \emptyset$ ，返回 BLOCK($\{\text{forbidden}\} \cup \text{ForbiddenHit}$)。
- 4: 若 $M \neq \emptyset$ 或 $\text{Seams} \not\subseteq \Gamma$ ，返回 BLOCK($\{\text{missing}\} \cup M \cup (\text{Seams} \setminus \Gamma)$)。
- 5: 否则返回 ALLOW。

算法 5.3：最小补洞一轮 MinimalFillRound

参数：有限依赖图 G ；当前已获证集 Γ （选取缺项生成集时必须相对 G 上的 Dep_G^* 判定）。

输入：目标 T ；当前 $M = \text{Missing}_\Gamma(T)$ （可含未获证接缝）。

输出：本轮补证对象集合 $B \subseteq M$ ，以及建议的动作序列（对每个 $u \in B$ ：Probe 然后条件性 Certify）。

- 1: 若 $M = \emptyset$, 返回 $B = \emptyset$ 。
- 2: 以参数 G 调用 $\text{DepStar}(T)$ 得 $D = \text{Dep}_G^*(T)$ 。
- 3: 在 M 中选取 $B \subseteq M$, 使 $\text{Dep}_G^*((D \cap \Gamma) \cup B) = \text{Dep}_G^*(T)$ (即第 3 章「相对 Γ 的缺项生成集」; 优先较小者; 有限图 G 上极小生成集存在)。
- 4: 对每个 $u \in B$: 输出动作「Probe(u); 验收通过则 Certify(u) 使 $\Gamma \leftarrow \Gamma \cup \{u\}$ 」。
- 5: 禁止把 B 选成「整份 T 顶层实现」来冒充补洞——补洞对象应是缺项, 不是跳过闭包。

推论 5.1 (补洞单调消缺项). 若一轮补洞成功得到 $\Gamma' \supseteq \Gamma$ 且 $B \subseteq \Gamma'$, 则 $\text{Missing}_{\Gamma'}(T) \subseteq \text{Missing}_{\Gamma}(T)$ (第 3 章缺项基本性质 3)。反复执行算法 5.3 直至 $M = \emptyset$, 在有限 V 上必终止。

5.3 四步工作流 = 外层算法

把「声明—展开—补洞—写回」写成调用上述子程序的外层算法。

算法 5.4: 任务工作流 TaskWorkflow

参数: 作业过程自动机的合法转移集 (第 3 章); 本章算法 5.1–5.3; 脏传播算法 4.5。

输入: 目标 T ; I/O 契约 $\mathcal{C}$; 级别 ℓ 。

输出: 接受 ($T \in \Gamma$ 且前沿清空) 或拒绝/冻结; 以及写回后的 Γ 。

- 1: **目标声明:** 固定 $(T, \mathcal{C}, \ell)$ 。
- 2: **闭包展开:** $\text{CT} \leftarrow \text{BuildClosureTable}(T, \mathcal{C}, \ell)$ 。
- 3: **门禁:** 若 $\text{GateCheck}(\text{CT}) = \text{BLOCK}(R)$, 则对 R 中缺项跑 MinimalFillRound , 每成功一次更新 Γ 并回到步骤 2; 若命中 forbidden, 执行 Freeze / 拒绝, 停止。
- 4: **允许写码:** 当 $\text{GateCheck} = \text{ALLOW}$ 后, 按 ℓ 实现 T ; 验收通过则 $\text{Certify}(T)$, 写回 INDEX/STATUS/节点卡。
- 5: **非单调维护:** 若随后某 u 的 interface 变更, 调用 $\text{DirtyPropagate}(\{u\})$ 更新 Γ , 并要求依赖锥内任务重新进入步骤 2。

输出长什么样: 四步一览 把算法 5.4 收成可扫读的表 (任一步未完成, 不得进入「大规模写码 / 开放试错」):

步	名称	做什么 / 完成标准
1	目标声明	写清 T 、I/O、验收级别; 没有级别声明 = 未声明任务。
2	闭包展开	跑 BuildClosureTable ; 标 ok / missing / dirty; 接缝与否定栏齐。
3	最小补洞	对 BLOCK 中的缺项跑 MinimalFillRound ; forbid 落笔。
4	闭环写回	$\text{Certify}(T)$ 入 INDEX/STATUS/节点卡; 失败则 Freeze; 该脏则 DirtyPropagate 。

与第 3 章转移的对照:

工作流动作	大致对应
闭包展开	Expand: 前沿与缺项暴露出来
探针 / 补洞取证	Probe: 为缺项跑通约定级别的证据
写入证书 / 晋级	Certify: 节点进入 Γ
上游变更作废	Dirty: Γ 收缩, 假绿不得继续

命题 5.1 (工作流轨迹是自动机路径的投影). 算法 5.4 的每一步成功更新, 都对应第 3 章作业过程自动机上的有限转移序列片段: 展开 $\sim$ Expand, 补洞 $\sim$ Probe; Certify, 写回 $\sim$ Certify(T), 作废 $\sim$ Dirty, 冻结 $\sim$ Freeze. 因此「守门禁」不是额外口号, 而是要求 Agent 的轨迹落在合法转移集合内。

5.4 合法性规则 (v0) 清单

由定理 5.1 与算法 5.2, Agent 侧可收成对照清单:

只允许	明确禁止
先交完整 CT 再写码	跳过闭包表直接写 examples/探针
引用 $v \in \Gamma$ 且未脏、未踩 Forbidden	把 INDEX 绿名 / 局部测通 / frozen / correctness 当模板
缺项先 Probe/Certify 再引用	同任务顺手发明未获证 P0/P1 并立刻引用
新接缝单独认证	两块各绿就宣称拼装已合法
验收看 I/O $\leftrightarrow$ golden/KAT	以「和参考源码长得像」为验收
上游变更后跑 DirtyPropagate	用旧 PASS 假绿下游

5.5 文书分工 (由算法接口推出)

由算法 5.1 与 5.2 的输入输出, 立刻得到:

文书	在算法中的角色	不能冒充
闭包表 CT	GateCheck 的主输入	实现细节说明书
customspec	ALLOW 之后的实现规格文书	「可不跑 BuildClosureTable」
STATUS / 对拍	Certify 的证据	无视 DirtyPropagate 的执照
INDEX	Menu / ρ 的候选	L 的成员证书

5.6 反面教材：放飞=算法前置条件失败

第 3 章引子中的 Encaps/Decaps 放飞, 可用本章谓词改写:

现象	失败的算法条件
提示很轻、无闭包表	算法 5.1 步骤 1 拒绝；无 CT 则 GateCheck 必 BLOCK
菜单在仍无限试错	命题 4.5：未跑 MissingOf
只保 correctness、形态粗暴	用对拍冒充 PermitCode；接缝/工程形态未进 Seams 认证
错误经验沉淀	未调用 DirtyPropagate / Freeze，失败轨迹仍当可复用
token 耗尽	搜索未限制在 TaskWorkflow 的合法转移内

5.7 域无关性

算法 5.1–5.4 的参数是 $G, \Gamma, \text{Forbidden}$ 与转移名, 不含密码代数。换域只需重填节点卡与 Forbidden 内容；骨架不变。ML-KEM 仍是接缝多、对照清的压力测试床。

5.8 下一章预告

规则与算法齐备后，用 `kem.keygen/kem.encaps` 复盘：历史轨迹是否可写成算法 5.4 的实例；偏离记为欠债或 forbidden。再用 `kem.decaps` 前瞻：能否先输出一张真 CT。

5.9 本章小结：本章推出了什么

1. 算法 5.1 由 Dep^* 、 Missing_Γ 、 Forbidden 构造闭包表；定理 5.1 给出写码许可的 v_0 判定。
2. 算法 5.2–5.4 把自动机转移落成 Agent 工作流；命题 5.1 说明二者同轨。
3. 文书分工与引子反面，都是这些算法前置条件的推论，不再是平行经验列表。

6 复盘：理论如何对照 `kem.keygen` 与 `kem.encaps`

本章把第 3–5 章已经写下的对象，套回两个已经交付的故事：`kem.keygen` 与 `kem.encaps`。写法是事后补一张闭包表、标三类边、回答第 2 章四组问题——不是宣称「当年就按理论做的」，而是检验：理论语言能不能把做过的事说清楚，以及哪里说不清（欠债）。说不清的地方，留给第 7 章用 `Decaps` 再压。

6.1 复盘的目的与方法

目的

1. 检查分层（P0–P3）和接缝能不能用依赖图 / 闭包 / 禁止集说清楚；
2. 找出偏离（correctness vendor、frozen 模板、先写码后补文书）并记成欠债或 Forbidden；
3. 总结：数学对象落到了哪些目录、launch 边界与证书文书上。

方法（只用已有工具，不新发明记号） 对每个目标 T ，按算法 5.1 的栏位事后填一张 CT（此时 M 对已交付目标应为 $\emptyset$ ，否则说明我们连「已获证」叙事都对不齐）。边按算法 4.2 标 semantic（语义依赖）、compose（接缝）、forbidden（禁止）。节点卡字段见算法 4.1；仓库锚点只指向活跃 stable / pass-fix，不把 frozen 或 correctness 当正向 repo。

复盘资产（查阅用，非抄码清单）

角色	路径 / 文书
KEM KeyGen 交付	examples/stable/ml-kem/ml-kem-1024/stable-fips203-mlkem-kem-keygen-k4/
KeyGen 行为基线	ascendc-tests/ml-kem/ml-kem-1024/pass-fix-f203-alg19-kem-keygen-device-k4/
KeyGen registry	docs/specs/fips203-mlkem1024-kem-keygen-baseline-registry.md
PKE KeyGen（先决）	examples/stable/ml-kem/ml-kem-1024/stable-fips203-mlkem-pke-keygen-k4/
KEM Encaps 交付	examples/stable/ml-kem/ml-kem-1024/stable-fips203-mlkem-kem-encaps-k4/
Encaps 行为基线	ascendc-tests/ml-kem/ml-kem-1024/pass-fix-f203-alg20-kem-encaps-device-k4/
Encaps registry	docs/specs/fips203-mlkem1024-kem-encaps-baseline-registry.md
PKE Encrypt（先决）	examples/stable/ml-kem/ml-kem-1024/stable-fips203-mlkem-pke-encrypt-k4/
对照标本（禁抄）	ascendc-tests/fix-f203-alg19-kem-keygen-correctness-k4/; fix-f203-alg20-kem-encaps-correctness-k4/

6.2 事后闭包表：kem.keygen

目标 $T = \text{kem.keygen.k4}$ (FIPS 203 Alg.19, $\ell = \text{deliverable}$)。I/O 契约 $\mathcal{C}$ ：黑盒输入含种子侧约定与 LUT；输出 ek (1568 B)、 dk (3168 B)；验收为 CPU+SIM 对拍 (SIM tick 量级约 7.07×10^5)，必要时 liboqs KAT。

栏	事后填写（2026-07 交付态）
目标 $T, \mathcal{C}, \ell$	如上；级别 deliverable (stable + registry)
已获证 Γ 视角	pke.keygen.k4 (stable)；设备侧哈希/采 z 等已随 device/stable 写入证书叙事；相关 P1 (NTT 等) 在 PKE 获证时已进入闭包， 不在 KEM 任务重写
依赖闭包 D	PKE.KeyGen (Alg.13) + Alg.16 尾缝 ($H(ek)$ 、采 z 、拼 dk) + 2-launch 集成边界 (prep mmad+ 尾融合)
缺项 M	$\emptyset$ (相对当前 Γ ；否则 stable 不应标交付)
接缝 Seams	Alg.16 尾；双 launch 汇合 (SIM 用 SyncAll 等工程约束已写入 customspec landmines)
否定 ForbiddenHit	计划若指向 correctness 的 3-launch / vendor oracle，或 frozen 正向源码 $\Rightarrow$ 硬冲突；当前交付计划与之 不相交
警告 Warn	历史：接缝与 launch 预算曾后知后觉（见欠债）；证书时序曾是 device 先绿、文书后补

校准问题单（对照第 2 章）——事后答

- P3 是否只依赖已获证 P2、不在 KeyGen 任务重写 NTT？**是**：stable KEM KeyGen 站在 stable PKE KeyGen 能力之上；NTT 等 P1 不在本任务「顺便发明」。

- 2-launch、Alg.16 尾、*dk* 展开是否写清？**交付态已写清**（*customspec* / *STATUS*）；但历史上 *correctness* 为 3-launch，*device* 才收成 2-launch——属「接缝后知」，记欠债而非否认终态。
- *baseline-registry* 的 *golden* 块能否在图上找到 *lemma*？**基本能**：oracle 侧挂 *liboqs_kem_ref* 等已登记块；胶水循环不另冒充 P1 证书。
- 否定边是否落笔？**有**：Host 禁冒充生产 *H/z/dk*；禁把 PKE C/Python 当 AscendC 规格；交付禁链 *device/frozen/*；*correctness* 标本显式非模板。

6.3 事后闭包表：*kem.encaps*

目标 $T = \text{kem.encaps.k4}$ (Alg.20 / 内部 Alg.17, $\ell = \text{deliverable}$)。I/O 契约 $\mathcal{C}$ ：输入 ek_{KEM} 与消息 m (32 B, 作 Alg.17 输入)；输出密文 c (1568 B) 与共享密钥 K (32 B)；**不**交付中间 h/r /噪声等。验收：CPU+SIM 对拍 (SIM tick 约 7.21×10^5)；liboqs KAT 批测已做过。

栏	事后填写 (2026-07 交付态)
目标 $T, \mathcal{C}, \ell$	如上； <i>deliverable</i>
已获证视角	<i>pke.encrypt.k4</i> (stable)；设备 SHA3-256/512； m 由 Host/GM 提供 (交付语义)，与 <i>correctness</i> 「由 $seed_d$ 设备采 m 」 不同 ——后者不得混进 Γ 叙事
依赖闭包 D	PKE.Encrypt (Alg.14) + KEM 头 (H/G 得 $K r$) 并入 prep 前段 + 与 Encrypt 同构的 launch 预算 (SIM 2 / CPU 5；禁第 3 次独立 launch)
缺项 M	$\emptyset$ (交付态)
接缝 Seams	「PKE Encrypt 已证 $\nRightarrow$ KEM 头尾自动获证」；头并入 prep、launch 合并须单独认证过
否定 ForbiddenHit	不得以 <i>fix-...-encaps-correctness</i> 的 frozen G5 vendor Encrypt 为模板；不得把 <i>correctness</i> 的 m 语义当成交付契约
警告 Warn	探索史： <i>correctness</i> $\rightarrow$ <i>device</i> (挂 stable Encrypt) $\rightarrow$ <i>incubating vendored</i> $\rightarrow$ <i>stable</i> ；语义漂移与模板更换说明：若当年先交 CT，本可少走一截

与 PKE Encrypt 的关系 (一句话) 语义依赖边指向 stable Encrypt 的 *interface*；交付目录选择**自包含 vendored** 快照，*device* 探针则曾**编译期引用** stable——两种工程落点都合法，但必须写进闭包，不能只说「觉得自己复用了」。

6.4 三类边在仓库里的实例

标签 τ	实例（真实锚点；禁把对照目录当正向实现）
semantic	kem.keygen.k4 依赖 pke.keygen.k4 的 interface（ ek 同形、不在 KEM 任务重做 NTT）；kem.encaps.k4 依赖 pke.encrypt.k4 的 interface（Alg.14 主体）；KeyGen 产出的 ek 作为 Encaps 输入契约的一部分
compose	Alg.13 + Alg.16 尾 $\rightarrow$ Alg.19（尾嵌 mmad、2-launch）；Alg.14 + prep 内 $H/G \rightarrow$ Alg.20；「单点 PASS」推不出这些接缝自动 PASS——须单独进 Γ
forbidden	fix-...-keygen-correctness（3-launch / vendor oracle）不得作交付模板；fix-...-encaps-correctness（frozen G5 vendor； m 语义异于 Alg.17 交付）不得作模板；任意 frozen/ 正向源码路径不得写入节点卡 repo

6.5 具象化对照表（理论 $\mapsto$ 工程）

理论对象	在本仓库长什么样	证书 / 证据形态
节点 v	pass-fix-* / exp-* / stable-* 目录	STATUS + run.sh CPU+SIM；必要时 KAT
semantic 边	「必须先有某某 stable/interface」	INDEX 引用、customspec 依赖声明、registry 块名
接缝（compose）	多 launch / 头尾融合 / Alg.16 · 17 边界	集成探针或 stable 全链 PASS；landmines 写入规格
Forbidden	correctness 标本、frozen/ 判决、Rule 禁抄	FROZEN.md / STATUS「非模板」/ 交付禁 #include 条文
Γ / 脏标记	已交付且未作废的能力	上游改 tiling/同步后应 Dirty（机制仍偏手工）
字 $w \in L$ 、CT	一次任务的闭包表 + 计划	理想 ：先交表再写码； 史实 ：常后补（欠债）

6.6 回看第 2 章四组问题（复盘后的判断）

表 2.3 是专题讨论时的诚实对照。本章复盘之后，判断可以收紧为：

问题组	复盘后已能说清的	仍欠、须第 7 章压测的
第一组	可指出证书锚点（stable + registry + device）；菜单（INDEX） $\neq$ 拼装说明书——说明书 $\approx$ CT+ 三类边	机器可读证书对象仍未工具化；Agent 不会自动先交表
第二组	「组合不免费」可用 compose + Seams $\subseteq \Gamma$ 表述；KeyGen/Encaps 终态接缝可点名	验收口径仍以 I/O 为主；接缝获证与 launch 预算的统一检查未自动化
第三组	correctness 负例成立；正向规则（Forbidden、先 CT）已写进第 5 章	门禁尚未挡住真实 Agent 绕开；Encaps 史实仍是「探索后收敛」
第四组	KeyGen/Encaps 可以事后写成 CT；Dirty 概念已有	统一 schema 未进日常流程；Dirty 传播无自动机制；「先交表再写码」正例当时未落地（第 7 章已补）

6.7 分析结论

1. **理论够不够复述已完成故事？够。**依赖闭包、接缝、禁止集、节点卡字段，都能在 KeyGen/Encaps 的交付态找到对应物；不必引入密码代数也能讲完「凭什么站在 PKE 上」。
2. **史实不等于方法论已生效。**两条交付线都曾经历 correctness 或混乱探索；文书与 launch 约束大量后置。因此本章结论是「理论可解释」，不是「当时已按门禁派生」。
3. **欠债清单（供第 7 章对照）**
 - 闭包表未始终领先实现（device 先绿、规格后补常见）；
 - 接缝与 launch 预算后知（KeyGen $3 \rightarrow 2$ launch；Encaps m 语义漂移）；
 - Dirty 失效传播仍靠人记；
 - 门禁未工具化，Agent 仍可能绕开 Forbidden。
4. **对下一章的输入。**若方法论有效，Decaps 应能在写码前交出真 CT（含多父依赖与 FO 失败路径接缝），并在门禁下实现到约定验收级别；过程应可明显区别于 correctness 探索史（比较轴见第 9 章）。若做不到，应如实判「弱成功未达成」，并回写理论欠债——而不是事后贴标签。

6.8 本章小结

1. 为 kem.keygen、kem.encaps 各补了一张事后 CT； $M = \emptyset$ 与交付态一致。
2. 三类边均有仓库实例；correctness/frozen/ 归入 Forbidden，不作正向 repo。
3. 第 2 章四组问题：复述与负例已加强；「先交表再写码」的正例见第 7 章（已判决）。

7 前瞻：用理论指导 kem.decaps（试金石）

上一章我们是在事后用理论复述 KeyGen 和 Encaps。这一章反过来：先按理论把 Decaps 的闭包表写出来，再在门禁下写码、验收，最后拿证据判断——方法论到底有没有用。

为了说清楚，我们并排看三条路（后文简称 A / B / C）：

- **A：**当初那条「只求能跑对」的 correctness 探索（probe-...-decaps-correctness-k4；只许对照，不许抄码）；
- **B：**这一章按方法论做出来的路（目录带 -ct，避免和下面那条交付树重名）；
- **C：**你长期带着优化出来的交付树（stable-...-kem-decaps-k4，无 -ct；仓库脚本默认指这里；SIM 默认单 session）。

数字和细表在仓库里另有一份：docs/research/2026-07-24-Decaps-correctness 与 CT 五指标对照.md、qa/active_sim_regress_summary.md，以及 7 月 24、25 日的纪要。本章只把该讲清的故事和判断写进教材。

整章可以先看成一张总览图（图 7.1）：左边是已经写好的三件工具，右边是落到 Decaps 上的实践链。

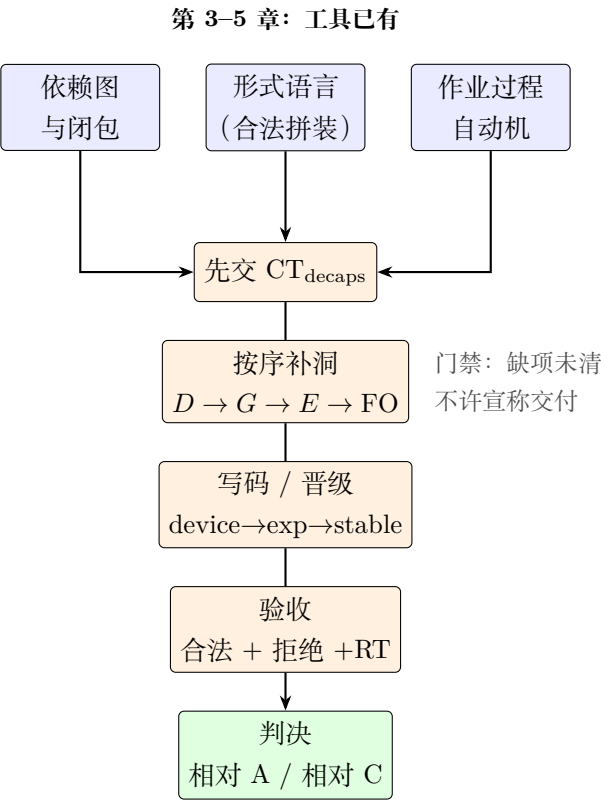


图 7.1：第 7 章总览：方法论工具如何落到 Decaps 实践

7.1 为什么拿 Decaps 当试金石

Decaps (Alg.21) 同时要 Decrypt、Encrypt，还要对付 FO 失败路径，依赖是「多头汇」——一张图上好几个父亲。上一章已经说明：理论能讲完已经做完的 KeyGen / Encaps。还差一刀：写码之前，理论能不能先把「该做什么、不该碰什么」列清楚，并真的走到能对拍的实现？列不出来、或者只能事后贴金，这一章就该老实写「没做成」，而不是硬说成功。

7.2 先把任务链理清：目标、依赖、缺什么

我们要交什么 目标记作 $T = \text{kem.decaps.k4}$ (FIPS 203 Alg.21，里头是 Alg.18)。动手时先按探针级别验收 (device 上 CPU+SIM 对拍)；同一天你授权 # 交付 # 之后，B 这条线才升到带 -ct 的 stable。输入输出约定如下：

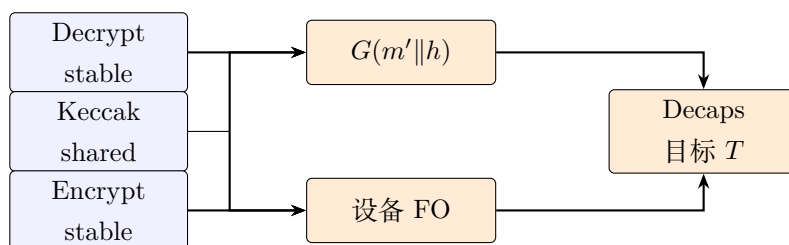
项	约定
输入	dk_{KEM} 3168 B (= $dk_{\text{PKE}} \ ek \ h \ z$)；密文 c 1568 B；Decrypt / Encrypt 各自的 LUT
输出	共享秘密 K 32 B (默认只写 $K.\text{bin}$)
合法路径	和同一次 Encaps 拿到的 K 字节一致
拒绝路径	重加密得到的 c' 对不上 c 时，只能出 $K = J(z \ c)$ ，不能把本该丢掉的 K' 漏出去 (Gate E3)
设备上必须做的事	m' 、 G 、重加密、 J 、最后选哪个 K ，都在 device 上完成；Host 不许用 memcmp 冒充 FO
SIM 怎么串 (B 线锁定)	默认 <code>decaps_2session</code> 。相对 C 线已经做到的单 session，这是 保底拆法 ，不是更优拆法——对照节再细说

往前追依赖（口语里的 Dep*） 从 Decaps 往回看，这些能力得齐（已经获证的不算缺项）：

1. stable 的 Decrypt (fused) ——Phase-D 解出 m' ；
2. stable 的 Encrypt——Phase-E 重加密得到 c' ；
3. 设备上算 $G(m' \| h) \rightarrow (K', r')$ ——头和 Encaps 对齐，Keccak 用 library/shared；
4. 设备上的 FO（比 c 和 c' 、算 J 、选出最终 K ）——这是接缝，**不是** Decrypt / Encrypt 各自测通就会自动附赠的；
5. 工程接缝：SIM 上合成一个设备库 (prepare_dec_shim)；以及 B 线选用的双 session 编排。

对拍要用的 dk 、 c 、 K ，可以由已经交付的 KeyGen / Encaps 提供；本任务**不再重写**那两条。

画成图更清楚（图 7.2）：只画**实现**要用的依赖——对拍用的 KeyGen/Encaps、禁止抄码等写在正文，不塞进这张图。箭头一律走折线，避免斜线交叉。



从左到右：已有模块 → 接缝 → 目标；箭头只走正交折线

图 7.2: Decaps 实现依赖（写码前从目标反推；仅实现相关）

缺什么，为什么不能一把写完 动手前明显还缺四块：device 探针本身、Phase-D 接线、Phase-E 接线、FO 尾巴。若图都懒得画、直接去抄 A 线的 vendor/，立刻撞上禁止项——这正是方法论相对「开放乱搜」的第一道闸。

7.3 写码前先交闭包表 CT_{decaps}

栏	写码前怎么填
目标与级别	同上；先按 device / lemma 验
已有的	Decrypt、Encrypt、KeyGen、Encaps；设备侧 SHA3 / SHAKE
依赖闭包	Decrypt $\rightarrow G \rightarrow$ Encrypt $\rightarrow$ FO；再加上合库和双 session 编排
还缺的	device 本体；D/E 两段接线；FO 尾
接缝	Decrypt 接 G ； G 接 Encrypt；FO；SIM 双 session
禁止碰到	correctness 里的 frozen G4/G5 vendor/；任何 frozen/ 正向源码；Host 代算 J 或 memcmp 冒充 FO
警告	单 session 的真修不塞进本轮验收；B 线先用双 session 保底

什么时候才许说「交付了」 按定理 5.1：缺项没清零之前，**不许**宣称交付或 stable 晋级。允许做的事只有一件：按下面的补洞顺序把缺项补齐并验过。（同一天你另外下了 # 交付 #，才打开 examples/...-ct-k4 的晋级。）

7.4 三种边，以及我们实际补洞的顺序

边的类型	落在 Decaps 上长什么样
语义依赖	要用 Decrypt / Encrypt 的对外约定； <i>dk</i> 怎么切依赖 KeyGen；对拍依赖 Encaps 的契约
接缝	Decrypt 测通了， $\neq G$ + 重加密 +FO 自动测通；全链要单独验
禁止	correctness 的 vendor/ 、 frozen/ ；Host 冒充 FO

补洞顺序（跟算法 5.3 同一套想法，按我们实际执行写）：

1. Phase-D：编译期挂上 stable Decrypt，先跑通 m' ；
2. 设备上算 G ，得到 (K', r') ；
3. Phase-E：编译期挂上 stable Encrypt，喂进 m'/r' ，得到 c' ；
4. FO 全链：合成单库；设备上比对并选出 K ；SIM 默认双 session；合法路径 K 对拍为 0；拒绝路径得到 $J(z||c)$ 。

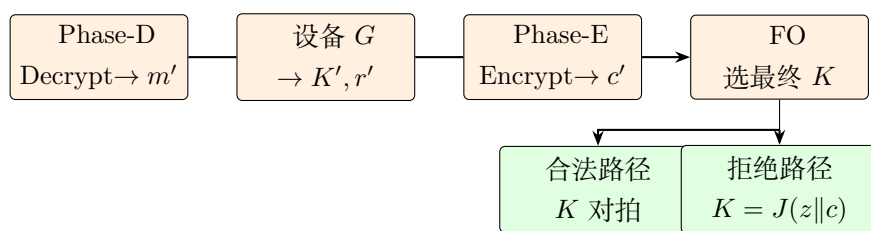


图 7.3：最小补洞顺序（B 线实际执行；末级分叉为合法 / 拒绝）

图 7.3 要传达的不是「四步串行」本身（正文枚举已够），而是补洞之后必须同时覆盖合法与拒绝两条验收——末级折线分叉才是这张图的信息。

7.5 文书和代码是怎么一步步落下来的

时间顺序大致如下（细证据在 git 和 qa/）：

1. 先把 `CTdecaps` 写进教材并提交——早于 device 源码；
2. 写 `INTEGRATION_PLAN.md` / `STATUS.md`，把禁止项、合库、SIM 默认、验收命令钉死；
3. 做出 `pass-probe-...-decaps-device-ct-k4`（kem/ 只管头尾；PKE 编译期引用 stable，没有 vendor/）；
4. 你下令 # 交付 # 之后：先 `exp-...-decaps-ct-k4`，再整树复制成 `stable-...-decaps-ct-k4`；
5. 收尾：拒绝路径 SIM、KAT、roundtrip；修好 `M_FILE` 和「对拍失败仍打印成功」；补中文注释；清幽灵引用；目录统一加 `-ct`；
6. 对照实验：给已有的 `stable_kem_liboqs_roundtrip.sh` 加上 `DECAPS_DIR` 和多次 trial，让 B、C 两条线都能跑「先 liboqs、再 AscendC」。

B 线代码允许从哪来：stable 的 Decrypt / Encrypt；Encaps device 的接线习惯；shared Keccak；FIPS 203 和 Alg.21 相关笔记。**明确不从哪来：**correctness / **frozen/** 里的正向实现。

单线时间轴画成流程图没有额外信息；改用**概念分层目录**（图 7.4）表达「先表后码、再分层落地」——名字是示意，**不是**仓库真路径，也不锁定日后目录拼法。

```

decaps-method-line/
  01-closure-table/      ← 最先落：闭包表 / 教材 CT
  02-plan-docs/          计划与状态文书
  03-device-probe/       设备探针（先于研究副本）
  04-research-copy/      研究副本（incubating 角色）
  05-release-copy/       定型副本（stable 角色）
  bench-compare/         压测与对照记录

```

示意分层，非仓库真路径；只读先后与角色，不读目录字面量

图 7.4: B 线落地物的概念分层（示意目录，非真路径）

图 7.4 要看的只有两件事：**闭包表层在源码层之上**；探针 → 研究副本 → 定型副本是角色分层，不是又一条可执行流水线。

7.6 测了什么，结果怎么读

B 线验收（Cloud）

测什么	结果
device 合法 CPU / SIM	K 对拍为 0；SIM 上 D286798+E763663；用例根目录无 stray dump
拒绝路径 CPU / SIM	$K = J(z c)$ 通过（device / exp / stable 都做过）
liboqs 分项 KAT	CPU 10 次 + SIM 3 次通过（合法路径要带上 <code>M_FILE</code> ；不要并行多路 SIM）
设备闭环 roundtrip	CPU、SIM 都过（先 agreement，再 E3 拒绝）
和 liboqs 同步的 roundtrip	每一轮都先抽 liboqs 随机向量，再喂 AscendC；B 线 CPU 5 次 + SIM 1 次全过（2026-07-25）
NPU	这台机器没卡，没跑（C 线同样缺）

假绿：必须老实说 方法论**没有**把假绿消灭掉。它只是把「整条路线乱飘」收成「门禁旁边的缝」：

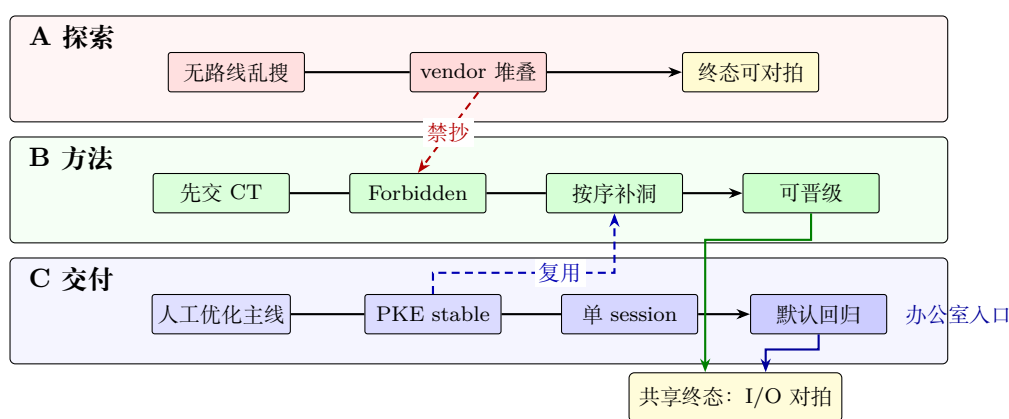
- CPU 孪生跑 Phase-E 时，若缺了和密文匹配的 m ，会误走拒绝——批测必须带 `M_FILE`；
- `run.sh` 曾在对拍失败后仍打印 [SUCCESS]——要对拍命令失败就非零退出；
- 同一个目录上并行多路 SIM，可能把结果搅脏。

五指标里，B 线假绿记了 **3** 次，和 A 线下界「至少 3」是一个量级。所以判决里要写死一句：**先交闭包表，不等于验收脚本没有洞。**

SIM tick (Total tick, 不是 msprof) B 线 stable 合法合计大约 **1050646** (D286866+E763780); device / exp 也在 1.05×10^6 附近; 拒绝路径同一量级。表在 `qa/active_sim_regress_summary.md` (2026-07-24 刷新)。

7.7 和人工优化出来的交付树 (C) 比一比

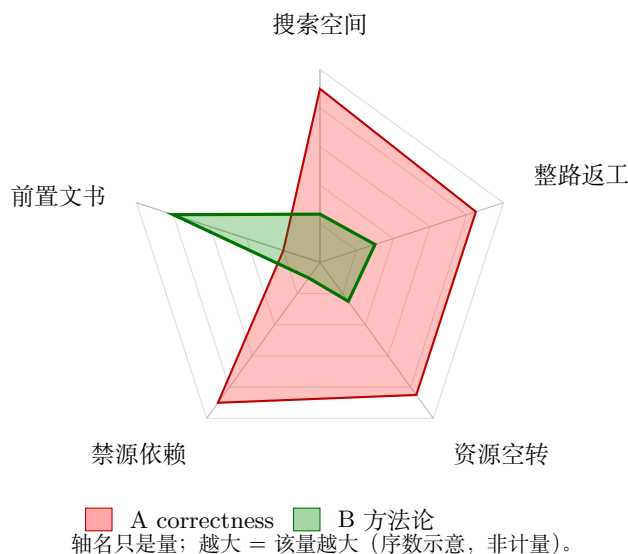
C 线是交付默认: T19i 之后 SIM 三个 launch, 默认单 session, 办公室脚本 `stable_kem_liboqs_roundtrip` 也指它。B 和 C 在核上的拆法其实一样——Decrypt 一段、Encrypt 一段, G 进 prep, FO 挂在 pack / 118_119 尾。差在**怎么走过来**、SIM 默认怎么设、文书谁先谁后。三条路若各自画成单线串行, 图并不比文字多说什么; 图 7.5 要传达的是**泳道之间的关系**: 谁禁止抄谁、谁复用谁、谁共享终态、谁当默认回归。过程开销形态与 tick 仍见图 7.6、图 7.7; 门禁二元五指标见对照表, 不挤进雷达。



虚线 = 跨泳道关系; 实线 = 泳道内阶段。

不要把 A/B/C 读成同一条优化曲线上的三个点。

图 7.5: A/B/C 泳道对照: 禁抄、复用与共享终态 (不是三条互不相干的串行)



轴名只是量; 越大 = 该量越大 (序数示意, 非计量)。

本图是**开销雷达**: 面积大 = 负担更重。A 大半圈; B 收小, 只在「前置文书」凸出。

门禁二元项仍见对照表, 不画进本图。

图 7.6: A/B 过程开销形态 (雷达示意): 量名中性; 面积大表示负担重

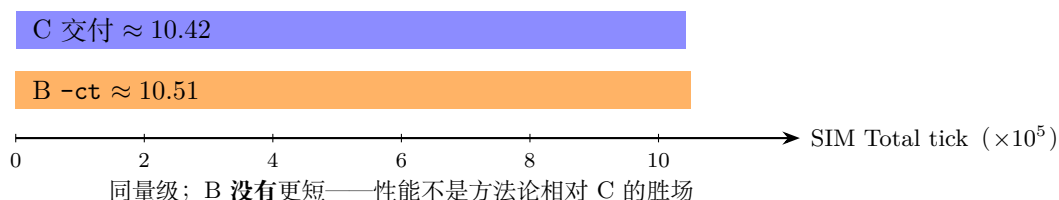


图 7.7: 合法路径 SIM tick: B 与 C 打平（量级对照）

怎么读这些图

1. **正确性**: 图 7.5 里 B、C 都折入「共享终态: I/O 对拍」——方法论没有变出「更正确」的密码学结果。
2. **性能**: 图 7.7 里两条棒几乎一样长——B 没有跑赢 C; 双 session 相对单 session 是保底, 不是优化胜利。
3. **过程**: 先看图 7.5 的跨泳道虚线 (A→B 禁抄、C→B 复用), 再看图 7.6——这是开销雷达 (面积大 = 负担重): A 在搜索/返工/空转/禁源上铺开, B 收小、只在前置文书上凸出。相对 C 赢的仍是轨迹说得清, 不是面积神话。
4. **顺便修到的洞**: B 线上的假绿回灌给 C 有用 (verify 必须失败退出), 这是实验带给主线的好处, 不是把核从 B 搬到 C。

细项对照表 (需要查数字时再用):

项	B (-ct)	C (交付)
SIM 默认	双 session	单 session (T2)
合法 SIM tick	≈1050646	登记 1041906
脚本默认	否	是

7.8 怎样算成功, 本章怎么判

本章自己的尺子

- **弱成功**: 闭包表能执行; 偏差说得清; device 合法路径 CPU+SIM 过;
- **强成功**: 再加上拒绝路径 (至少 CPU、SIM 都有据); 相对 A, 过程更干净、结构不靠 vendor;
- **没做成**: 在禁止项卡住、交不出可对拍实现。

「SIM tick 比 C 短」或「默认单 session」不写进强成功——那是另一场优化比赛, 不是这一章的试金石。

判决 相对 A: **强成功**。「能不能先交表再写码」: **能**, 这一章就是正例。「方法论是不是已经取代人工优化主线」: **没有**——脚本默认和 SIM 定论仍在 C; B 是对照实验副本 (-ct)。

回扣第 2 章那四组问题

组	这一章之后怎么说
第一组	多头依赖和 FO 接缝可以写进闭包表；缺项列得出来，也能按序补
第二组	验收仍主要看输入输出；接缝和 launch 还没有自动检查——假绿还得人抓
第三组	B 线上禁止抄码真的拦住了；门禁还没做成工具，脚本缝还在
第四组	「先交表再写码」有了正例；证书变脏之后怎么自动作废，仍欠

7.9 方法论到底有没有用：说人话的总结

1. 有用的地方（相对 A）：闭包表把「该挂谁、别抄谁、先补哪几块」变成可执行清单；在禁止项之下，Agent 走出了没有 vendor、还能晋级的实现；拒绝路径一开始就进验收，而不是事后补丁。五指标里四个二元项 4 比 1，把这道门禁差量化了。
2. 还没证明、也不该吹的地方：假绿还在；没有自动赢过 C 的性能或单 session 定标；仍然要靠人下 # 交付 #、确认锁参；两边都没有认真对过 token，别编效率神话。
3. 理论账本怎么改：上一章欠的「Decaps 还没证明先交表」——可以划掉；「门禁没工具化、脏证书靠人记」——还挂着，假绿三例就是证据。
4. 工程上怎么用：日常回归继续默认 C；B 留给教材和对照；文书和脚本门禁可以继续从 B 回灌 C，别把双 session 默认倒灌回 C。

7.10 本章小结

1. 写码前交了 CT_{decaps}，按补洞顺序做完 B 线的 device，再晋级到带 -ct 的 incubating / stable（总览见图 7.1，落地分层见图 7.4）。
2. 该跑的测试都绿了；假绿三例说得清。
3. 相对 A：强成功（图 7.5 的禁抄/过程、图 7.6）。相对 C：对得一样齐，tick 不更短（图 7.7），过程更好讲。
4. 一句话：先交表这道门有用；它代替不了人工优化主线，也堵不死验收脚本上的洞。

8 套别例：ML-KEM-768 从分析到 incubating 的完整复盘

第 2-7 章讲的是 ML-KEM-1024 ($k = 4$) 这条主链：先有特例，再抽理论，再用 Decaps 当试金石。这一章换一块石头：真做 ML-KEM-768 ($k = 3$)，从分析、计划、实现与测试、改错，一直写到结果与判决。

读者带走三句话就够：

1. 768 不是「把 1024 的 k 改成 3」；奇数维会逼出一整套分核与 I/O 重锁。
2. 有效路径是先锁参数卡与波次，再积木 →device→incubating→ 胶水；禁零垫、禁抄 frozen/。
3. 判决是有条件完成至 incubating：CPU+SIM 与 AscendC-only roundtrip 绿；本阶段不建 stable-768，也不宣称 liboqs-768 交叉已齐。

仓库锚点（查数字与 STATUS 时用，不必背路径）：参数卡 docs/specs/fips203-mlkem768-parameter-card.md；计划 docs/research/MLKEM-768- 0 exp .md；探针树 ascendc-tests/ml-kem/ml-kem-768/；incubating examples/incubating/ml-kem/ml-kem-768/；当日收尾 qa/2026-07/2026-07-27-768 .md。细 tick 见 qa/active_sim_regress_summary.md。

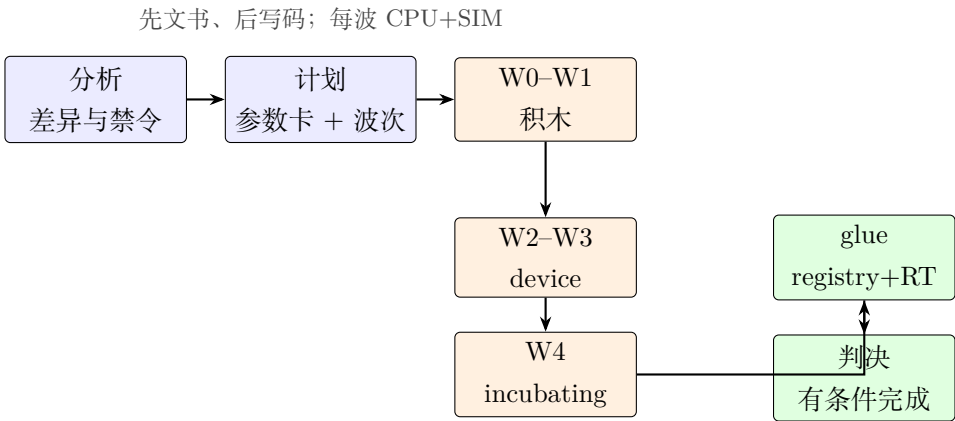


图 8.1：第 8 章总览：768 不是改参冒充，而是「锁卡 → 波次 → 绿线 → 判决」

8.1 为什么拿 768 当第二块试金石

第 7 章的压力主要在**接缝**：Decrypt、Encrypt、FO 多父汇合，先交闭包表再写码。768 的压力换到另一处：**参数组迁移**——同一套算法叙事，换一组 (k, d_u, d_v) 与字节契约，方法论还认不认得「哪些能继承、哪些必须重做」？

若允许「末 poly 置零凑成 4/8」，工程上往往能先绿一把，但语义已经不是真 768，而且会把日后真正奇数维布局的坑藏起来。用户锁定的前提是：**禁止零垫**；后缀用 -k3；本阶段终点停在 incubating $(PKE \times 3 + KEM \times 3 + decaps-ct)$ ，**不建 stable-768**。

8.2 怎么分析：相对 1024，洞在哪里

先把 FIPS 203 与 liboqs 宏对拍清楚（长度已用脚本核过）：

项	ML-KEM-768	ML-KEM-1024	工程含义
k	3 （奇数）	4	分核难均分；禁 pad
(d_u, d_v)	10,4	11,5	Compress/pack 重做
$ ek / dk_{KEM} / c $	1184 / 2400 / 1088	1568 / 3168 / 1568	一切 I/O 重锁
$\hat{A}$	3×3 (9 poly)	4×4 (16)	prep 分片重推
噪声批	polyvec 6 ($s e$)	polyvec8	不能复用 8 路常量

再画「可继承 vs 必须重做」（只记策略，不抄 1024 源码当规格）：

能力块	策略	为何
SHAKE / Keccak 设备	继承 API	改长度/次数即可
NTT 数学契约	继承数学	设备布局按 $k = 3$ 重做
polyvec8 / k4 内积几何	不作实现模板	只借不变量叙述
Compress/ByteEncode	树内自建 (C-1)	主路径 $d \in \{10, 4\}$
FO / G/H/J	继承算法	stash 与密文长重锁
CPU/SIM 工程壳	继承	camodel、超时、run.sh 习惯

分析阶段的硬结论只有一条：形状常数不能从 k4 默默搬过来。遇阻就停、回参数卡；不许为了编译过而改已锁的 M/N /分核。

8.3 怎么计划：参数卡、决议与波次

计划拆成四段（文书先行，写码后置）：

P	W	产出	退出
P0	—	参数卡 + CT 长度 + 分核选项 + 目录壳 + registry 骨架	用户锁决议
P1	—	补洞图 + 必建用例表（含 PKE exp 与 CT）	表定稿
P2	W0–W3	探针全绿	每波 CPU+SIM_DIRECT=1 sim
P3	W4	incubating exp 全绿	有条件完成（本阶段）
P3	glue	registry + roundtrip	AscendC-only RT

用户决议（2026-07-26，写进参数卡）

- 1. 分核主路线 **T-B**：噪声侧 polyvec6； $\hat{A}$ **独立 prep**；拒绝 pad-to-4/8。
- 2. **保留** KEM device D19–D21；**要求** reject/CT（D21ct 与 E21ct）。
- 3. **PKE exp 必做**（E13–E15），不只做 KEM。
- 4. 命名后缀 -k3；本阶段**不建** stable-768。

数值 tiling 后来也锁进卡里 例如：InnerProduct AIV **2+1**（不是 $P/2$ ）； $\hat{A}[9]$ prep 分片 **5+4**；Encrypt INTT **batch4**(真 4, 不垫成 8);D21 默认 decaps_1session,D21ct 默认 decaps_2session。这些数字一旦入卡，实现阶段**禁止**擅自改参绕过。

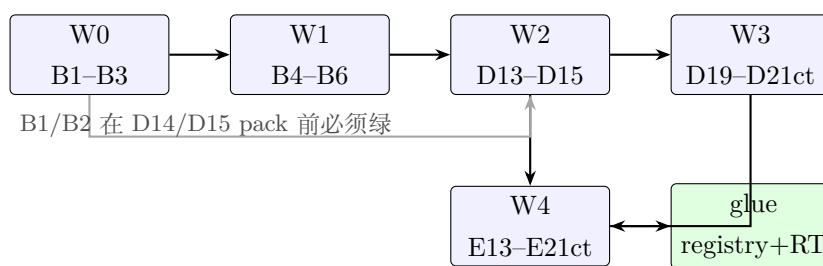


图 8.2: 波次依赖 (有汇合): 积木喂 PKE, PKE+KEM device 再汇入 incubating

Skill 节奏与仓规一致: 探针可走预研; 进 examples/incubating/ 写码前须有活跃 *-customspec.*; 本阶段不触发 # 交付 #。

8.4 怎么实现：按波次落码

每一波固定同一条流水 (门禁, 不是口号):

分析形状/同步/I/O → 写 INTEGRATION_PLAN 或 customspec → 实现 → bash
 run.sh -r cpu -v Ascend910B4 → SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
 → 打勾 STATUS + 当日 qa/ 追加。

合法代码来源: 活跃 1024 树的模式 (launch 编排、自包含习惯、注释密度) + 参数卡几何; **禁止**从 **/frozen/** 抄实现或把 frozen customspec 当依据。1024 迁入 ml-kem/ml-kem-1024/ 之后, 768 落在并列的 ml-kem/ml-kem-768/, 避免在 1024 目录里改 k 冒充。

W0 (编码压缩) B1 Compress/Decompress $d_u=10, d_v=4$; B2 ByteEncode/Decode (含 $d=12$ 密钥域); B3 CBD $\eta=2$ 的 polyvec6 打包。这一波主要证明「768 树能挂住定点与长度」, 算法大多可继承。

W1 (采样与 NTT, 最高风险) B4 SampleNTT 覆盖 $(i, j) \in \{0, 1, 2\}^2$; B5 Stage1-3 真 polyvec6 (S1-S3 禁 Gather); B6 MultiplyNTTs + InnerProduct ($P=3$, AIV **2+1**)。这里最容易出现「先 pad 成 4 再说」的诱惑——计划里把它写成**停问条件**: 一冒头就停, 回参数卡, 不偷偷改。

W2 (PKE device) D13 KeyGen (2 launch: prep $\hat{A}$ 5+4 + compute polyvec6); D14 Encrypt (2 launch; INTT batch4; d_{10}/d_4 pack; $|c|=1088$); D15 Decrypt (fused 1 launch)。从活跃 k4 device **复制结构再改几何**, 不是 fork frozen。一例真实返工: prep 队列按 PRF-Y 分配的 UB, 复用给 $\hat{a}$ 时越界——扩大队列容量, 不改 GM 锁定形状。

W3 (KEM device + CT) D19 在 D13 上接 Alg.16 尾; D20 接 D14; D21 接 D15 + 重加密 + FO (交付默认单 session); D21ct 走双 session, 专门压拒绝路径。正确性口径: accept 与 Encaps 的 K 字节一致; reject 必须是 $J(z||c)$, 且与 accept 不等。

W4 (incubating) 先 customspec (含 PDF), 再自包含复制 device→exp-*. 顺序: E13-E15 (PKE) → E19-E21ct (KEM)。每个 exp 目录实体化 compute//scripts/ 等, 避免运行时软链指回探针; vendor_sync.sh 只做本树存在性检查, 不从 k4/frozen 覆盖。

glue 六份 fips203-mlkem768-*-baseline-registry.md 按绿线补登记（诚实标未跑项）；新增 scripts/exp_kem768_liboqs_roundtrip.sh：默认 E19→E20→E21，并抽检 reject。脚本名带 liboqs，当前实现是 AscendC-only——仓内 helper 仍按 1024 尺寸写死；名实分离写进脚本头与纪要，避免假绿「交叉」。

8.5 怎么测，以及测挂了怎么改

验收矩阵（最小）

对象	CPU	SIM_DIRECT	备注
B1-B6	✓	✓	B5/B6 须有 ref
D13-D15	✓	✓	I/O golden; PKE RT 脚本可选后补
D19-D21ct	✓	✓	ct: reject≠accept
E13-E21ct	✓	✓	须活跃 customspec
roundtrip	✓	✓	AscendC-only; 含 reject 抽检

声称通过前：同用例目录双模式都绿；用例根无 stray dump；默认即全量路径（不要求用户手搓一串「与 default 相同」的 export）。

典型坑与改法（过程证据，不是荣誉墙）

- 1. 嵌套路径：1024/768 迁入 ml-kem/ 后，encode12 等脚本向上找共享件的层数错了——按新树修正 walk-up。
- 2. derand 字符残留：注释改成 k3 后，设备侧标志字符仍停在 '4'——改回 '3'，并对拍重跑。
- 3. host/device 双用符号：把 dual-use 的 GetPRange 放进 host 头导致 SIM 链接炸——改为 kernel Init 内联。
- 4. 奇数行均分幻觉：Inner 若写成 $P/2$ 会 silently 错分；卡上写死 $2+1$ 。
- 5. INTT 假 batch8：Encrypt 侧禁止「垫成 8 再跑 k4 路径」；真 batch4。
- 6. exp SIM 缺本地 multiply include：MULTIPLY_DIR 未传入 E20 CMake——补本地头路径。
- 7. roundtrip 名实：保留计划里的脚本名，但正文与脚本头写清「非 liboqs 交叉」，避免验收口径漂移。

这些修改有一个共同点：修工程接缝或实现 bug，不改已锁语义参数。参数有歧义就停；这和第 5 章「缺项未清不许宣称交付」是同一类纪律。

8.6 结果如何

完成口径

层	状态
探针 W0-W3	CPU + SIM_DIRECT=1 sim 全绿 (含 D21ct accept/reject)
incubating W4	E13-E15、E19-E21ct: 均有 customspec; CPU+SIM 全绿
glue	registry 补绿; AscendC-only roundtrip CPU×1+SIM×1 PASS
stable-768	未建 (须日后 # 交付 #)
liboqs-768 交叉 / NPU	未做 (挂 T768-post, 非本阶段门禁)

正确性 (黑盒 I/O) 各用例对拍 max = 0; CT reject = $J(z||c)$ 且 $\neq$ accept; KEM 闭环 (Key-Gen→Encaps→Decaps) 在 AscendC 树上绿。**没有宣称**: 与参考 C 源码同构、或已通过 liboqs-768 交叉 KAT。

性能 (SIM Total tick, Cloud / Ascend910B4, 非 msprof) 作登记对照, 不是优化 SLA。

能力	768 ($k=3$)	1024 ($k=4$) 量级	读法
D13 KeyGen	373426	~54 万 (stable)	维数更小, tick 更低
D14 Encrypt	507605	~63 万	同左
D15 Decrypt	222032	~28 万	同左
D19 / D20	510775 / 592129	~70 万 / ~72 万	同左
D21 / D21ct	818285 / ~82-83 万	~104 万量级	同左

相对同结构 k4 device, 768 大约低 **18%–31%**, 方向「活少了」的预期; E* 与对应 D* 同量级 (incubating 未另开性能叙事)。

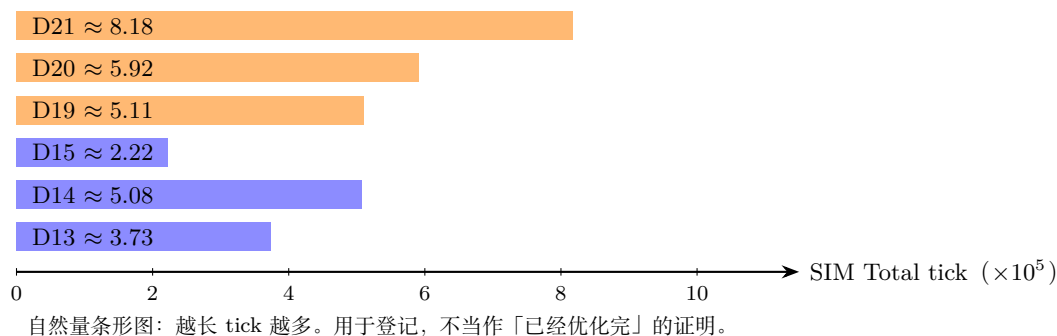


图 8.3: 768 device 代表 SIM tick (Cloud 登记值)

8.7 完整复盘: 什么有效, 什么仍欠

有效路径

1. **先锁卡再写码**: T-B、I/O 长度、§3.1–§3.3 tiling、CT/PKE exp 范围一次写清, 后面波次少扯皮。
2. **波次门禁**: 积木 → PKE → KEM → exp → glue; 每波双绿才往下走 (用户可按波授权, 也可一并授权剩余波次)。
3. **奇数维写死**: 2+1、polyvec6、batch4、 $\hat{A}[9]$ 分片——把「均分幻觉」挡在卡外。
4. **活跃树复制 + 自包含**: 只借 1024 模式, 不碰 frozen; exp 实体化目录, 避免软链漂移。

和 Decaps 试金石（第 7 章）比，同与不同

	第 7 章 Decaps	第 8 章 768
压力点	多父接缝 + FO/拒绝	参数组迁移 + 奇数维分核
先交什么	CT _{decaps}	参数卡 + P1 必建表
对照臂	A correctness / C 交付	1024 模式（禁抄 frozen）
终点	可到 stable（另线）	刻意停在 incubating
方法论用处	先表后码压缩搜索	先卡后波次挡住零垫捷径

仍欠（诚实列出）

- liboqs-768 helper / device 交叉 KAT；D14↔D15 独立 PKE roundtrip 脚本；
- stable-768（须用户 # 交付 #）；
- NPU 真机；把「奇数 k 分核」升格进 docs/notes/ 定稿（过程仍留 research）。

一句话判决 有条件完成（至 incubating）：真 $k=3$ 从参数卡走到探针与 incubating，并闭合 AscendC-only KEM roundtrip；先锁卡与禁零垫这套门禁**有用**。它**不**代替 stable 晋级，也**不**证明 liboqs 交叉或 NPU；脚本名里的 liboqs 字样，目前还不能当交叉证据读。

8.8 题外话：配额与墙钟上的体感

这一节**不是**对照实验，也不冒充已对账的 token 计量。它只把维护者当时的**直接感受**写下来——和第 9 章保留 correctness 标本的理由连着读：开放搜索有多贵，门禁把作业面收小之后，体感上能便宜多少。

对照的两端（同一类「让 Agent 把密码算子做出来」，不是同一次任务）

- **以前：**公司 Agent 与家里 Agent 各自去做 Encaps 的 correctness 路线。两边大约各用掉一天墙钟；合计直接吃掉当月 Cursor Pro+ 订阅配额的大约三分之二。过程形态接近第 7 章臂 A：没有先交闭包、没有禁源纪律、没有波次停问——主要在开放文本空间里搜「怎样才能对」。
- **昨晚（768）：**从计划到测试完成（参数卡 → W0-W4+glue），配额使用大约从 41% 走到 55%（增量约十四个百分点）；Agent 墙钟**没有超过四小时**；代码跑到 incubating，效果「还不错」（本章正文的绿线与 tick，不是这句形容词）。

怎么读这段体感

1. 差的不是「Agent 突然变聪明」，而是作业面被收成了：先锁参数卡、禁零垫、按波双绿、改接缝不改锁参。
2. 省下的主要是**无效派生**（乱改形状、抄冻结树、为编译过而改锁参、名实不符的假绿），不是魔法提速，也代替不了人工优化主线。
3. 数字来自订阅面板与当场印象：**序数级体感**，未做同任务、同模型、同提示的严格复现对账。因此本章**不**据此宣称「快 N 倍 / 省 N 倍 token」——只承认：门禁开着时，维护者觉得「值了」；门禁关着时，配额会以天为单位被搜索烧掉。

一句话：**体感本身也是证据的一种**——它证明「先锁边界再写码」值得付前置文书的成本；细账仍以 git、qa/、STATUS 与对拍为准。

8.9 本章小结

1. 768 复盘证明：方法论下一刀可以是「换参数组」，不必只围着同一条 1024 Decaps 接缝打转。
2. 分析 → 计划 → 波次实现 → 双模式验收 → 改接缝不改锁参——这条链跑通了（图 8.1、8.2）。
3. 结果：W0-W4+glue 绿；tick 见图 8.3；稳定与交叉仍明示未做。
4. 回扣第 5 章：缺项与锁参歧义就停；假绿（名实不符的 roundtrip）要写进文书，不能靠文件名装扮。
5. 题外话 (§8.8)：correctness 式开放搜索曾以「天 + 月配额大头」计价；768 有门禁的一夜大约只动了十四个百分点配额、墙钟不到四小时——体感证据，不作倍数宣称。

9 对照组：correctness 标本的意义

9.1 为何保留 correctness

-correctness- 在**完全不考虑优化**的前提下，只追求能跑、结果对：多 kernel launch、多搬运与同步——这是「一定能搭出正确积木」的保守拼法，性能谈不上，后续 P0/P1 的工程优化也几乎接不进去。

它还记下了一段真实的 Agent 过程：故意不给确定的拼装路线时，探索成本极高（失败、卡死、回退；短时间内曾烧掉大量 token），最后只得到「结果正确」的代码。

9.2 在方法论里扮演什么角色

角色	含义
正确性锚	下界存在性：积木搭得出来，I/O 也对得上
结构反例	展示「只求对」会长成怎样的 launch/同步膨胀
过程对照	没有确定路线时，搜索要付多大代价
实验基线	和方法论 Agent、人工指导路径三方比较

纪律：correctness 是**基线标本**，不是活跃实现模板；可读、可对拍、可对比，**禁止**当抄码来源（和 frozen「出门不带码」同一条规矩）。

9.3 建议比较的几条轴

1. **过程**：token / 轮次 / 回退；有没有先交闭包表；非法边有没有被拦住；
2. **结构**：launch 数、同步点、lemma 复用、接缝是否写清楚；
3. **结果**：CPU+SIM；SIM tick 相对 correctness 是几倍。

三档：A 旧式无路线探索 $\sim$ correctness；B 新方法论下的 Agent（本章 -ct 树）；C 人工指导的交付 device/stable（无 -ct）。第 7 章已给出一轮证据：B 明显好过 A（强成功），且**不必**打赢 C 的性能/1-session 定标；五指标精简表见 docs/research/2026-07-24-Decaps-correctness 与 CT 五指标对照.md。

10 未决议事项、日程与维护

10.1 刻意先不拍板的事

- 节点要不要和目录名 1:1 对应；图存在 Markdown 表里，还是 YAML/JSON 里；
- 主叙事选工作流网、Petri 网、属性文法、判断式里的哪一种；
- 什么时候写进 Rule/Skill；可判定性和复杂度要不要单开一条理论线。

10.2 近期填空顺序（仍然是「先特殊、后一般」）

1. 已完成（第 6 章）：KeyGen / Encaps 事后 CT、三类边实例、四组问题复盘结论与欠债清单；
2. 已完成（第 7 章）：Decaps 前瞻 CT $\rightarrow$ 实现/测试 $\rightarrow$ 与 A/C 臂对照 $\rightarrow$ 方法论效果判决（强成功相对 A；不取代 C）；
3. 已完成（第 8 章）：ML-KEM-768 分析 $\rightarrow$ 计划 $\rightarrow$ W0-W4+glue $\rightarrow$ 有条件完成至 incubating（禁零垫；AscendC-only RT）；
4. 按第 7/8 章反馈，必要时回改第 3-5 章定义（只记欠债；假绿/工具化门禁仍欠）；
5. correctness 对照实验写成可执行协议（与第 9 章及五指标表对齐；细表可续）；
6. 768 特有结论（奇数 k 分核等）稳定后，再考虑迁到 docs/notes/。

10.3 文档与分支约定

- 本文件是本专题**唯一**的 TeX/PDF 工作副本（不加日期后缀），持续修订；
- 讨论纪要：形式语言与自动机预研讨论纪要.md（同样单文件刷新）；
- 工作分支：research/formal-lang-dag；未经用户明确指令，**不合入 main**；
- 不记录题外的知识产权保护策略讨论。

10.4 编译

```
bash scripts/xelatex-clean.sh \
  docs/research/从已验证能力到合法派生-面向Agent预研的形式方法教材草案.tex
```